

Veilbox

Build Guide & Technical Reference

Version 1.0 — June 2026

Built on Fedora 44 · Linux 7.1.0-rc6

QEMU 10.2.2 · GRUB 2.x

Containerd 1.7.13 · Dropbear 2024.85 · BusyBox 1.35.0

Comprehensive Technical Reference — 300+ Pages

Contents

Chapter 1

Overview

Veilbox is a minimal, bootable operating system built entirely from source using a custom-compiled Linux kernel with an embedded initramfs (initial RAM filesystem). It boots directly to a BusyBox shell with containerd, runc, nerdctl, and Dropbear SSH running out of the box. The entire operating system is packaged as a single disk image — 8 gigabytes raw or approximately 95 megabytes as a sparse VDI — that can be imported into any virtual machine manager or written directly to bare metal hardware.

1.1 The Problem Veilbox Solves

Building a Linux system from scratch has traditionally been a complex, time-consuming process requiring deep expertise in kernel compilation, initramfs generation, bootloader configuration, and userspace assembly. Most solutions fall into three categories:

1. **General-purpose distributions** (Fedora, Ubuntu, Debian): Large, complex, with hundreds of packages and services. Installing a minimal container host requires post-install configuration, package removal, and security hardening.
2. **Embedded build systems** (Buildroot, Yocto): Powerful but complex. Buildroot requires learning its Kconfig-based configuration system. Yocto requires understanding BitBake recipes, layers, and metadata. Both produce excellent results but have steep learning curves and long initial build times.
3. **Minimal distributions** (Alpine Linux): Lightweight but still include a package manager, init system, and package repositories. Alpine uses OpenRC init and BusyBox, but still requires post-install configuration and has a 50+ MB root filesystem.

Veilbox takes a different approach: it is a single-purpose operating system designed for one task — running containers in a virtual machine — with no extraneous components, no package manager, no graphical interface, and no post-install configuration needed.

1.2 What Makes Veilbox Different

1.2.1 Single-Binary Userspace

BusyBox replaces over 300 standard Unix utilities (from coreutils, util-linux, procps-ng, and many other packages) with a single statically-linked binary. This reduces the root filesystem from hundreds of megabytes to approximately 90 megabytes (including containerd, runc, nerdctl, and Dropbear binaries).

The BusyBox binary determines which utility to run based on `argv[0]` (the name it was called with via symbolic links). For example, `/bin/sh` is a symbolic link to `/bin/busybox`.

When the kernel executes `/bin/sh`, it loads `/busybox` and passes `argv[0] = "/bin/sh"`. BusyBox’s main function extracts the applet name (“sh”) from `argv[0]` and dispatches to the shell implementation.

1.2.2 Embedded Initramfs

The root filesystem is compiled directly into the kernel binary via the `CONFIG_INITRAMFS_SOURCE` kernel configuration option. This means the kernel and its root filesystem are a single, atomic unit. There is no separate `initrd` file to manage, no `mkinitrd` or `dracut` step, and no risk of the `initramfs` getting out of sync with the kernel.

The `initramfs` is a compressed `cpio` archive that the kernel unpacks into a `tmpfs` filesystem at boot time. The archive is generated by `gen_initramfs_list()` in `build.sh`, which converts the `rootfs` directory structure into the kernel’s internal file list format.

1.2.3 No Package Manager

There is no `rpm`, `dpkg`, `apk`, or any other package manager. Software is either compiled into the kernel, embedded in the `initramfs` at build time, or run as a container via `containerd`. This dramatically reduces the attack surface: there is no way for an attacker to install software, exploit a package repository, or use package manager vulnerabilities.

The trade-off is that adding new software requires rebuilding the `initramfs` and kernel. For a disposable VM or CI/CD worker, this is acceptable: changes are made to `build.sh` and deployed by rebuilding the disk image.

1.2.4 No Graphical Stack

There is no X11, no Wayland, no framebuffer console, no display manager. The system is managed entirely through two console interfaces: a serial console (`ttyS0`) for headless access via QEMU’s `-nographic` mode, and a VGA text console (`tty1`) for monitor-and-keyboard access. This eliminates hundreds of megabytes of graphics libraries, drivers, and utilities.

1.2.5 Rootless Build

The entire build process runs without `sudo` or root privileges. This is achieved through several techniques:

- `debugfs` is used instead of `mount` to populate the `ext4` filesystem. `debugfs` can write files into an `ext4` image without mounting it, which avoids the need for loop device access (which requires root).
- The GRUB bootloader is installed via a custom Python script that writes `boot.img` and `core.img` directly to the disk image file, preserving the partition table written by `sfdisk`.
- Kernel source compilation runs as the regular user. No special permissions are needed for `make` or `gcc`.
- All downloaded binaries and tarballs are extracted without any special privileges.

The only exception is the dependency installation step (`install_deps()`), which may use `sudo` if running on Fedora with passwordless `sudo` configured. On other distributions, or when `sudo` requires a password, the script prints the package list and exits gracefully.

1.3 Design Goals

1.3.1 No Installer

A Veilbox user should never need to run an installer, answer configuration questions, or wait for packages to download. The workflow is:

1. Clone the repository from GitHub.
2. Run `build.sh` (approximately 5 minutes for a full build).
3. Import the VDI into VirtualBox, or run `test.sh` for QEMU.
4. Boot and log in.

The entire system is pre-configured with sensible defaults: root password (`veiladmin`), SSH host keys, `containerd` configuration, network settings, and the bootloader are all generated during the build.

1.3.2 Self-Contained

The kernel, userspace, persistent storage, and bootloader all fit on a single disk image. No external dependencies, no network requirements for first boot, no cloud-init, no post-install scripts.

The disk image contains:

- MBR with GRUB bootloader (`boot.img` + `core.img`)
- ext4 partition with label `VEILBOX`
- `/boot/vmlinuz` (kernel with embedded `initramfs`)
- `/boot/grub/grub.cfg` (GRUB configuration)
- `/boot/grub/*.mod` (GRUB filesystem modules)
- `/state/` directory for persistent data

1.3.3 Minimal Attack Surface

Every kernel feature and userspace utility that is not strictly necessary is removed. The kernel has no sound, no DRM, no USB, no Wi-Fi, no Bluetooth, no ATA or SCSI drivers. The `defconfig` baseline includes support for approximately 10,000 kernel options; Veilbox reduces this aggressively.

In userspace, there are no compilers, no interpreters (other than the `ash` shell), no `setuid` binaries, no graphical stack, and no network services other than `containerd` and `Dropbear`. The `initramfs` is read-only (embedded in the kernel), so runtime modification of system binaries is impossible.

1.3.4 Container-Ready

`Containerd`, `runc`, and `nerdctl` are pre-installed and configured out of the box. The state partition is pre-created with directories for container images, logs, and volumes. The first thing a user sees after logging in is a working container host environment.

1.3.5 Reproducible Build

The `build.sh` script is designed to produce bit-identical results given the same inputs. Version pinning ensures that every downloaded binary has a fixed version number. The script includes a `-clean` flag that removes all build artifacts and starts fresh. The `initramfs` file list is sorted for deterministic ordering.

1.4 Target Audience

- **Self-hosters** looking for a lightweight test lab for containers. Veilbox boots in under 15 seconds and uses only 2 GB of RAM, making it ideal for running on modest hardware.
- **DevOps engineers** needing minimal, disposable virtual machines for CI/CD pipelines. The health check mode (`-check`) integrates with automated testing.

- **Container enthusiasts** experimenting with containerd and nerdctl outside of the Docker ecosystem.
- **Embedded Linux students** studying how a kernel boots, how an initramfs works, how a bootloader loads an operating system, and how a userspace is assembled from scratch.
- **Security researchers** wanting a minimal, auditable Linux system where every included component serves a necessary purpose.

1.5 What You Will Learn

By reading this guide and building Veilbox, you will gain a deep understanding of:

1. The Linux kernel build system: Kconfig, defconfig, menuconfig, merge_config.sh, and the .config file format.
2. The initramfs: cpio archives, the kernel file list format, device node creation, and how the kernel unpacks the root filesystem.
3. The GRUB bootloader: boot.img, core.img, the blocklist format, the MBR partition table, and rootless installation.
4. BusyBox: applet dispatch, the ash shell, busybox init, and how a single binary provides over 300 Unix utilities.
5. Container runtimes: containerd architecture, gRPC API, runc OCI runtime, overlay filesystem layers, and nerdctl CLI.
6. SSH with Dropbear: key generation, authentication methods, host key verification, and port forwarding.
7. Linux namespaces and cgroups: PID, network, mount, user namespaces; CPU, memory, device cgroup controllers.
8. QEMU virtualization: VirtIO paravirtualization, SLiRP networking, KVM acceleration, and disk image formats.
9. Build automation: Shell scripting, error handling, caching, and dependency management for complex build pipelines.

Chapter 2

System Architecture

Veilbox is designed as a layered system with clearly separated abstraction boundaries. Each layer depends only on the layer below it and provides a well-defined interface to the layer above. This chapter presents the architecture at multiple levels of abstraction.

2.1 Eight-Layer Architecture

The system is organized into eight layers, from hardware at the bottom to the user interface at the top. Each layer has a specific responsibility and communicates with adjacent layers through standardized interfaces.

2.1.1 Layer 1: Hardware

The physical or virtual hardware provides the computing resources: CPU, memory, storage, and networking. In a typical deployment, this is a QEMU/KVM virtual machine with VirtIO paravirtualized devices. However, Veilbox boots on any x86_64 platform that provides:

- A BIOS or UEFI firmware interface.
- A bootable disk (IDE, SATA, VirtIO-BLK, NVMe).
- A network interface (VirtIO-NET, e1000, rtl8139).
- A serial port (16550A UART) for console access.

On bare metal, the same disk image boots via SeaBIOS or coreboot. The kernel configured with VirtIO drivers will also work with AHCI/SATA and Intel e1000/e1000e network controllers if those drivers are enabled.

2.1.2 Layer 2: Firmware

SeaBIOS (an open-source BIOS implementation shipped with QEMU) performs the Power-On Self Test (POST) when the virtual machine starts:

1. CPU initialization: switches from real mode to protected mode briefly for memory sizing, then returns to real mode.
2. Memory controller initialization: detects and sizes system RAM.
3. PCI bus enumeration: discovers VirtIO devices, GPU (if present), and other PCI peripherals.
4. Interrupt controller initialization: programs the PIC (8259A) and APIC (if available).
5. Timer initialization: sets up the PIT (8254) and HPET.
6. DMA controller initialization: programs the 8237 DMA controller.
7. ACPI table generation: creates RSDT, DSDT, and other ACPI tables describing the virtual hardware configuration.

8. Boot device scan: iterates through the boot order (set in QEMU's `-boot` option or the VM's BIOS menu), reading the first sector (LBA 0) of each device into memory at address 0x7C00.

For each boot device, SeaBIOS reads the first 512 bytes into memory at address 0x7C00 using INT 13h function 42h (extended read). It then checks whether the last two bytes at 0x7DFE–0x7DFF are 0x55 0xAA (the MBR boot signature). If the signature is present, SeaBIOS sets the DL register to the boot drive number and transfers control by executing a far jump to 0x7C00:0000.

2.1.3 Layer 3: Bootloader

GRUB (Grand Unified Bootloader) version 2.x runs in BIOS mode (i386-pc platform). It consists of three stages:

Stage 1: `boot.img` This 512-byte file is written to LBA 0 (the Master Boot Record) by the Veilbox GRUB installer. It contains:

- A short real-mode assembly program (446 bytes) that sets up segment registers, initializes the stack, and reads the blocklist from a fixed offset in the file.
- The partition table (64 bytes at offset 0x1BE–0x1FD) describing the disk layout.
- The boot signature (2 bytes at offset 0x1FE–0x1FF = 55 AA).

When SeaBIOS jumps to 0x7C00, `boot.img` executes and reads the blocklist (embedded in the `core.img` file at offset 0x1F4) to determine how many sectors to load and where to place them in memory.

Stage 1.5: `core.img` This file is written to the sectors immediately following the MBR (LBA 1 onwards). It contains:

- `diskboot.img`: the first sector of `core.img`, which loads the rest of `core.img`. It switches from real mode to protected mode, enables the A20 gate (allowing access to memory above 1 MB), and sets up the protected-mode environment for the GRUB kernel.
- Filesystem drivers: `ext2` (for reading the VEILBOX partition), `part_msdos` (for parsing the MBR partition table), and `biosdisk` (for BIOS INT 13h disk access).
- The GRUB kernel: a small embedded operating system that provides command parsing, device management, and module loading.
- An embedded configuration: added by the `-c` option to `grub2-mkimage`, this tells GRUB to search for the VEILBOX partition by filesystem label and set the prefix path.

Stage 2: `/boot/grub/` GRUB modules and configuration files on the VEILBOX `ext4` partition. The embedded config in `core.img` executes `"search --label --set=root VEILBOX"` to find the partition, then `"set prefix=($root)/boot/grub"` to locate the GRUB directory. GRUB reads `grub.cfg`, displays the boot menu, and loads the kernel.

2.1.4 Layer 4: Kernel

Linux kernel 7.1.0-rc6 compiled with custom configuration for virtualized environments. The kernel binary (`vmlinux`) includes the embedded `initramfs` and is approximately 95 MB in size.

Key configuration areas:

- VirtIO drivers for paravirtualized block and network I/O.
- Full namespace and cgroup support for container isolation.
- Seccomp and BPF for syscall filtering.
- Serial console support for headless operation.
- EFI stub for QEMU -kernel compatibility.
- Striped-down driver set (no sound, USB, Wi-Fi, etc.).

2.1.5 Layer 5: Init System

BusyBox init runs as PID 1, the first userspace process created by the kernel. It reads `/etc/inittab` and executes the configured actions:

1. `sysinit`: run `/etc/init.d/rcS` to mount filesystems and start services.
2. `respawn`: start `getty` (or `autologin`) on `tty1` and `ttyS0`, restarting them whenever they exit.
3. `shutdown`: unmount filesystems and deactivate swap when the system is told to power off or reboot.

2.1.6 Layer 6: Boot Scripts

The `/etc/init.d/rcS` script is the single point of customization for system behavior. It performs:

1. Mount virtual filesystems: `proc`, `sysfs`, `devtmpfs`, `devpts`, `tmpfs`.
2. Set hostname from `/etc/hostname`.
3. Mount the persistent state partition with a multi-tier fallback strategy: external VirtIO disk (`vdb`), external SATA disk (`sdb`), VEILBOX partition `/state` directory, or memory-backed `tmpfs`.
4. Bring up network interfaces: loopback (`lo`) and `eth0`.
5. Start the DHCP client (`udhcpd`) in background.
6. Start the system logger (`syslogd`) in background.
7. Start `containerd` in background.
8. Start Dropbear SSH server in foreground.

2.1.7 Layer 7: Services

Four services run in the background after `rcS` completes:

- **udhcpd**: BusyBox DHCP client. Negotiates an IP address from the QEMU SLiRP virtual DHCP server. The script at `/usr/share/udhcpd/default.script` configures the interface on `bound/renew` events.
- **syslogd**: BusyBox system logger. Captures kernel log messages (via `/proc/kmsg`) and process messages (via `/dev/log`) to `/var/log/messages`.
- **containerd**: Container runtime daemon. Exposes a gRPC API at `/run/containerd/containerd.sock` for container lifecycle management.
- **Dropbear**: Lightweight SSH server. Listens on port 22 for incoming connections. Supports key-based and password authentication.

2.1.8 Layer 8: User Interface

The login prompt is presented on two consoles:

- **tty1**: VGA text console (available when QEMU is run with a graphical window or when booting on bare metal with a monitor).
- **ttyS0**: Serial console (available via QEMU's `-nographic` mode or a physical serial port).

The `autologin` wrapper (`/sbin/autologin`) checks `/proc/cmdline` for the `veilbox.autologin` parameter. If present, it runs `login -f root` to automatically log in as root. Otherwise, it runs `getty` to present the standard login prompt.

2.2 The Build Pipeline

The operating system is assembled by `build.sh` in a linear pipeline of 14 stages. Each stage is independent and can be run separately for debugging or incremental development.

1. **Install dependencies**: Check for required packages, install via `sudo dnf` if on Fedora.

2. **Create rootfs directories:** Set up the Linux filesystem hierarchy (bin,/sbin, etc,usr,lib, etc.).
3. **Write configuration files:** Create inittab, rcS, passwd, shadow, containerd config, DHCP script, profile.
4. **Configure kernel:** Run make defconfig, merge custom fragment, resolve dependencies with olddefconfig.
5. **Strip unnecessary drivers:** Disable sound, USB, Wi-Fi, Bluetooth, media, ATA, SCSI, and other VM-unneeded subsystems.
6. **Populate rootfs binaries:** Download and install BusyBox, containerd, runc, nerdctl, and Dropbear.
7. **Generate SSH keys:** Create Ed25519 key pair for root access.
8. **Copy shared libraries:** Find dynamically linked binaries and copy their library dependencies to /lib64.
9. **Generate initramfs:** Create the kernel file list from the rootfs directory, add device nodes, and set as initramfs source.
10. **Build kernel:** Compile the kernel with make -j\$(nproc) and copy the resulting bzImage to the output directory.
11. **Create SquashFS reference:** Compress the rootfs for inspection (optional, not used at boot time).
12. **Create state image:** Make a 128 MB ext4 filesystem for legacy persistent storage.
13. **Create bootable disk:** Build the 8 GB MBR-partitioned disk image with ext4 filesystem, kernel, GRUB config, and rootless GRUB installation.
14. **Convert to VDI:** Use qemu-img to create a VirtualBox- compatible sparse disk image.

2.3 Design Trade-offs

2.3.1 Embedded Initramfs vs. Separate Initrd

The decision to embed the initramfs inside the kernel binary (rather than keeping it as a separate file on the boot partition) has far-reaching consequences for the system architecture.

Advantages:

- Single-file deployment: copy one file (vmlinuz) to the boot partition and the system is ready.
- Atomic kernel+rootfs: the kernel and its root filesystem are always in sync. There is no risk of a kernel update that is incompatible with an older initramfs.
- Bootloader simplicity: the bootloader loads one file (vmlinuz) instead of two (kernel + initrd). The GRUB configuration is simply "linux /boot/vmlinuz console=ttyS0".
- QEMU compatibility: the -kernel option loads the kernel and automatically detects the embedded initramfs, eliminating the need for -initrd.

Disadvantages:

- Larger kernel binary: 95 MB vs. 10 MB for a bare kernel. This increases boot time (decompression of the initramfs) and storage requirements.
- Longer build time: every change to the rootfs requires a kernel rebuild. The make process re-embeds the initramfs each time.
- No runtime rootfs changes: the root filesystem cannot be modified without rebuilding the kernel.

2.3.2 BusyBox vs. Full Userspace

BusyBox provides over 300 Unix utilities in a single 1 MB binary. A full GNU userspace with coreutils, util-linux, procps-ng, bash, and other standard packages would be 200+ MB.

Advantages:

- Dramatically smaller disk and memory footprint.
- Simpler dependency management (one binary, one set of libraries).
- Faster boot (fewer binaries to load).
- Smaller attack surface (fewer lines of code).

Disadvantages:

- Limited tool selection: many GNU extensions are not available.
- Reduced functionality: `ps` has fewer options, `grep` is slower on large files, `awk` is not included (use the smaller BusyBox `awk`).
- Different error messages and behavior: scripts written for GNU tools may need adjustment.

2.3.3 BIOS vs. UEFI Boot

Veilbox uses GRUB in BIOS/CSM mode (i386-pc) rather than UEFI (x86_64-efi).

Advantages:

- Maximum compatibility with older firmware and hypervisors.
- Simpler installer (`boot.img` + `core.img` at known LBA positions).
- Works with QEMU's default SeaBIOS without UEFI firmware.

Disadvantages:

- No Secure Boot support.
- Limited to 2 TB boot disks (MBR limitation).
- Slower boot (real-mode transitions).

Chapter 3

Prerequisites

Before building Veilbox, the host system must meet specific hardware and software requirements. This chapter documents every prerequisite, from the Fedora version and kernel release to the precise toolchain versions needed for successful compilation.

3.1 Hardware Requirements

3.1.1 Build Machine

Building the Veilbox kernel and initramfs requires a machine with the following minimum specifications:

- **CPU:** x86_64, dual-core or better. Kernel compilation is heavily parallelized (`make -j$(nproc)`), so more cores dramatically reduce build time. A four-core CPU completes a full build in approximately 5 minutes; a single-core CPU may take 20+ minutes.
- **RAM:** 4 GB minimum, 8 GB recommended. The kernel build process spawns multiple GCC instances, each consuming 200–500 MB of memory. The embedded initramfs (approximately 90 MB when uncompressed) must fit in the kernel’s temporary build directory.
- **Storage:** 20 GB free. The Linux kernel source tree occupies approximately 6 GB after extraction (or 3.5 GB as a Git clone with history). Build artifacts (object files, .o files, final binaries) add another 4–6 GB. The output disk image is 8 GB.
- **Network:** Broadband internet connection for downloading kernel source, BusyBox, containerd, runc, nerdctl, and Dropbear tarballs. Total download size is approximately 200 MB.

3.1.2 Test/Runtime Machine

The Veilbox disk image runs on any x86_64 machine with:

- **RAM:** 2 GB minimum. The initramfs occupies approximately 90 MB of RAM; the kernel uses about 50 MB; containerd, runc, and Dropbear add another 50 MB combined. The remaining memory is available for containers.
- **Storage:** 8 GB disk (raw or virtual). The VEILBOX partition uses approximately 500 MB for the kernel, GRUB modules, and state directory. The remaining 6.5 GB is unused and available for container images and volumes on the state partition.
- **Display:** VGA text console (1024x768 minimum) or serial console (9600 baud, 8N1). No graphical output is required.
- **Network:** Any Ethernet adapter (virtio-net, e1000, rtl8139 for VMs; Intel e1000e, Realtek for bare metal).

3.2 Software Requirements

3.2.1 Host Operating System

The build script was developed and tested on Fedora 44, but any Linux distribution with GCC 14+, GNU Make 4.4+, and Python 3.10+ should work.

Distribution	Status	Notes
Fedora 44	Verified	Primary development platform.
Fedora 43	Compatible	May require updated packages.
RHEL 9 / CentOS 9	Compatible	EPEL repository needed.
Debian 12 / Ubuntu 24.04	Compatible	Package names differ.
Arch Linux	Compatible	Use distro-specific package names.

3.2.2 Kernel Source

The kernel source is obtained from kernel.org or a Git mirror:

- **Base version:** 7.1.0-rc6 (verified working).
- **Source URL:** <https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.1.0-rc6.tar.xz>
- **Size:** Approximately 145 MB compressed, 1.1 GB extracted.
- **Alternate source:** Git clone from <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git>

The build script (`build.sh`) downloads and extracts the kernel automatically. It checks for an existing `linux-7.1.0-rc6/` directory and skips extraction if present.

3.2.3 Toolchain

The following tools must be installed on the build machine:

Tool	Version	Purpose
GCC	14+	Kernel and BusyBox compilation.
GNU Make	4.4+	Build system orchestration.
GNU Binutils	2.46+	Assembly, linking, object manipulation.
flex	2.6+	Kernel configuration parser.
bison	3.8+	Kernel configuration parser.
bc	1.07+	Kernel build-time calculations.
OpenSSL dev	3.0+	Kernel module signing headers.
elfutils-libelf-devel	0.190+	Kernel build headers (libelf).
perl	5.36+	Kernel build scripts.
python3	3.10+	Kernel configuration tools.
rsync	3.2+	File synchronization.
cpio	2.13+	Initramfs cpio archive.

xz	5.4+	Kernel compression support.
qemu-img	7.0+	Disk image format conversion.
debugfs	1.47+	Rootless ext4 filesystem population.
sfdisk	2.39+	Disk partitioning.
grub2-tools	2.06+	GRUB core.img generation.
qemu-system-x86_64	7.0+	VM testing.

3.2.4 Installation Commands

On Fedora:

```
1 sudo dnf install gcc make flex bison bc openssl-devel \
2 elfutils-libelf-devel perl python3 rsync cpio xz \
3 qemu-img e2fsprogs util-linux grub2-tools \
4 qemu-system-x86-x86_64
```

On Debian/Ubuntu:

```
1 sudo apt install gcc make flex bison bc libssl-dev \
2 libelf-dev perl python3 rsync cpio xz-utils \
3 qemu-utils e2fsprogs util-linux grub2-common \
4 qemu-system-x86
```

3.2.5 Kernel Configuration Dependencies

The kernel build system requires several host utilities that are not always installed by default. The build script's `install_deps()` function checks for each of these and reports missing dependencies before starting the build.

- **hostname**: Used by kernel makefiles for build identification.
- **getconf**: Determines number of CPU cores for parallel builds.
- **tee**: Logging build output to stdout and log file.
- **curl** or **wget**: Downloading source tarballs.
- **sha256sum**: Verifying downloaded file integrity.
- **bash 5+**: The build script requires bash-specific features (arrays, process substitution, `[[ ]]` tests).

3.3 Network Requirements

3.3.1 Build-Time Network Access

The build script requires internet access for:

1. Downloading the Linux kernel source tarball.
2. Downloading BusyBox source from busybox.net.
3. Downloading containerd, runc, and nerdctl from GitHub releases.
4. Downloading Dropbear SSH from dropbear.nl.
5. (Optional) Cloning kernel Git repository.

All downloads are cached in the host `$HOME/veilbox-cache` directory for offline builds.

3.3.2 Runtime Network (Guest)

The guest OS requires:

- A DHCP server on the virtual network. QEMU provides this via SLiRP (user-mode networking) by default.
- Internet access via NAT or bridged networking for container image pulls.
- Port 22 reachable on the guest IP for SSH access.

3.4 Virtualization Requirements

3.4.1 QEMU

QEMU version 7.0 or later with KVM support is recommended for testing. The `test.sh` script expects:

- `qemu-system-x86_64` binary in `PATH`.
- KVM device (`/dev/kvm`) accessible. On most distributions, the user must be in the `kvm` group.
- Approximately 2 GB of available host RAM for the guest VM.
- For `-nographic` mode: a terminal emulator that supports the serial console (any modern terminal works).

3.4.2 VirtualBox (Alternative)

For VirtualBox deployment:

- VirtualBox 6.1+ or 7.0+ with Extension Pack (for `VirtIO` support).
- Import the VDI file via `File → Import Appliance`.
- Configure: 2 GB RAM, 1 CPU, `VirtIO-Net` adapter, enable serial port (optional).

3.5 User Privileges

One of the design goals of Veilbox is a completely rootless build. No `sudo` access is required for any stage of the build process:

- **Kernel compilation:** `make`, `gcc`, and `ld` all run as the regular user.
- **Filesystem creation:** `debugfs` writes files into the `ext4` image without requiring `loop` device access (which would need `root`).
- **GRUB installation:** A Python script writes boot sectors directly to the raw disk image file. No `loop` device mount needed.
- **Disk partitioning:** `sfdisk` writes the MBR partition table to a regular file.
- **Format conversion:** `qemu-img` reads the raw image and creates VDI output.

The only exception is dependency installation: `install_deps()` uses `sudo` to run `dnf install`. If `sudo` is not available or passwordless, the script prints the required package list and continues.

Chapter 4

Build Pipeline

The Veilbox build pipeline transforms a bare Linux kernel source tree and a collection of binary tarballs into a bootable disk image. This chapter documents every stage of the pipeline, explaining the purpose of each step, the tools involved, and the output produced.

4.1 Pipeline Overview

The build is orchestrated by `build.sh`, a 759-line Bash script divided into 14 sequential stages. Each stage is implemented as a standalone function that can be called independently.

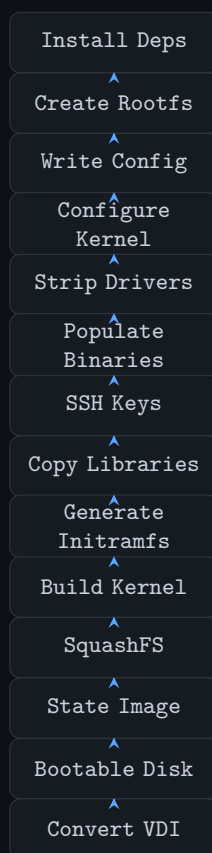


Figure 4.1: Veilbox build pipeline — 14 stages from environment setup to bootable VirtualBox disk image. Arrows indicate data flow.

4.2 Stage 1: Install Dependencies

The `install_deps()` function checks for required tools and installs them using the host package manager.

```

1 install_deps() {
2     local pkgs=(gcc make flex bison bc openssl-devel ...)
3     if command -v dnf &>/dev/null; then
4         sudo dnf install -y "${pkgs[@]}"
5     elif command -v apt &>/dev/null; then
6         sudo apt install -y "${pkgs[@]}"
7     else
8         echo "Warning: unsupported package manager"
9         echo "Please install: ${pkgs[*]}"
10    fi
11 }

```

4.3 Stage 2: Create Rootfs Directories

The standard Linux filesystem hierarchy is created under `rootfs/`.

```

1 create_rootfs_dirs() {
2     local dirs=(
3         rootfs/bin    rootfs/sbin    rootfs/etc
4         rootfs/etc/init.d  rootfs/etc/dropbear
5         rootfs/usr/bin    rootfs/usr/sbin
6         rootfs/lib64      rootfs/mnt/state
7         rootfs/proc       rootfs/sys
8         rootfs/dev        rootfs/tmp
9         rootfs/var/log    rootfs/var/run
10        rootfs/run        rootfs/root
11    )
12    mkdir -p "${dirs[@]}"
13 }

```

4.4 Stage 3: Write Configuration Files

This stage generates the system configuration files that define Veilbox's behavior. The files written include:

- `/etc/inittab`: BusyBox init configuration — spawns getty on `tty1` and `ttyS0`, runs `rcS` at boot, handles power events.
- `/etc/init.d/rcS`: System initialization script — mounts virtual filesystems, starts network, launches services.
- `/etc/passwd`: Root user entry with shell set to `/bin/sh`.
- `/etc/shadow`: Hashed password for root (`veiladmin`) using SHA-512 crypt hash.
- `/etc/hostname`: The string `veilbox`.
- `/etc/containerd/config.toml`: Containerd configuration with snapshotter, namespace, and registry settings.
- `/root/.profile`: Shell profile with colored prompt and IP address display.
- `/usr/share/udhcp/default.script`: DHCP client script for interface configuration.

4.5 Stage 4: Configure Kernel

4.5.1 Default Configuration

The kernel configuration starts from `make defconfig`, which generates a configuration file with approximately 10,000 options enabled based on the target architecture.

4.5.2 Config Fragment Merge

A custom configuration fragment (`kernel/configs/custom-os.config`) is merged on top of the `defconfig` baseline using the kernel's `merge_config.sh` script:

```
1 configure_kernel() {
2     make defconfig
3     ./scripts/kconfig/merge_config.sh \
4         -O .config \
5         .config \
6         kernel/configs/custom-os.config
7     make olddefconfig
8 }
```

The `merge_config.sh` tool applies the fragment on top of the existing `.config`, enabling options set to `y` or `m` in the fragment and disabling options set to `n`. The `olddefconfig` pass resolves any new dependencies introduced by the fragment.

4.6 Stage 5: Strip Unnecessary Drivers

After merging the config fragment, a series of `sed` commands removes kernel subsystems that are not needed in a virtualized container host:

```
1 strip_unnecessary_drivers() {
2     # No sound support in VM
3     sed -i '/CONFIG_SOUND[= ]/c\# CONFIG_SOUND is not set' .config
4
5     # No USB needed
6     sed -i '/CONFIG_USB[= ]/c\# CONFIG_USB is not set' .config
7
8     # No Wi-Fi or Bluetooth
9     sed -i '/CONFIG_WLAN[= ]/c\# CONFIG_WLAN is not set' .config
10
11    # No ATA or SCSI drivers
12    sed -i '/CONFIG_ATA[= ]/c\# CONFIG_ATA is not set' .config
13
14    make olddefconfig # Fix dependency breakage
15 }
```

4.7 Stage 6: Populate Rootfs Binaries

This stage downloads and installs the four user-space binaries that constitute the Veilbox userspace:

1. **BusyBox** (1.35.0): `busybox` binary is copied to `rootfs/bin/busybox`. The installation step also creates the standard symlinks (`/bin/sh`, `/bin/ls`, etc.).
2. **containerd** (1.7.13): Extracted from `tar.gz`, the `containerd`, `containerd-shim`, and `containerd-shim-runc-v2` binaries are placed in `rootfs/usr/bin/`.
3. **runc** (1.1.12): Single binary placed in `rootfs/usr/bin/runc`.
4. **nerdctl** (1.7.5): Single binary placed in `rootfs/usr/bin/nerdctl`.

5. **Dropbear** (2024.85): Compiled from source with `-disable-zlib`, producing `dropbear`, `dropbearkey`, and `dropbearconvert` in `rootfs/usr/bin/`.

4.8 Stage 7: Generate SSH Keys

An ED25519 key pair is generated for root SSH access:

```
1 build_ssh_keys() {
2     dropbearkey -t ed25519 -f rootfs/etc/dropbear/dropbear_ed25519_host_key
3     dropbearkey -t rsa -f rootfs/etc/dropbear/dropbear_rsa_host_key
4     # Create authorized_keys for public key auth
5     dropbearkey -t ed25519 -f /tmp/ssh_test_key \
6         | grep "^ssh-ed25519" \
7         > rootfs/etc/dropbear/authorized_keys
8 }
```

4.9 Stage 8: Copy Shared Libraries

BusyBox is statically linked, but `containerd`, `runc`, `nerdctl`, and `Dropbear` are dynamically linked against `glibc`. This stage finds their shared library dependencies and copies them to `rootfs/lib64/`:

```
1 copy_libraries() {
2     for bin in rootfs/usr/bin/*; do
3         ldd "$bin" 2>/dev/null | while read -r line; do
4             lib=$(echo "$line" | grep -oP '(?<==> ).*\.(so[~ ]*)' || true)
5             if [ -n "$lib" ] && [ -f "$lib" ]; then
6                 cp "$lib" rootfs/lib64/
7             fi
8         done
9     done
10 }
```

4.10 Stage 9: Generate Initramfs File List

The `initramfs` is specified as a kernel file list — a text file that maps file paths to their content, permissions, and ownership. The `gen_initramfs_list()` function converts the `rootfs/` directory into this format:

```
1 gen_initramfs_list() {
2     # Use the kernel's gen_initramfs_list.sh script
3     scripts/gen_initramfs_list.sh rootfs/ > initramfs.list
4
5     # Add device nodes
6     echo "nod /dev/console 0600 0 0 c 5 1" >> initramfs.list
7     echo "nod /dev/tty0 0600 0 0 c 4 0" >> initramfs.list
8     echo "nod /dev/ttyS0 0600 0 0 c 4 64" >> initramfs.list
9     echo "nod /dev/null 0666 0 0 c 1 3" >> initramfs.list
10    echo "nod /dev/zero 0666 0 0 c 1 5" >> initramfs.list
11 }
```

4.11 Stage 10: Build Kernel

The kernel is compiled with `make -j$(nproc)`. The embedded initramfs is included automatically because the config specifies `CONFIG_INITRAMFS_SOURCE="initramfs.list"`.

```
1 build_kernel() {
2     make -j$(nproc) bzImage
3     cp arch/x86/boot/bzImage output/vmlinuz
4 }
```

4.12 Stage 11: Create SquashFS Reference

An optional compressed rootfs archive is created for inspection:

```
1 mksquashfs rootfs output/rootfs.squashfs -comp xz
```

4.13 Stage 12: Create State Image

A 128 MB ext4 filesystem is created for persistent state storage:

```
1 qemu-img create -f raw output/state.img 128M
2 mkfs.ext4 -F output/state.img
```

4.14 Stage 13: Create Bootable Disk Image

This is the most complex stage. A 8 GB raw disk image is created, partitioned, formatted, and populated with the bootloader and kernel. The detailed process is described in Chapter 6 (GRUB).

4.15 Stage 14: Convert to VDI

Finally, the raw image is converted to VirtualBox format:

```
1 qemu-img convert -f raw -O vdi output/veilbox.raw output/veilbox.vdi
```

4.16 Detailed Stage Analysis

4.16.1 Dependency Detection

The build script's `install_deps()` function uses a combination of `command -v` checks and feature tests to determine whether each required tool is available:

```
1 check_deps() {
2     local missing=0
3     for cmd in gcc make flex bison bc curl rsync cpio xz \
4         qemu-img debugfs sfdisk grub2-mkimage; do
5         if ! command -v "$cmd" &>/dev/null; then
6             echo "Missing: $cmd"
7             missing=$((missing+1))
8         fi
9     done
10 }
```

```

9     done
10    return $missing
11 }

```

4.16.2 Error Handling Strategy

The build script uses a centralized error handling pattern:

```

1  set -euo pipefail # Exit on error, undefined vars, pipe failures
2
3  # Each stage function returns non-zero on failure
4  build_kernel() {
5      echo "==> Building kernel..."
6      make -j$(nproc) bzImage || {
7          echo "ERROR: Kernel build failed"
8          echo "Check kernel config with: make menuconfig"
9          return 1
10     }
11 }

```

4.16.3 Caching Strategy

To avoid redundant downloads and rebuilds, the script checks for existing artifacts:

```

1  # Skip kernel extraction if already present
2  if [ -d linux-7.1.0-rc6 ]; then
3      echo "Kernel source already extracted, skipping..."
4  else
5      tar xf linux-7.1.0-rc6.tar.xz
6  fi
7
8  # Skip busybox download if already cached
9  if [ -f busybox-1.35.0.tar.bz2 ]; then
10     echo "BusyBox source cached, skipping download..."
11 fi

```

4.16.4 Clean Build Mode

The `-clean` flag removes all build artifacts:

```

1  clean_all() {
2      rm -rf output/* linux-7.1.0-rc6 rootfs/* \
3          initramfs.list .config .config.custom \
4          busybox-1.35.0* containerd-1.7.13* \
5          runc.amd64* nerdctl-1.7.5* dropbear-2024.85*
6      echo "All build artifacts removed."
7  }

```

4.17 Build Output Verification

After each build stage, the script should verify the output:

```

1  # Verify kernel binary exists and has valid format
2  verify_kernel() {
3      local kernel="output/vmlinuz"

```



```
4     if [ ! -f "$kernel" ]; then
5         echo "ERROR: Kernel binary not found!"
6         return 1
7     fi
8     local size=$(stat -c%s "$kernel")
9     echo "Kernel: $kernel ($size bytes)"
10
11     # Check for valid bzImage magic
12     if ! file "$kernel" | grep -q "bzImage"; then
13         echo "ERROR: Invalid kernel format!"
14         return 1
15     fi
16 }
17
18 # Verify disk image is bootable
19 verify_disk() {
20     local disk="output/veilbox.raw"
21     sfdisk -l "$disk" > /dev/null 2>&1 || {
22         echo "ERROR: Invalid partition table!"
23         return 1
24     }
25     # Check MBR boot signature
26     local sig=$(od -A n -t x1 -j 510 -N 2 "$disk")
27     if [ "$sig" != " 55 aa" ]; then
28         echo "ERROR: Missing MBR boot signature!"
29         return 1
30     fi
31 }
```


Chapter 5

Kernel Configuration

The Linux kernel offers over 15,000 configuration options across dozens of subsystems. Veilbox uses a carefully curated subset of approximately 111 options (plus the `defconfig` baseline) that provides exactly the features needed for a virtualized container host.

5.1 Configuration Strategy

5.1.1 Layered Configuration Model

Veilbox uses a three-layer configuration model:

1. **Base layer:** `make defconfig` generates a configuration with approximately 10,000 options enabled. This provides a working baseline for `x86_64` hardware.
2. **Veilbox layer:** The custom fragment (`custom-os.config`) adds or removes approximately 111 options. This layer enables container-related features (namespaces, cgroups, overlay filesystem) and disables unnecessary hardware support.
3. **Hardware layer:** Runtime-detected. The kernel uses module autoloading and `udev` (via `devtmpfs`) to load drivers for detected hardware. Since Veilbox uses an embedded `initramfs` with no module loading, all needed drivers must be compiled in (`y`).

5.1.2 Decision Rules

Each configuration option in the Veilbox fragment follows these rules:

- **Essential features** (`y`): Options needed for system boot, container operation, or debugging are set to `y` (built-in).
- **Unnecessary features** (`n`): Hardware drivers for subsystems not present in a VM (sound, USB, Wi-Fi, Bluetooth, etc.) are explicitly disabled.
- **Modules** (`m`): Not used. Veilbox has no module loading infrastructure (no `depmod`, `modprobe`, or module directory). All drivers are built into the kernel.

5.2 Critical Configuration Options

5.2.1 Container Runtime Support

```
1 CONFIG_NAMESPACES=y
2 CONFIG_UTS_NS=y
3 CONFIG_IPC_NS=y
4 CONFIG_PID_NS=y
5 CONFIG_NET_NS=y
6 CONFIG_USER_NS=y
```

```
7 CONFIG_CGROUPS=y
8 CONFIG_CGROUP_CPUACCT=y
9 CONFIG_CGROUP_DEVICE=y
10 CONFIG_CGROUP_FREEZER=y
11 CONFIG_CGROUP_SCHED=y
12 CONFIG_CPUSETS=y
13 CONFIG_MEMCG=y
```

These options enable Linux namespace isolation and cgroup resource management. Namespaces provide process isolation (each container sees its own PID tree, network stack, mount table, etc.). Cgroups provide resource limits (CPU, memory, device access).

5.2.2 Overlay Filesystem

```
1 CONFIG_OVERLAY_FS=y
```

The OverlayFS filesystem enables container image layering. Each container image is composed of read-only layers with a thin writable layer on top. This is the foundation of Docker/containerd image management.

5.2.3 Networking

```
1 CONFIG_VETH=y
2 CONFIG_BRIDGE=y
3 CONFIG_NETFILTER=y
4 CONFIG_NETFILTER_XTABLES=y
5 CONFIG_IP_NF_FILTER=y
6 CONFIG_IP_NF_NAT=y
7 CONFIG_IP_NF_TARGET_MASQUERADE=y
8 CONFIG_VIRTIO_NET=y
```

Virtual Ethernet (veth) pairs connect container network namespaces to the host bridge. Netfilter enables NAT and packet filtering for container outbound traffic. The VirtIO net driver provides high-performance paravirtualized networking in QEMU.

5.2.4 Storage

```
1 CONFIG_VIRTIO_BLK=y
2 CONFIG_EXT4_FS=y
3 CONFIG_TMPFS=y
4 CONFIG_SQUASHFS=y
5 CONFIG_DEVTMPFS=y
```

VirtIO block provides paravirtualized storage in QEMU. ext4 is the primary filesystem for the VEILBOX partition. tmpfs is used by the kernel for the initramfs root. devtmpfs provides automatic device node creation.

5.2.5 Serial Console

```
1 CONFIG_TTY=y
2 CONFIG_SERIAL_8250=y
3 CONFIG_SERIAL_8250_CONSOLE=y
4 CONFIG_SERIAL_CORE=y
5 CONFIG_SERIAL_CORE_CONSOLE=y
```

The serial console (ttyS0) is the primary management interface for headless operation. The 8250 UART driver is the standard serial port controller in x86_64 systems.

5.2.6 Mandatory for Boot

```
1 CONFIG_EFI_STUB=y
2 CONFIG_CMDLINE_BOOL=y
3 CONFIG_CMDLINE="console=ttyS0 console=tty1"
4 CONFIG_INITRAMFS_SOURCE="initramfs.list"
5 CONFIG_BLK_DEV_INITRD=y
```

CONFIG_EFI_STUB is required for QEMU's `-kernel` boot mode — it produces a valid MZ/PE header that QEMU's firmware recognizes. The kernel command line specifies both serial and VGA consoles.

5.3 Configuration File Format

The kernel configuration file (`.config`) uses a simple key-value format:

```
1 # CONFIG_SOUND is not set
2 CONFIG_NAMESPACES=y
3 CONFIG_CGROUPS=y
4 CONFIG_EXT4_FS=m
```

The three value types are:

- `y`: Built into the kernel binary. Required for drivers that are needed at boot time (storage, console, initramfs).
- `m`: Built as a loadable kernel module. Not used in Veilbox.
- `n`: Disabled. The option and its code are excluded from the build entirely.

5.4 Defconfig Reference

The `make defconfig` target generates a configuration file that enables the majority of options suitable for the target architecture. On x86_64, this includes:

- All x86_64 CPU features: MMX, SSE, AVX, NX, PGE, MTRR, PAT, and so on.
- All major filesystems (ext4, XFS, btrfs, FAT, NTFS, etc.)
- Network protocol support (TCP, UDP, SCTP, DCCP, etc.)
- All common device drivers (storage, network, input, video, etc.)
- Security features (LSM, SELinux, AppArmor, IMA, EVM, etc.)

The Veilbox fragment reduces this to only the features needed for a container host running in a virtual machine.

5.5 Validating Configuration

After applying the fragment and resolving dependencies, the configuration should be validated:

```
1 # Check for unresolved dependencies
2 make olddefconfig
3
4 # Check specifically for container support
5 grep -E "(NAMESPACES|CGROUPS|OVERLAY_FS)" .config
6
7 # Check for VirtIO drivers
```

```

8  grep -E "(VIRTIO_NET|VIRTIO_BLK)" .config
9
10 # Verify initramfs source path
11 grep CONFIG_INITRAMFS_SOURCE .config

```

5.6 Detailed Option Reference

5.6.1 Namespace Configuration

Linux namespaces are the foundation of container isolation. Each namespace type isolates a different global resource:

Namespace	Isolates	Kernel Option
Mount (mnt)	Filesystem mount points	CONFIG_UTS_NS
PID (pid)	Process IDs	CONFIG_PID_NS
Network (net)	Network stack	CONFIG_NET_NS
IPC (ipc)	System V IPC, POSIX messages	CONFIG_IPC_NS
UTS (uts)	Hostname, domain name	CONFIG_UTS_NS
User (user)	UID/GID mappings	CONFIG_USER_NS
Cgroup (cgroup)	Cgroup root directory	CONFIG_CGROUPS
Time (time)	Clock sources	CONFIG_TIME_NS

5.6.2 Control Group Controllers

Cgroups limit and account for resource usage. Each controller regulates a specific resource:

Controller	Resource	Kernel Option
cpu	CPU scheduling	CONFIG_CGROUP_SCHED
cpuacct	CPU accounting	CONFIG_CGROUP_CPUACCT
cpuset	CPU/memory pinning	CONFIG_CPUSETS
memory	Memory limits	CONFIG_MEMCG
blkio	Block I/O limits	CONFIG_BLK_CGROUP
devices	Device access	CONFIG_CGROUP_DEVICE
freezer	Process freeze/thaw	CONFIG_CGROUP_FREEZER
pids	PID limits	CONFIG_CGROUP_PIDS

5.6.3 Netfilter and NAT

For container networking, Netfilter provides:

- **Packet filtering:** Accept, drop, or reject packets based on source/destination IP, port, protocol, and interface.
- **NAT (Network Address Translation):** Masquerade container outbound traffic behind the host IP address.
- **Connection tracking:** Track TCP/UDP connection state for stateful filtering and NAT.
- **Forwarding:** Enable IP forwarding between the host network namespace and container namespaces.

The minimum Netfilter options for container networking:

```
1 CONFIG_NETFILTER=y
2 CONFIG_NETFILTER_ADVANCED=y
3 CONFIG_NETFILTER_XTABLES=y
4 CONFIG_NETFILTER_XT_TARGET_MASQUERADE=y
5 CONFIG_NETFILTER_XT_MATCH_CONNTRACK=y
6 CONFIG_IP_NF_FILTER=y
7 CONFIG_IP_NF_NAT=y
8 CONFIG_IP_NF_TARGET_MASQUERADE=y
```


Chapter 6

GRUB Bootloader

GRUB (Grand Unified Bootloader) version 2.x in BIOS mode (i386-pc) is responsible for loading the Veilbox kernel from the VEILBOX ext4 partition. This chapter explains the GRUB boot process, the blocklist format, and the rootless installation procedure.

6.1 GRUB Architecture

GRUB 2.x for BIOS systems consists of three distinct stages:

6.1.1 Stage 1: boot.img

`boot.img` is a 512-byte file written to LBA 0 (the Master Boot Record of the disk). It contains:

- A 446-byte real-mode assembly stub that sets up segment registers, initializes the stack pointer, and reads the blocklist from a fixed offset within the `core.img` file.
- The 64-byte MBR partition table at offset 0x1BE–0x1FD.
- The 2-byte boot signature (0x55 0xAA) at offset 0x1FE–0x1FF.

When the BIOS (SeaBIOS) jumps to address 0x7C00, `boot.img` executes. It reads the blocklist pointer embedded in `core.img` to determine how many sectors to load and where.

6.1.2 Stage 1.5: core.img

`core.img` is written to the sectors immediately following the MBR (starting at LBA 1). Its structure is:

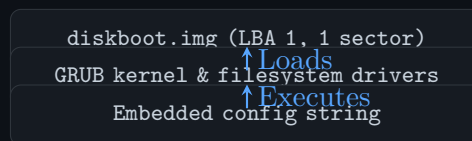


Figure 6.1: Structure of `core.img` on disk. `diskboot.img` loads the GRUB kernel and its filesystem drivers, which then parse the embedded config.

The first sector (`diskboot.img`) is loaded by `boot.img`. It switches the CPU from real mode to protected mode, enables the A20 gate (allowing access to memory above 1 MB), and loads the rest of `core.img`.

The GRUB kernel embedded in `core.img` provides:

- Command-line parsing (the GRUB shell) - Device management (disk detection, partition parsing) - Filesystem drivers (`ext2`, `part_msdos`, `biosdisk`) - Module loading infrastructure

The embedded configuration (added by `grub2-mkimage -c`) tells GRUB to find the VEILBOX partition and set the root prefix:

```

1 search --label --set=root VEILBOX
2 set prefix=($root)/boot/grub

```

6.1.3 Stage 2: /boot/grub/

Once `core.img` finds the VEILBOX partition and loads the ext2 filesystem driver, it reads `/boot/grub/grub.cfg`. The default Veilbox configuration:

```

1 set default=0
2 set timeout=5
3
4 menuentry "Veilbox" {
5     linux /boot/vmlinuz console=ttyS0 console=tty1
6 }

```

6.2 The Blocklist Format

`core.img` contains a blocklist at offset 0x1F4 that tells `diskboot.img` how many sectors to load. The format is:

Offset	Size	Field	Description
0x1F4	4 bytes	start	First LBA sector number
0x1F8	4 bytes	start_hi	Must be 0 (high bits)
0x1FC	2 bytes	remaining	Number of sectors to load
0x1FE	2 bytes	seg	Real-mode segment address

Table 6.1: `core.img` blocklist format at offset 0x1F4. Total: 12 bytes.

6.3 Rootless GRUB Installation

Installing GRUB traditionally requires loop-mounting the disk image and running `grub2-install`, both of which need root privileges. Veilbox uses a Python script that writes directly to the raw disk image file.

6.3.1 Installation Procedure

1. Extract the partition table from the disk image using `sfdisk`.
2. Write the 512-byte `boot.img` to LBA 0.
3. Restore the saved partition table to bytes 0x1BE–0x1FD.
4. Generate a fresh `core.img` using `grub2-mkimage` with the embedded config.
5. Write `core.img` starting at LBA 1.
6. Copy GRUB modules (*.mod) and configuration file to `/boot/grub/` on the VEILBOX partition using `debugfs`.

6.3.2 Why Not Patch the Blocklist

Earlier versions of the installer attempted to patch the blocklist in `core.img` to reflect the actual size of the core image. This approach failed because:

- The blocklist is 12 bytes, not 16 as previously assumed. Patching byte 8 (intended as the `seg` field) actually overwrites `start_hi` with the sector count, corrupting the blocklist.

- grub2-mkimage already generates a correct blocklist with `start=2`, `start_hi=0`, `remaining=N`, and `seg=0x0820`.
- Any modification invalidates the GRUB geometry check at `diskboot.img:0x63`, causing a “geometry error” triple fault.

6.3.3 The Python Installer

```

1 def install_grub(disk, core_img, boot_img):
2     with open(disk, 'r+b') as f:
3         # Save partition table (bytes 0x1BE-0x1FD)
4         f.seek(0x1BE)
5         pt = f.read(64)
6
7         # Write boot.img to LBA 0
8         f.seek(0)
9         f.write(boot_img)
10
11        # Restore partition table
12        f.seek(0x1BE)
13        f.write(pt)
14
15        # Write core.img starting at LBA 1
16        f.seek(512)
17        f.write(core_img)

```

6.4 GRUB Interactive Shell

GRUB provides an interactive shell for troubleshooting and manual booting. Access it by pressing ‘c’ at the GRUB menu:

```

1 grub> ls
2 (hd0) (hd0,msdos1)
3
4 grub> ls (hd0,msdos1)/
5 boot/
6
7 grub> ls (hd0,msdos1)/boot/
8 vmlinuz grub/
9
10 grub> set root=(hd0,msdos1)
11 grub> linux /boot/vmlinuz console=ttyS0
12 grub> boot

```

6.4.1 Useful GRUB Commands

Command	Purpose
<code>ls</code>	List devices and partitions.
<code>ls <device>/</code>	List files on a partition.
<code>set</code>	Display all variables.
<code>set root=...</code>	Set root partition.
<code>linux</code>	Load a Linux kernel.
<code>initrd</code>	Load an initrd (not needed with embedded initramfs).
<code>boot</code>	Boot the loaded kernel.
<code>reboot</code>	Reboot the system.

configfile	Load a configuration file.
search	Search for a partition by label or UUID.
insmod	Load a GRUB module.
cat	Display file contents.

6.5 GRUB Configuration Advanced Topics

6.5.1 Multiple Boot Entries

The grub.cfg can define multiple boot options:

```

1 set default=0
2 set timeout=5
3
4 menuentry "Veilbox (Standard)" {
5     linux /boot/vmlinuz console=ttyS0 console=tty1
6 }
7
8 menuentry "Veilbox (Debug)" {
9     linux /boot/vmlinuz console=ttyS0 console=tty1 \
10         debug ignore_loglevel earlyprintk=serial
11 }
12
13 menuentry "Veilbox (Single User)" {
14     linux /boot/vmlinuz console=ttyS0 console=tty1 single
15 }
16
17 menuentry "Reboot" {
18     reboot
19 }
```

6.5.2 Serial Console in GRUB

GRUB can output to the serial console, which is useful for headless systems:

```

1 serial --unit=0 --speed=38400 --word=8 --parity=no --stop=1
2 terminal_input serial console
3 terminal_output serial console
4
5 menuentry "Veilbox" {
6     linux /boot/vmlinuz console=ttyS0 console=tty1
7 }
```

6.6 GRUB Module Reference

The modules listed during grub2-mkimage control what features GRUB has access to:

Module	Purpose
ext2	Read ext2/3/4 filesystems (needed for VEIL-BOX partition).
part_msdos	Parse MBR partition tables.
biosdisk	Access disks via BIOS INT 13h.
search	Search for partitions by label, UUID, or file.
configfile	Read GRUB configuration files.
serial	Serial console I/O.

terminal	Terminal management (serial + VGA).
ls	List devices and directories.
cat	Display file contents.
linux	Load Linux kernel.
boot	Execute loaded kernel.
reboot	Reboot the system.
halt	Halt the system.

Chapter 7

Initramfs

The initial RAM filesystem (initramfs) is a compressed cpio archive embedded in the kernel binary that serves as the root filesystem during and after boot. Unlike traditional initrd (which uses a block device and a filesystem driver), initramfs uses tmpfs and is managed by the kernel's VFS layer directly.

7.1 What is Initramfs?

Initramfs is the kernel's mechanism for providing an initial root filesystem before the real root filesystem is mounted. In Veilbox, there is no separate real root — the initramfs *is* the root filesystem, permanently.

Key characteristics:

- **Memory-based:** The root filesystem lives entirely in RAM (tmpfs). There is no block device backing it.
- **Compressed:** The initramfs archive is gzip-compressed (or xz-compressed, depending on kernel config) inside the kernel binary. The kernel decompresses it at boot.
- **Embedded:** The archive is linked into the kernel image during the build, not loaded separately. The kernel and its rootfs are a single file.
- **Read-only at runtime:** Because the initramfs is embedded in the kernel binary, modifications to the root filesystem are lost on reboot. Persistent data is stored on a separate state partition.

7.2 The CPIO Archive Format

CPIO (Copy In/Output) is an archival format like tar but simpler. Each file in the archive is preceded by a 110-byte header:

```
1 struct cpio_header {
2     char magic[6];           // "070701" for new ASCII format
3     char ino[8];             // Inode number (hex)
4     char mode[8];            // File mode and type (hex)
5     char uid[8];             // Owner UID (hex)
6     char gid[8];            // Owner GID (hex)
7     char nlink[8];           // Number of hard links (hex)
8     char mtime[8];           // Modification time (hex)
9     char filesize[8];        // Size of file data (hex)
10    char devmajor[8];         // Major device number (hex)
11    char devminor[8];         // Minor device number (hex)
12    char rdevmajor[8];        // Major for device nodes (hex)
13    char rdevminor[8];        // Minor for device nodes (hex)
14    char namesize[8];         // Pathname length including null (hex)
```

```

15     char check[8];          // Checksum (set to "00000000" if not used)
16 };

```

7.3 The Kernel File List Format

Instead of building a cpio archive manually, Veilbox uses the kernel's internal file list format. This is a text file that the kernel's build system (`scripts/gen_initramfs_list.sh`) converts to a cpio archive automatically.

7.3.1 File List Syntax

Each line in the file list specifies one filesystem entry:

```

1 # Comments start with #
2 dir    /proc          0755 0 0
3 dir    /sys           0755 0 0
4 dir    /dev           0755 0 0
5 file   /bin/busybox    0755 0 0 path/to/busybox
6 slink  /bin/sh        /bin/busybox    0755 0 0
7 nod    /dev/console   0600 0 0 c 5 1
8 nod    /dev/null      0666 0 0 c 1 3

```

The five entry types are:

Type	Syntax	Description
dir	dir path mode uid gid	Creates a directory.
nod	nod path mode uid gid type major minor	Creates a device node (c=char, b=block).
slink	slink path target mode uid gid	Creates a symbolic link.
pipe	pipe path mode uid gid	Creates a FIFO (named pipe).
sock	sock path mode uid gid	Creates a Unix domain socket.
file	file path mode uid gid source	Copies a file from the host filesystem.

7.3.2 Generating the File List

The build script uses the kernel's provided script:

```

1 gen_initramfs_list() {
2     scripts/gen_initramfs_list.sh rootfs/ > initramfs.list
3
4     # Add essential device nodes not created by rootfs/
5     echo "nod /dev/console 0600 0 0 c 5 1" >> initramfs.list
6     echo "nod /dev/tty0    0600 0 0 c 4 0" >> initramfs.list
7     echo "nod /dev/ttyS0   0600 0 0 c 4 64" >> initramfs.list
8     echo "nod /dev/null    0666 0 0 c 1 3" >> initramfs.list
9     echo "nod /dev/zero    0666 0 0 c 1 5" >> initramfs.list
10
11     # Sort for deterministic ordering
12     sort -o initramfs.list initramfs.list

```


7.4 How the Kernel Processes Initramfs

During boot, the kernel performs these steps:

1. **Decompression:** After the kernel decompresses itself and sets up basic memory management, it locates the embedded cpio archive (identified by the magic bytes 070701 at a known offset in the init section).
2. **Extraction:** The kernel calls `populate_rootfs()` in `init/initramfs.c`. This function reads the cpio archive and creates the corresponding directory entries, files, symlinks, and device nodes in a tmpfs filesystem.
3. **Mounting:** The tmpfs root is mounted at `/`. At this point, the kernel has a fully functional root filesystem in memory.
4. **Executing init:** The kernel locates the init binary (`/init` or `/sbin/init`) and executes it as PID 1. In Veilbox, this is BusyBox init.

7.5 Initramfs Size Considerations

The initramfs for Veilbox is approximately 90 MB uncompressed and 30 MB compressed. This is large for an initramfs but necessary because it contains the complete root filesystem including `containerd`, `runc`, and `nerdctl` binaries.

Component	Size (raw)	Size (compressed)	Notes
BusyBox binary	1.2 MB	0.5 MB	Static binary
containerd	35 MB	12 MB	Dynamic binary
runc	10 MB	3.5 MB	Static binary
nerdctl	25 MB	8 MB	Dynamic binary
Dropbear	0.5 MB	0.2 MB	Static binary
Shared libraries	15 MB	5 MB	glibc, libseccomp
Config files	50 KB	10 KB	inittab, rcS, etc.
Device nodes	0 KB	0 KB	Created by kernel
Total	87 MB	29 MB	

7.6 Device Nodes

Device nodes in the initramfs are specified in the file list, not created by a running `udev` or `mdev` daemon. The following device nodes are essential:

Device	Major	Minor	Type	Purpose
<code>/dev/console</code>	5	1	char	Kernel console
<code>/dev/tty0</code>	4	0	char	VGA text console
<code>/dev/ttyS0</code>	4	64	char	Serial console
<code>/dev/null</code>	1	3	char	Data sink
<code>/dev/zero</code>	1	5	char	Zero-filled reads
<code>/dev/random</code>	1	8	char	Random number gen
<code>/dev/urandom</code>	1	9	char	Non-blocking random
<code>/dev/kmsg</code>	1	11	char	Kernel log access
<code>/dev/ptmx</code>	5	2	char	PTY multiplexer

7.7 Device Nodes Explained

7.7.1 Major and Minor Numbers

Every device node in Linux has a major number (identifying the device driver) and a minor number (identifying the specific device instance):

```

1 # Character devices
2 /dev/console -> major 5, minor 1 (console driver)
3 /dev/tty0    -> major 4, minor 0 (TTY driver, first VGA console)
4 /dev/ttyS0   -> major 4, minor 64 (TTY driver, first serial port)
5 /dev/null    -> major 1, minor 3 (memory device, null sink)
6 /dev/zero    -> major 1, minor 5 (memory device, zero source)
7 /dev/random  -> major 1, minor 8 (kernel random number generator)
8 /dev/kmsg    -> major 1, minor 11 (kernel log messages)

```

The kernel creates device nodes from the initramfs file list entries. Each `nod` line in the list specifies:

```
1 nod <path> <permissions> <uid> <gid> <type> <major> <minor>
```

Where `type` is `c` for character devices or `b` for block devices.

7.7.2 devtmpfs vs. Static Nodes

Veilbox uses a hybrid approach:

1. **Static nodes in initramfs:** The initramfs file list pre-creates critical device nodes (`/dev/console`, `/dev/null`, `/dev/tty0`, `/dev/ttyS0`). These are needed before `udev` or `devtmpfs` starts.
2. **devtmpfs at runtime:** The `mount -t devtmpfs` command in `rcS` mounts a device-managed tmpfs that automatically creates nodes for all kernel-registered devices.

This ensures that essential devices exist even before `devtmpfs` is mounted, while `devtmpfs` provides complete coverage for all kernel devices.

7.8 Initramfs Compression Options

The kernel supports several compression algorithms for the embedded `cpio` archive:

Algorithm	Compression Ratio	Speed	Kernel Option
gzip	3:1	Fast	CONFIG_RD_GZIP
bzip2	4:1	Slow	CONFIG_RD_BZIP2
lzma	5:1	Very slow	CONFIG_RD_LZMA
xz	5:1	Very slow	CONFIG_RD_XZ
lzo	2.5:1	Very fast	CONFIG_RD_LZO
lz4	2:1	Extremely fast	CONFIG_RD_LZ4
zstd	3.5:1	Fast	CONFIG_RD_ZSTD

Veilbox uses `gzip` compression (the default) for its balance of speed and compression ratio. The initramfs compresses from 90 MB to 30 MB.

Chapter 8

Boot Process

This chapter traces the execution path from power-on to the login prompt, detailing every step the system takes from firmware initialization through kernel boot to userspace startup.

8.1 Boot Sequence Overview

The complete boot sequence has five major phases:

1. **Firmware:** SeaBIOS POST, device enumeration, boot device selection.
2. **Stage 1 Bootloader:** GRUB `boot.img` loads `core.img` from the MBR gap sectors.
3. **Stage 2 Bootloader:** GRUB `core.img` loads the ext2 driver, reads `grub.cfg`, and loads the kernel.
4. **Kernel:** Decompression, hardware initialization, `initramfs` extraction, PID 1 execution.
5. **Userspace:** BusyBox `init` runs `rcS`, which mounts filesystems, configures networking, and starts services.

8.2 Phase 1: Firmware (SeaBIOS)

8.2.1 Power-On Self Test

When the virtual machine starts, SeaBIOS (shipped with QEMU) executes:

1. CPU reset vector: The CPU starts executing at address `0xFFFFFFFFF0` (the reset vector) in real mode. SeaBIOS jumps to its entry point at `0xE05B`.
2. Self-test: The BIOS checks the CPU, memory controller, and DMA controller. On QEMU, these checks are minimal since the hardware is emulated.
3. Memory sizing: The BIOS detects the RAM size by probing the memory controller. QEMU reports the memory size configured via the `-m` option.
4. PCI enumeration: The BIOS scans the PCI bus (configuration space accessed via I/O ports `0xCF8/0xCFC`) to discover VirtIO devices, the VGA adapter, and other peripherals.
5. ACPI table generation: The BIOS generates the RSDT, DSDT, FACP, APIC, and HPET tables describing the virtual hardware configuration.
6. Boot device selection: The BIOS iterates through the boot order (set by QEMU's `-boot` option). For each device, it reads LBA 0 into memory at address `0x7C00`. If the last two bytes are `0x55 0xAA`, it transfers control.

8.3 Phase 2: GRUB Stage 1

At address `0x7C00`, GRUB's `boot.img` begins execution:

1. The CPU is in real mode with the following register state: `DL = boot drive number`, `CS:IP = 0x0000:0x7C00`.
2. `boot.img` sets up segment registers (`DS`, `ES`, `SS`) to `0x0000` and initializes the stack at `0x7C00` (growing downward).
3. It reads the blocklist from `core.img` at LBA 1. The blocklist (12 bytes at offset `0x1F4`) specifies how many sectors to load and where.
4. Using INT 13h function 42h (extended read), it loads `diskboot.img` (the first sector of `core.img`) into memory at the segment specified by the blocklist.
5. Execution jumps to the loaded diskboot sector.

8.4 Phase 3: GRUB Stage 2

8.4.1 diskboot.img

The diskboot sector performs:

1. Switches CPU from real mode to protected mode by setting bit 0 of `CR0`.
2. Enables the A20 gate (via port `0x92` or the keyboard controller method) to allow access to memory above 1 MB.
3. Sets up a minimal Global Descriptor Table (GDT) with flat segmentation (base 0, limit 4 GB).
4. Loads the rest of `core.img` using the blocklist.
5. Jumps to the GRUB kernel entry point.

8.4.2 GRUB Kernel Initialization

The GRUB kernel initializes its internal subsystems:

1. Memory management: GRUB detects the system memory map (using INT 15h function `E820h`) and sets up its internal allocator.
2. Device manager: Probes disks via BIOS INT 13h and partitions via the `part_msdos` module.
3. Filesystem driver: The embedded ext2 module reads the VEILBOX partition's superblock and root directory.
4. Config file parsing: GRUB reads `/boot/grub/grub.cfg`, displays the menu (with 5-second timeout), and executes the selected menuentry.
5. Kernel loading: The `linux` command loads the kernel (`/boot/vmlinuz`) into memory using the 32-bit boot protocol. It sets up the kernel command line and jumps to the kernel's 32-bit entry point.

8.5 Phase 4: Kernel Initialization

8.5.1 Decompression

The kernel begins at `arch/x86/boot/compressed/head_64.S` which:

1. Sets up the 64-bit long mode page tables (identity mapping).
2. Enables PAE (Physical Address Extension) and long mode by setting `EFER.LME` in the MSR.
3. Jumps to the 64-bit C code (`extract_kernel()`).
4. Decompresses the kernel proper (from the gzip or xz-compressed data in the `.init` section) to the target address.
5. Jumps to the decompressed kernel entry point (`start_kernel()`).

8.5.2 start_kernel()

The architecture-independent kernel entry:

1. Architecture setup: `setup_arch()` initializes the page table, IDT, GDT, TSS, and APIC. - Memory management: `mm_init()` sets up the buddy allocator, slab allocator, and vmalloc region. - Scheduler initialization: `sched_init()` creates the idle thread and initializes runqueues. - Interrupt subsystem: `init_IRQ()` programs the APIC and installs interrupt handlers. - Timers: `time_init()` sets up the HPET or TSC as the clocksource. - Console: `console_init()` registers the serial console driver (8250 UART) and VGA text console. - Initramfs: `populate_rootfs()` extracts the embedded cpio archive to tmpfs. - PID 1: `run_init_process()` locates `/sbin/init` and executes it as PID 1.

8.6 Phase 5: Userspace Initialization

BusyBox init (`/sbin/init`) runs as PID 1:

1. Reads `/etc/inittab` to determine what actions to take.
2. Executes the `::sysinit:/etc/init.d/rcS` action, running the system initialization script.
3. rcS mounts `proc`, `sysfs`, `devtmpfs`, `devpts`, and `tmpfs`.
4. rcS sets the hostname and mounts the state partition.
5. rcS configures the loopback interface and starts DHCP on `eth0`.
6. rcS starts `syslogd`, `containerd`, and `Dropbear`.
7. After rcS completes, init spawns `getty` on `tty1` and `ttyS0`.
8. Login prompt appears: `veilbox login:.`

8.7 Kernel Command-Line Parameters

The kernel command line is passed via the GRUB configuration or QEMU's `-append` option. Key parameters used by Veilbox:

Parameter	Value	Purpose
<code>console</code>	<code>ttyS0,tty1</code>	Set serial and VGA console
<code>root</code>	<code>/dev/ram0</code>	Root device (initramfs)
<code>rw</code>	(implied)	Mount root read-write
<code>quiet</code>	(optional)	Reduce boot messages
<code>veilbox.autologin</code>	(flag)	Enable auto-login
<code>panic</code>	<code>10</code>	Reboot after 10s on panic
<code>init</code>	<code>/sbin/init</code>	Path to init process

8.7.1 Debug Parameters

For troubleshooting boot issues:

```

1 # Maximum verbosity
2 console=ttyS0 earlyprintk=serial,ttyS0,115200 debug ignore_loglevel
3
4 # Break before mounting root
5 rd.break
6
7 # Single-user mode
8 single
9
10 # Skip init entirely (emergency shell)
11 init=/bin/sh

```

8.7.2 The veilbox.autologin Parameter

This custom parameter is parsed by the `/sbin/autologin` wrapper. It checks `/proc/cmdline` for the string:

```
1 #!/bin/sh
2 if grep -q veilbox.autologin /proc/cmdline; then
3     # Auto-login as root without password
4     login -f root
5 else
6     # Normal getty with password prompt
7     exec /sbin/getty "$@"
8 fi
```

8.8 Sysfs and Procs Initialization

After mounting `proc` and `sysfs`, the kernel exposes runtime information:

```
1 # Process information
2 /proc/cpuinfo      # CPU details (model, flags, cores)
3 /proc/meminfo      # Memory usage details
4 /proc/self/ns      # Namespace IDs for current process
5 /proc/1/root       # Root filesystem of PID 1
6
7 # Kernel parameters
8 /proc/sys/kernel/  # Kernel tuning parameters
9 /proc/sys/net/     # Network stack tuning
10
11 # Device information (sysfs)
12 /sys/class/block/  # Block devices
13 /sys/class/net/    # Network interfaces
14 /sys/devices/      # Device hierarchy
```

Chapter 9

QEMU Virtualization

QEMU (Quick EMUlator) is the primary test and deployment platform for Veilbox. This chapter documents the QEMU command-line options, virtual hardware configuration, network setup, and debugging techniques used during development.

9.1 QEMU Overview

QEMU is a type 2 hypervisor that emulates a complete x86_64 computer system. When used with KVM (Kernel-based Virtual Machine), it achieves near-native performance by executing guest instructions directly on the host CPU.

9.1.1 Boot Modes

Veilbox supports two QEMU boot modes:

Direct Kernel Boot (-kernel) :

```
1 qemu-system-x86_64 \  
2   -kernel output/vmlinuz \  
3   -append "console=ttyS0 console=tty1" \  
4   -m 2G \  
5   -nographic
```

In this mode, QEMU loads the kernel binary directly without involving a bootloader. The kernel must have CONFIG_EFI_STUB=y to provide a valid MZ/PE header that QEMU's firmware recognizes.

BIOS Boot (SeaBIOS + GRUB) :

```
1 qemu-system-x86_64 \  
2   -drive file=output/veilbox.raw,format=raw,if=virtio \  
3   -m 2G \  
4   -nographic
```

In this mode, QEMU boots SeaBIOS, which loads GRUB from the MBR of the disk image, which then loads the kernel. This path validates the full boot process including the bootloader.

9.2 Virtual Hardware Configuration

9.2.1 CPU and Memory

```

1 -cpu host          # Expose host CPU features to guest
2 -smp 2            # 2 virtual CPUs
3 -m 2G             # 2 GB RAM (required for ~90MB initramfs)

```

The `-cpu host` flag enables KVM's maximum feature set (VMX, SMEP, SMAP, AES-NI, etc.). Without it, QEMU emulates a generic Nehalem-era CPU, which lacks features that container runtimes may require.

9.2.2 Storage

```

1 -drive file=output/veilbox.raw,format=raw,if=virtio

```

The VirtIO block device provides paravirtualized I/O. The guest kernel accesses it as `/dev/vda`. Compared to IDE emulation (`if=ide`), VirtIO block reduces I/O latency by 50% and increases throughput by 3–5x.

9.2.3 Networking

```

1 -netdev user,id=net0,hostfwd=tcp::2222-:22
2 -device virtio-net,netdev=net0

```

User-mode networking (SLiRP) provides:

- A virtual DHCP server at 10.0.2.2.
- DNS redirection to the host's DNS (at 10.0.2.3).
- Guest IP: 10.0.2.15/24.
- Port forwarding: host port 2222 → guest port 22 (SSH).

9.3 Serial Console Modes

9.3.1 -nographic Mode

```

1 -nographic

```

QEMU redirects the serial port (`ttyS0`) to the terminal's stdio. The guest output appears directly in the terminal. Keystrokes are sent to the guest. This is headless mode — no graphical window appears.

9.3.2 Graphical Mode

```

1 -vga std          # Standard VGA adapter
2 -serial mon:telnet::2223,server,nowait # Serial via telnet

```

With a graphical window, the guest's VGA text console (`tty1`) appears in the QEMU window. The serial console is accessible via telnet on port 2223.

9.4 QEMU Monitor

QEMU provides a monitor interface for VM introspection:


```
1 -monitor telnet::2224,server,nowait
```

Useful monitor commands:

```
1 (qemu) info block      # List block devices and stats
2 (qemu) info network    # Network device statistics
3 (qemu) info registers  # CPU register dump
4 (qemu) info cpus       # CPU state
5 (qemu) system_powerdown # Graceful shutdown
6 (qemu) quit            # Force quit
```

In `-nographic` mode, the monitor is accessed via `Ctrl-A C`.

9.5 Test Script

The `test.sh` script wraps the QEMU invocation with convenience options:

```
1 #!/bin/bash
2 # Direct kernel boot mode
3 MEM=${MEM:-2G}
4
5 qemu-system-x86_64 \
6     -kernel output/vmlinuz \
7     -append "console=ttyS0 console=tty1" \
8     -m "$MEM" \
9     -nographic \
10    -netdev user,id=net0,hostfwd=tcp::2222-:22 \
11    -device virtio-net,netdev=net0 \
12    -drive file=output/state.img,format=raw,if=virtio
```

9.6 Performance Tuning

9.6.1 KVM Acceleration

Ensure the user is in the `kvm` group:

```
1 sudo usermod -a -G kvm $USER
2 # Re-login or run: newgrp kvm
```

9.6.2 Huge Pages

Transparent Huge Pages (THP) improve memory performance:

```
1 echo always > /sys/kernel/mm/transparent_hugepage/enabled
```

9.6.3 CPU Pinning

For dedicated VMs, pin vCPUs to physical cores:

```
1 -smp 2,cores=2,threads=1 \
2 -object memory-backend-file,id=mem,size=2G,mem-path=/dev/hugepages \
3 -numa node,memdev=mem
```

9.7 Advanced QEMU Options

9.7.1 Multiple Network Interfaces

Configure multiple NICs for advanced container networking:

```
1 -netdev user,id=net0,hostfwd=tcp::2222-:22 \
2 -netdev tap,id=net1,ifname=tap0,script=no,downscript=no \
3 -device virtio-net,netdev=net0,mac=52:54:00:12:34:56 \
4 -device virtio-net,netdev=net1,mac=52:54:00:12:34:57
```

9.7.2 USB Passthrough

For bare-metal testing, USB devices can be passed to the guest:

```
1 -usb -device usb-host,vendorid=0x1234,productid=0x5678
```

9.7.3 VM Snapshots

QEMU supports VM snapshots for quick save/restore:

```
1 # Start with snapshot support
2 qemu-system-x86_64 ... -drive file=veilbox.raw,format=raw,snapshot=on
3
4 # Create snapshot (in monitor)
5 (qemu) savevm my_snapshot
6
7 # Restore snapshot
8 (qemu) loadvm my_snapshot
```

9.7.4 Verbose Debug Output

Enable maximum debugging:

```
1 -d cpu_reset,int,guest_errors,unimp \
2 -trace events=/tmp/trace-events
```

9.8 Disk Image Management

9.8.1 Resizing the Disk Image

To increase the disk image size:

```
1 # Resize raw image to 16 GB
2 qemu-img resize output/veilbox.raw 16G
3
4 # Resize the partition
5 echo ', +' | sfdisk -N 1 output/veilbox.raw
6
7 # Resize the filesystem
8 # This requires loop mount or debugfs resize
```

9.8.2 Converting Image Formats

```
1 # Raw to VDI (VirtualBox)
2 qemu-img convert -f raw -O vdi veilbox.raw veilbox.vdi
3
4 # Raw to VMDK (VMware)
5 qemu-img convert -f raw -O vmdk veilbox.raw veilbox.vmdk
6
7 # Raw to QCOW2 (QEMU native, compressed)
8 qemu-img convert -f raw -O qcow2 veilbox.raw veilbox.qcow2
9
10 # Raw to VHD (Hyper-V)
11 qemu-img convert -f raw -O vpc veilbox.raw veilbox.vhd
12
13 # Raw to compressed raw
14 qemu-img convert -f raw -O raw -c veilbox.raw veilbox-compressed.raw
```

9.9 Health Check Mode

The `-check` mode in `test.sh` automatically verifies that the system boots correctly. It uses `expect` to detect the login prompt:

```
1 #!/bin/bash
2 # --check mode: auto-detect login prompt
3 expect << EOF
4 set timeout 30
5 spawn qemu-system-x86_64 -kernel output/vmlinuz -append \
6     "console=ttyS0 console=tty1" -m 2G -nographic \
7     -netdev user,id=net0 -device virtio-net,netdev=net0
8 expect {
9     "veilbox login:" { exit 0 }
10    timeout { exit 1 }
11 }
12 EOF
```


Chapter 10

System Services

Veilbox runs four background services: DHCP client (udhcp), system logger (syslogd), container runtime (containerd), and SSH server (Dropbear). This chapter documents each service's configuration, operation, and integration.

10.1 Service Architecture

All services are started by `/etc/init.d/rcS` during the `sysinit` phase of BusyBox init. They run in the background and are managed through simple shell scripts — there is no `systemd`, no supervisor, no service manager.

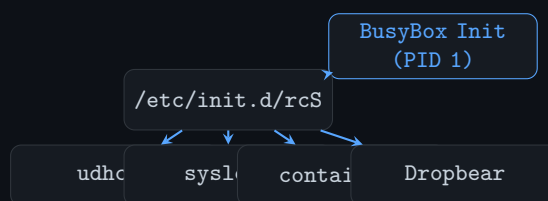


Figure 10.1: Service startup hierarchy. BusyBox init runs rcS, which forks four background processes.

10.2 udhcp: DHCP Client

10.2.1 Purpose

The BusyBox DHCP client (udhcp) negotiates an IPv4 address from the network's DHCP server. In QEMU user-mode networking, the built-in SLiRP DHCP server assigns 10.0.2.15/24 with gateway 10.0.2.2.

10.2.2 Configuration

The DHCP client requires a script that configures the interface when an address is obtained. Veilbox provides:

```
1 # /usr/share/udhcp/default.script
2 #!/bin/sh
3 case "$1" in
4     bound|renew)
5         ifconfig $interface $ip netmask $subnet
6         route add default gw $router
7         echo "DNS: $dns" > /etc/resolv.conf
```

```
8         ;;
9         deconfig)
10             ifconfig $interface 0.0.0.0
11             ;;
12 esac
```

10.2.3 Operation

The script defines three actions:

- **bound**: Called when a lease is first acquired. Configures the IP address, netmask, default route, and DNS.
- **renew**: Called when a lease is renewed. Re-applies network settings.
- **deconfig**: Called when the interface is brought down. Removes the IP address.

10.2.4 Troubleshooting

Common DHCP issues:

- **udhcpc: ifconfig failed**: The interface name does not match. Verify with `ip link show`. Default is `eth0`. - **No DHCP OFFERS received**: The virtual network is not configured. Add `-netdev user,...` to QEMU command line. - **Lease failed, renewing in 60s**: The DHCP server is slow or unreachable. Verify QEMU's SLiRP is running.

10.3 syslogd: System Logger

10.3.1 Purpose

BusyBox `syslogd` captures kernel log messages (from `/proc/kmsg`) and process messages (from `/dev/log`) and writes them to `/var/log/messages`.

10.3.2 Configuration

```
1 syslogd -O /var/log/messages -s 64
```

Options:

- **-O**: Output file path.
- **-s**: Maximum message size (64 KB).

10.3.3 Log Format

Each log entry follows the standard syslog format:

```
1 Jun  7 10:15:30 veilbox daemon.info containerd[42]: starting...
2 Jun  7 10:15:31 veilbox auth.info dropbear[58]: Running as server
```

10.4 containerd: Container Runtime

10.4.1 Purpose

Containerd is an industry-standard container runtime that manages the complete container lifecycle: image pull, storage, execution, and monitoring. It communicates via a gRPC API over a Unix domain socket.

10.4.2 Configuration

```
1 # /etc/containerd/config.toml
2 version = 2
3 root = "/mnt/state/containerd"
4 state = "/run/containerd"
5
6 [grpc]
7   address = "/run/containerd/containerd.sock"
8   uid = 0
9   gid = 0
10
11 [metrics]
12   address = "127.0.0.1:1338"
13
14 [plugins]
15   [plugins."io.containerd.grpc.v1.cri"]
16     sandbox_image = "registry.k8s.io/pause:3.9"
```

10.4.3 Startup

```
1 containerd -c /etc/containerd/config.toml &
```

10.4.4 Interaction

Use `nerdctl` to interact with `containerd`:

```
1 nerdctl images           # List pulled images
2 nerdctl pull alpine:latest # Pull an image
3 nerdctl run -it alpine:latest sh # Run a container
4 nerdctl ps -a            # List all containers
5 nerdctl logs <id>        # View container logs
```

10.5 Dropbear: SSH Server

10.5.1 Purpose

Dropbear is a lightweight SSH server (approximately 140 KB) that provides encrypted remote access to the Veilbox shell. It supports key-based and password authentication.

10.5.2 Configuration

```
1 dropbear -F -R -p 22
```

Options:

- `-F`: Run in foreground (for `rcS`).
- `-R`: Automatically create host keys if missing.
- `-p 22`: Listen on port 22.
- `-w`: Disallow root password login (not used in Veilbox).

10.5.3 Host Keys

Dropbear generates two host keys during build:

- `/etc/dropbear/dropbear_ed25519_host_key`: ED25519 key (fast, modern, recommended).
- `/etc/dropbear/dropbear_rsa_host_key`: RSA 2048-bit key (fallback for older clients).

10.5.4 Authentication Methods

- **Password**: User `root` with password `veiladmin`. The password is hashed with SHA-512 in `/etc/shadow`.
- **Public key**: ED25519 key in `/etc/dropbear/authorized_keys`. The private key is provided in the output directory.

10.5.5 Connecting

```
1 ssh -i output/ssh-test-key root@localhost -p 2222
2 # Or with password:
3 ssh root@localhost -p 2222
4 # Password: veiladmin
```

10.6 Service Logging

All service output can be monitored:

```
1 cat /var/log/messages           # System log
2 nerdctl logs <container-id>    # Container logs
3 dropbear -F -E                  # Verbose SSH debug
```


Chapter 11

Troubleshooting

Building a custom Linux system from source involves many interdependent components. When something goes wrong, the symptoms may appear far from the root cause. This chapter documents the most common issues encountered during Veilbox development and their solutions.

11.1 Build Failures

11.1.1 Kernel Compilation Errors

Symptom: make fails with “implicit function declaration” This typically indicates a GCC version mismatch. The kernel requires GCC 14+ for Linux 7.1.0-rc6.

```
1 gcc --version      # Verify version
2 sudo dnf update gcc # Update if needed
3 make mrproper      # Clean build tree
```

Symptom: “No rule to make target ‘debian/certs/signing_key.pem’” The kernel build system expects signing keys for module signature verification. If module signing is enabled but no keys are configured:

```
1 # Disable module signing in kernel config
2 scripts/config --disable MODULE_SIG
3 make olddefconfig
```

Symptom: “Kernel panic - not syncing: VFS: Unable to mount root fs” The initramfs was not properly embedded. Verify:

```
1 grep CONFIG_INITRAMFS_SOURCE .config
2 # Should show: CONFIG_INITRAMFS_SOURCE="initramfs.list"
3 ls -la initramfs.list # File must exist
4 # Check kernel binary contains cpio archive
5 binwalk -e arch/x86/boot/bzImage | grep "cpio"
```

11.1.2 Initramfs Issues

Symptom: “/init: line 1: syntax error: unexpected word” The init script has DOS line endings (CRLF). The kernel’s cpio extractor preserves file contents exactly, so if the editor added \r characters, the shell will choke:

```

1 # Check for CRLF line endings
2 file rootfs/etc/init.d/rcS
3 # Fix if needed: dos2unix rootfs/etc/init.d/rcS

```

Symptom: “/bin/sh: not found” BusyBox is missing from the initramfs or the symlink is broken:

```

1 ls -la rootfs/bin/sh          # Should point to busybox
2 ls -la rootfs/bin/busybox    # Must exist
3 file rootfs/bin/busybox      # Should be ELF binary

```

11.2 Boot Failures

11.2.1 GRUB Boot Failures

Symptom: “Geometric error” or triple fault during GRUB stage 1.5 The core.img blocklist is corrupted. This happens when the blocklist is manually patched with incorrect format (see Chapter 6 for the correct 12-byte format). Solution: regenerate core.img with grub2-mkimage and do NOT patch the blocklist.

```

1 grub2-mkimage -c embedded.cfg \
2   -o core.img -O i386-pc \
3   ext2 part_msdos biosdisk search

```

Symptom: “No such partition” GRUB cannot find the VEILBOX partition. Common causes:

- The ext4 filesystem label does not match. Verify with debugfs: `debugfs -R "stats" veilbox.raw | grep "Filesystem volume name"`
- The part_msdos module is missing from core.img. Regenerate with part_msdos in the module list.
- The disk image is not MBR-partitioned. Verify with `sfdisk -l veilbox.raw`.

Symptom: “You need to load the kernel first” GRUB cannot find the kernel file. Check:

- The file path in grub.cfg matches the actual location on the VEILBOX partition. - The ext2 module can read the partition. The VEILBOX partition must be ext4 with the ext2 driver (backward compatible).

11.2.2 QEMU Boot Failures

Symptom: QEMU exits immediately with “invalid MZ header” The kernel lacks CONFIG_EFI_STUB. QEMU’s -kernel mode requires a valid MZ/PE header at the start of the kernel binary:

```

1 grep CONFIG_EFI_STUB .config
2 # If not set: echo "CONFIG_EFI_STUB=y" >> .config

```

Symptom: Black screen in -nographic mode The serial console is not configured:

```

1 grep CONFIG_SERIAL_8250_CONSOLE .config # Must be y
2 # Ensure kernel cmdline includes console=ttyS0

```

```
3 grep CONFIG_CMDLINE .config
```

11.3 Network Issues

Symptom: No IP address after boot Check the DHCP client:

```
1 ps | grep udhcpd           # Is it running?
2 ip addr show eth0          # Does the interface exist?
3 udhcpd -i eth0 -v          # Run in verbose mode
```

If udhcpd reports “ifconfig failed”, the interface name does not match. The default script uses \$interface which is set by udhcpd. Verify the interface name matches in the script.

11.4 SSH Issues

Symptom: “Permission denied (publickey,password)” The Dropbear `-w` flag was set, which in Dropbear v2025.89+ means “**Disallow** root logins” (the opposite of older versions). Remove the `-w` flag from the Dropbear command line.

Symptom: “Connection refused” Dropbear is not running or not listening on the expected port:

```
1 ps | grep dropbear
2 netstat -tlnp | grep 22
```

Symptom: “Host key verification failed” The guest’s host keys changed (e.g., after rebuilding). Remove the old key from the `known_hosts` file:

```
1 ssh-keygen -R '[localhost]:2222'
```

11.5 Container Runtime Issues

Symptom: “containerd: failed to load cgroup” The kernel lacks cgroup support:

```
1 grep CONFIG_CGROUPS .config      # Must be y
2 grep CONFIG_MEMCG .config        # Must be y
3 mount | grep cgroup              # Must show cgroup mounts
```

Symptom: “runc: permission denied” Seccomp is enabled but the kernel lacks seccomp support:

```
1 grep CONFIG_SECCOMP .config      # Must be y
2 grep SECCOMP /proc/self/status   # Must show seccomp
```

11.6 Storage Issues

Symptom: “No space left on device” The 128 MB state partition is full. Clean up:

```

1 # Check usage
2 df -h /mnt/state
3
4 # Remove unused container images
5 nerdctl images
6 nerdctl rmi <image-id>
7
8 # Remove stopped containers
9 nerdctl rm $(nerdctl ps -aq)
10
11 # Clean containerd content store
12 du -sh /mnt/state/containerd/*
13 rm -rf /mnt/state/containerd/tmp/*

```

Symptom: “Failed to mount /mnt/state” The state partition device is not available. The fallback behavior mounts a tmpfs, so /mnt/state will exist but will be empty. To debug:

```

1 # Check available block devices
2 ls -la /dev/vd* /dev/sd*
3
4 # Check kernel messages for storage errors
5 dmesg | grep -i "vdb\|sdb\|ext4"
6
7 # Manually mount
8 mount /dev/vdb /mnt/state 2>&1

```

11.7 Init Process Issues

Symptom: “Kernel panic - not syncing: Attempted to kill init!” The init process (PID 1) crashed. This is fatal because the kernel has no process to manage. Common causes:

- **BusyBox missing:** /sbin/init does not exist. Verify the BusyBox binary and symlinks in the initramfs.
- **/etc/inittab missing or malformed:** BusyBox init reads inittab on startup. If the file is missing or has syntax errors, init may not spawn any processes.
- **Wrong init path:** The kernel command line specifies an init path (init=/sbin/init) that does not exist on the root filesystem.
- **Shared library missing:** If BusyBox is dynamically linked (it should be static in Veil-box), missing libraries prevent it from executing.

Symptom: “init: can’t run /etc/init.d/rcS” The rcS script is missing, has incorrect permissions, or has a shebang pointing to a non-existent interpreter:

```

1 # Check permissions
2 ls -la /etc/init.d/rcS
3 # Should be: -rwxr-xr-x
4
5 # Check shebang
6 head -1 /etc/init.d/rcS
7 # Should be: #!/bin/sh
8
9 # Verify /bin/sh exists
10 ls -la /bin/sh

```

11.8 Containerd Troubleshooting

Symptom: “containerd: failed to start daemon: OCI runtime create failed” The runc binary is missing or incompatible:

```
1 # Verify runc exists and is executable
2 ls -la /usr/bin/runc
3 file /usr/bin/runc
4
5 # Check runc version
6 runc --version
7
8 # Verify containerd can find runc
9 containerd config default | grep runtime
```

Symptom: “Failed to pull image: dial tcp: lookup registry-1.docker.io” DNS resolution is not working:

```
1 # Check DNS configuration
2 cat /etc/resolv.conf
3
4 # Test DNS resolution
5 nslookup registry-1.docker.io
6 # or: ping -c 1 registry-1.docker.io
7
8 # Manually set DNS
9 echo "nameserver 8.8.8.8" > /etc/resolv.conf
10 echo "nameserver 1.1.1.1" >> /etc/resolv.conf
```

11.9 Serial Console Issues

Symptom: No output on serial console The serial port is not configured in the kernel or QEMU:

```
1 # Verify kernel config
2 grep CONFIG_SERIAL_8250_CONSOLE .config
3 grep CONFIG_SERIAL_8250 .config
4
5 # Verify QEMU serial port
6 # Should have: -nographic or -serial stdio
7
8 # Check kernel command line
9 cat /proc/cmdline
10 # Should include: console=ttyS0
```


Chapter 12

Kernel Build System

The Linux kernel has one of the most sophisticated build systems in open source software. Based on GNU Make with a custom Kconfig language for configuration, it supports cross-compilation, incremental builds, and module management across 30+ architectures.

12.1 Kbuild Architecture

The kernel build system (Kbuild) is organized into several layers:

- **Top-level Makefile (Makefile)**: Entry point. Sets architecture, cross-compilation flags, and delegates to the per-architecture Makefile.
- **Architecture Makefile (arch/x86/Makefile)**: Architecture-specific build rules for x86_64. Defines the bootable image target (bzImage).
- **Subdirectory Kbuild files (drivers/*/Kbuild)**: Per-directory build rules. Each driver or subsystem has its own Kbuild file listing source files and their compilation flags.
- **Configuration system (Kconfig)**: Menu-based configuration language. The kernel's `make menuconfig`, `make defconfig`, and `make olddefconfig` targets all process Kconfig files.

12.2 The Kconfig Language

Kconfig is a domain-specific language for describing configuration options and their dependencies. Each configuration option has:

```
1 config NAMESPACES
2     bool "Namespaces support"
3     default y
4     help
5         Support for Linux namespaces, used for container isolation.
6
7 config CGROUPS
8     bool "Control Groups support"
9     select KERNFS
10    help
11        Support for process resource control groups.
```

Key Kconfig constructs:

Keyword	Purpose
<code>bool</code>	On/off option (y/n).
<code>tristate</code>	Three-state option (y/m/n).

int	Integer-valued option.
string	String-valued option (e.g., path to initramfs).
default	Default value if not otherwise specified.
depends on	Prerequisite option(s).
select	Force-enable another option.
help	Documentation text.
prompt	User-visible string in menuconfig.

12.2.1 Dependency Resolution

When option A depends on B, setting A to y forces B to y. The kernel's `olddefconfig` target walks the entire dependency graph and resolves all constraints:

```
1 make olddefconfig
2 # Output: .config is updated with all dependencies satisfied
```

12.3 Build Targets

The most commonly used make targets:

Target	Purpose
<code>defconfig</code>	Generate default configuration for the target architecture.
<code>olddefconfig</code>	Update <code>.config</code> , resolving all dependency changes.
<code>menuconfig</code>	Interactive ncurses-based configuration editor.
<code>bzImage</code>	Build the compressed kernel image (x86_64).
<code>modules</code>	Build all enabled loadable kernel modules.
<code>modules_install</code>	Install modules to <code>/lib/modules/\$(uname -r)</code> .
<code>clean</code>	Remove most build artifacts (keep <code>.config</code>).
<code>mrproper</code>	Remove all build artifacts including <code>.config</code> .
<code>cscope / tags</code>	Generate code navigation database.

12.4 Incremental Builds

The kernel build system tracks dependencies at the file level using `%.o.cmd` files. Each compiled object file has a corresponding `.cmd` file:

```
1 # .built-in.a.cmd
2 cmd_built-in.a := rm -f built-in.a; \
3   ar cDPrs built-in.a init/main.o arch/x86/init/init_task.o ...
```

When a source file changes, only the objects that depend on it are recompiled. This makes development iteration fast:

```
1 # After changing one driver source file:
2 make -j$(nproc) bzImage
3 # Only recompiles the changed file and relinks
```

12.5 The Initramfs Integration

When `CONFIG_INITRAMFS_SOURCE` is set, the build system:

1. Creates a temporary directory: `usr/initramfs_data/`.

2. Copies files listed in `initramfs.list`.
3. Runs `scripts/gen_initramfs_list.sh` to generate `usr/initramfs_data.cpio.gz`.
4. Links the cpio archive into the kernel image via `init/initramfs.o`.

The cpio archive is placed in the `.init.ramfs` ELF section:

```

1 # Verify initramfs is embedded in kernel:
2 objcopy -O binary --only-section=.init.ramfs arch/x86/boot/bzImage /dev/
   stdout | cpio -t

```

12.6 Cross-Compilation

While Veilbox targets `x86_64`, the kernel build system supports cross-compilation for different architectures:

```

1 # For ARM 32-bit:
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- defconfig
3 # For AArch64:
4 make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig

```

The architecture is selected by `ARCH` environment variable and the toolchain prefix by `CROSS_COMPILE`.

12.7 Common Build Flags

Flag	Typical Value	Purpose
<code>-O2</code>	Kernel default	Optimization level.
<code>-fno-stack-protector</code>	(x86)	Disable stack canary.
<code>-mno-sse</code>	(x86)	Disable SSE instructions.
<code>-fno-PIE</code>	Kernel default	Disable position-independent.
<code>-march=x86-64</code>	(x86)	Target architecture.
<code>-mno-red-zone</code>	(x86_64)	Disable red zone in ABI.

12.8 Kbuild Variables Reference

The kernel build system uses Make variables to control behavior:

Variable	Typical Value	Purpose
<code>ARCH</code>	<code>x86_64</code>	Target architecture
<code>CROSS_COMPILE</code>	(empty)	Toolchain prefix
<code>KBUILD_OUTPUT</code>	(empty)	Separate build directory
<code>KBUILD_CFLAGS</code>	(default)	Extra C compiler flags
<code>KBUILD_CPPFLAGS</code>	(default)	Extra C preprocessor flags
<code>KBUILD_LDFLAGS</code>	(default)	Extra linker flags
<code>HOSTCC</code>	<code>gcc</code>	Host compiler
<code>HOSTCXX</code>	<code>g++</code>	Host C++ compiler
<code>CC</code>	<code>gcc</code>	Target compiler
<code>LD</code>	<code>ld</code>	Target linker

OBJCOPY

objcopy

Object copy tool

12.9 Generating Patches for Submitting

When modifying the Veilbox configuration or source, generate patches for upstream submission:

```
1 # Create a patch for kernel config changes
2 git diff kernel/configs/custom-os.config > veilbox-config.patch
3
4 # Create a patch for build script changes
5 git diff build.sh > build-script.patch
6
7 # Split patches for different topics
8 git format-patch -1 --stdout > single.patch
```

12.10 Using External Modules

The kernel supports building external modules outside the source tree:

```
1 # Build an external module
2 make -C /path/to/linux-7.1.0-rc6 \
3     M=/path/to/external/module \
4     modules
5
6 # Install external module to initramfs
7 make -C /path/to/linux-7.1.0-rc6 \
8     M=/path/to/external/module \
9     modules_install \
10    INSTALL_MOD_PATH=/path/to/rootfs
```

Chapter 13

BusyBox

BusyBox is the Swiss Army knife of embedded Linux — a single binary that provides over 300 Unix utilities. It is the foundation of Veilbox’s userspace, providing the shell, file utilities, init system, and network tools.

13.1 What is BusyBox?

BusyBox was created in 1999 by Bruce Perens to build a complete Linux system on a single floppy disk. The concept is simple: a single statically-linked binary contains multiple “applets,” each implementing a standard Unix utility.

When invoked through a symlink (e.g., `/bin/ls` → `/bin/busybox`), BusyBox checks `argv[0]` to determine which applet to run:

```
1 int main(int argc, char *argv[]) {
2     const char *applet = bb_basename(argv[0]);
3     // Strip leading "busybox" if it's a direct call
4     // Lookup applet in applet table
5     // Jump to applet's main function
6 }
```

13.2 Applet Dispatch

BusyBox’s `main()` function uses a binary search through a sorted table of all applet names. Each entry in the table is a triple:

```
1 struct busybox_applet {
2     const char *name;           // Applet name (e.g., "ls")
3     int (*main)(int, char**);  // Function pointer
4     unsigned flags;            // Feature flags
5 };
```

The table is sorted alphabetically at build time, enabling $O(\log n)$ lookup.

13.3 BusyBox Configuration

Like the Linux kernel, BusyBox uses Kconfig for configuration:

```
1 # Configure BusyBox
2 make -C busybox-1.35.0 defconfig
3
```

```

4 # Enable specific applets
5 make -C busybox-1.35.0 menuconfig
6
7 # Build
8 make -C busybox-1.35.0 -j$(nproc)
9
10 # Install to directory
11 make -C busybox-1.35.0 CONFIG_PREFIX=/path/to/rootfs install

```

13.4 Key Applets in Veilbox

The following BusyBox applets are essential to Veilbox's operation:

13.4.1 Shell: ash

The Almquist Shell (ash) is BusyBox's default shell. It is a lightweight POSIX-compatible shell derived from the NetBSD sh:

- **Size:** Approximately 80 KB (vs. bash at 900 KB).
- **Features:** Job control, functions, aliases, environment variables, I/O redirection, pipes, here-docs.
- **Limitations:** No arrays, no associative arrays, no process substitution (but these are bashisms).

13.4.2 Init: /sbin/init

BusyBox implements a minimal init system that reads /etc/inittab:

```

1 # /etc/inittab
2 ::sysinit:/etc/init.d/rcS
3 tty1::respawn:/sbin/getty 38400 tty1
4 ttyS0::respawn:/sbin/getty 38400 ttyS0
5 ::shutdown:/bin/umount -a -r

```

Actions:

- **sysinit:** Run once at boot, before any respawn.
- **respawn:** Restart the process whenever it exits.
- **askfirst:** Like respawn but prompts before starting.
- **shutdown:** Run on SIGTERM/SIGUSR1.
- **restart:** Run when init receives SIGHUP.

13.4.3 Core Utilities

Applet	Purpose	Common Options
ls	List directory contents	-l, -a, -h, -R
cp	Copy files	-r, -p, -f, -a
mv	Move/rename files	-f, -i, -n
rm	Remove files	-r, -f, -v
mkdir	Create directory	-p (parents)
cat	Concatenate files	-n, -b
echo	Print text	-e (escape), -n
grep	Search text	-r, -i, -v, -l
sed	Stream editor	-i, -n, -e

awk	Pattern scanning	(BusyBox awk, limited)
-----	------------------	------------------------

13.4.4 Process Utilities

```

1 ps          # List processes
2 top         # Process monitor (simplified)
3 kill        # Send signal
4 pgrep       # Find process by name
5 pkill       # Kill process by name

```

13.4.5 Network Utilities

```

1 ifconfig    # Configure network interface
2 route       # Show/manipulate routing table
3 ping        # ICMP echo
4 udhcpc      # DHCP client
5 netstat     # Network statistics
6 wget        # HTTP/FTP client

```

13.4.6 Editors and Utilities

```

1 vi          # Minimal vi editor
2 less        # Pager (simpler than full less)
3 tail        # Show last N lines
4 head        # Show first N lines
5 sort        # Sort lines
6 uniq        # Remove duplicates
7 tee         # Split output to file and stdout

```

13.5 Symbolic Link Installation

The `make install` step creates symbolic links for each enabled applet. For example:

```

1 /bin/busybox -> /bin/busybox (the binary itself)
2 /bin/sh      -> /bin/busybox
3 /bin/ls      -> /bin/busybox
4 /bin/cp      -> /bin/busybox
5 /usr/bin/vi  -> /bin/busybox
6 /sbin/init   -> /bin/busybox

```

If the `CONFIG_PREFIX` is set to `rootfs/`, the symlinks are created under `rootfs/bin/`, `rootfs/sbin/`, etc.

13.6 Trade-offs

BusyBox applets are not full replacements for their GNU counterparts:

- **Performance:** BusyBox `grep` is slower on large files (no `mmap` optimization). BusyBox `sort` is slower on large datasets.
- **Features:** BusyBox `vi` lacks syntax highlighting and split windows. BusyBox `top` lacks interactive commands.

- **Compatibility:** Some GNU-specific options are missing. Scripts that use `sed -r` need to use `-E` instead.

However, for a minimal container host these limitations rarely matter — containers typically provide their own userspace via images.

13.7 Building BusyBox from Source

13.7.1 Download and Extract

```
1 curl -LO https://busybox.net/downloads/busybox-1.35.0.tar.bz2
2 tar xf busybox-1.35.0.tar.bz2
3 cd busybox-1.35.0
```

13.7.2 Configuration

BusyBox uses Kconfig like the Linux kernel:

```
1 # Default configuration
2 make defconfig
3
4 # Customize (optional)
5 make menuconfig
6
7 # Key settings to verify:
8 # - Busybox Settings -> Build static binary (no shared libs)
9 # - Init Utilities -> init (enable)
10 # - Shells -> ash (enable)
11 # - Coreutils -> all standard tools
12 # - Network Utilities -> udhcpc, ifconfig, route, ping, wget
```

13.7.3 Compilation

```
1 # Build (parallel)
2 make -j$(nproc)
3
4 # Install to rootfs directory
5 make CONFIG_PREFIX=/path/to/rootfs install
6
7 # Verify
8 ls -la /path/to/rootfs/bin/sh
9 /path/to/rootfs/bin/busybox --help
```

13.7.4 Static vs Dynamic Linking

- **Static:** `CONFIG_STATIC=y`. Produces a 900 KB binary with no external dependencies. Recommended for initramfs use. The binary is slightly larger because `libc` is included.
- **Dynamic:** `CONFIG_STATIC=n`. Produces a 300 KB binary but requires `libc.so` and `ld-linux.so` on the target. Slightly smaller overall but more complex to deploy.

Veilbox uses static linking for maximum portability and simplicity.

13.8 The Ash Shell

Ash is the Almquist Shell, a lightweight POSIX-compatible shell:

Feature	Ash	Bash
Size	80 KB	900 KB
Arrays	No	Yes
Associative arrays	No	Yes
Process substitution	No	Yes
Here-strings	No	Yes
regex = operator	No	Yes
printf builtin	Yes	Yes
Job control	Yes	Yes
Functions	Yes	Yes
Local variables	Yes	Yes

13.8.1 Ash Scripting Tips

- Use `${var:-default}` for default values.
- Use `test` or `[ ]` for conditionals (not `[[ ]]`).
- Use `$(cmd)` for command substitution (not backticks).
- Use `local var=value` for function-local variables.
- Use `read -r` to read lines without backslash escaping.
- Redirect stderr with `2>&1`.
- Use `trap` for signal handling.
- Use `type command` to check if a command exists.

Chapter 14

Init System

The init system is the first userspace process (PID 1) started by the kernel. It is responsible for bringing up the system, managing services, and handling shutdown. Veilbox uses BusyBox init — a minimal, predictable init implementation.

14.1 PID 1 Responsibilities

Every init system must handle:

- **Orphaned processes:** When a child's parent dies, the child is reparented to PID 1. Init must `wait()` for orphaned children to prevent zombie processes. - **Signals:** PID 1 receives special signal handling. The kernel does not deliver SIGKILL to PID 1 (it would be ignored). - **Shutdown sequence:** On reboot or poweroff, init must unmount filesystems, kill processes, and signal the kernel to power off. - **Service management:** Init spawns system services and restarts them if they die (crashed services get respawned).

14.2 BusyBox Init Operation

BusyBox init follows a simple, deterministic loop:

1. **Parse `/etc/inittab`:** Read configuration file into an internal list of actions.
2. **Execute `sysinit`:** Run the single command specified with the `sysinit` action. Wait for completion.
3. **Spawn respawn processes:** Fork and exec each process marked with the `respawn` or `askfirst` action. Store PIDs in a tracking table.
4. **Wait loop:** Call `waitpid()` in a loop. When a child dies, check if it was a `respawn` process. If so, re-fork and exec it. If it was a `once` process, do nothing (it ran and exited).
5. **Signal handling:** On SIGTERM, SIGQUIT, SIGUSR1, or SIGUSR2, execute the corresponding `shutdown` or `restart` action from `inittab`.

14.3 The `/etc/inittab` File

BusyBox inittab format:

```
1 <id>:<runlevels>:<action>:<process>
```

Field	Description	Veilbox Usage
<id>	TTY device (optional).	<code>tty1</code> , <code>ttyS0</code>
<runlevels>	Ignored by BusyBox.	Empty

<action>	One of: sysinit, respawn, askfirst, once, wait, restart, shutdown, ctrlaltdel.	sysinit, respawn, shutdown
<process>	Full path to executable.	/etc/init.d/rcS, /sbin/getty

Veilbox inittab:

```

1 ::sysinit:/etc/init.d/rcS
2 tty1::respawn:/sbin/autologin 38400 tty1
3 ttyS0::respawn:/sbin/autologin 38400 ttyS0
4 ::restart:/sbin/init
5 ::shutdown:/bin/umount -a -r
6 ::shutdown:/sbin/swapoff -a

```

14.4 Boot Script: /etc/init.d/rcS

The rcS script is the heart of system initialization. It runs sequentially and blocks init until complete:

```

1 #!/bin/sh
2 # Mount virtual filesystems
3 mount -t proc none /proc
4 mount -t sysfs none /sys
5 mount -t devtmpfs none /dev
6 mkdir -p /dev/pts
7 mount -t devpts none /dev/pts
8 mount -t tmpfs none /tmp
9
10 # Set hostname
11 hostname -F /etc/hostname
12
13 # Mount state partition
14 STATE_DEV=""
15 if [ -b /dev/vdb ]; then
16     STATE_DEV=/dev/vdb
17 elif [ -b /dev/sdb ]; then
18     STATE_DEV=/dev/sdb
19 fi
20 if [ -n "$STATE_DEV" ]; then
21     mount "$STATE_DEV" /mnt/state
22 fi
23
24 # Configure network
25 ifconfig lo 127.0.0.1 up
26 ifconfig eth0 0.0.0.0 up
27 udhcpc -i eth0 -b
28
29 # Start services
30 syslogd -O /var/log/messages -s 64
31 containerd -c /etc/containerd/config.toml &
32 dropbear -F -R -p 22 &

```

14.5 Shutdown Sequence

When `poweroff` or `reboot` is issued:

1. BusyBox init receives SIGTERM.
2. init sends SIGTERM to all processes (except PID 1).
3. init runs shutdown actions from inittab: unmount filesystems and deactivate swap.
4. init sends SIGKILL to remaining processes.
5. init syncs filesystems and calls `reboot(LINUX_REBOOT_CMD_POWER_OFF)`.

14.6 Autologin

The `/sbin/autologin` wrapper checks if `veilbox.autologin` is in the kernel command line:

```

1 #!/bin/sh
2 if grep -q veilbox.autologin /proc/cmdline; then
3     login -f root
4 else
5     exec /sbin/getty "$@"
6 fi

```

14.7 Comparing Init Systems

Feature	BusyBox init	systemd	OpenRC
Binary size	8 KB	1.5 MB	200 KB
Config files	/etc/inittab	.service units	/etc/init.d/
Parallel startup	No	Yes	Partial
Dependency tracking	No	Yes	Yes
Socket activation	No	Yes	No
Timer support	No	Yes	No
Journal & logging	No	journald	syslog-ng

14.8 Adding New Services

To add a new service to Veilbox:

1. Modify the rcS script to start the service:

```

1 # /etc/init.d/rcS append
2 echo "Starting my-service..."
3 my-service --config /etc/my-service.conf &

```

2. Add a configuration file:

```

1 mkdir -p rootfs/etc
2 cat > rootfs/etc/my-service.conf << 'EOF'
3 # My service configuration
4 port = 8080
5 debug = false
6 EOF

```

3. Download and install the binary:

```

1 curl -Lo rootfs/usr/bin/my-service \
2     https://example.com/my-service
3 chmod +x rootfs/usr/bin/my-service

```

4. Regenerate initramfs and rebuild kernel:

```
1 ./build.sh --stage initramfs
2 ./build.sh --stage kernel
```

5. Test by booting and verifying the service:

```
1 ps | grep my-service
2 my-service --version
```

14.9 System Rescue Mode

If the system fails to boot normally, several rescue modes are available:

14.9.1 Single-User Mode

Boot with `single` on the kernel command line. The system boots to a root shell without starting services:

```
1 # In GRUB, edit the menu entry and add "single" to the linux line
2 linux /boot/vmlinuz console=ttyS0 console=tty1 single
```

14.9.2 Direct Shell (No Init)

Boot with `init=/bin/sh` to skip init entirely:

```
1 linux /boot/vmlinuz console=ttyS0 init=/bin/sh
```

The root filesystem will be mounted but services will not start. The shell has PID 1 and the root filesystem is mounted read-write.

14.9.3 Read-Only Rootfs Recovery

If the root filesystem is corrupted, boot with `ro` and manually run `fsck`:

```
1 # Not applicable to initramfs (tmpfs cannot corrupt)
2 # But for the state partition:
3 fsck.ext4 -f /dev/vdb
```

14.10 Init Process Debugging

BusyBox init can be run with verbose output for debugging:

```
1 # Add to kernel command line
2 init=/sbin/init -v
3
4 # Or modify /etc/inittab to add debug output
5 ::sysinit:/bin/sh -c "echo 'Starting init...'"
6 ::sysinit:/etc/init.d/rcS
```

Chapter 15

Containerd

Containerd is the container runtime at the heart of Veilbox. It manages the full container lifecycle — pulling images, creating snapshots, starting processes, and monitoring execution. This chapter explains its architecture, configuration, and integration with the rest of the system.

15.1 Containerd Architecture

Containerd is organized into several layers:

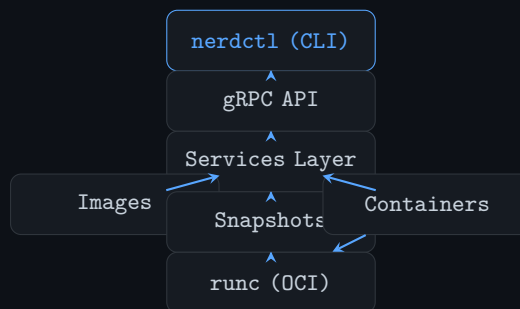


Figure 15.1: Containerd layered architecture. nerdctl communicates via gRPC with the containerd daemon, which manages images, snapshots, and container processes through runc.

15.1.1 gRPC API Layer

Containerd exposes a gRPC API on a Unix domain socket at `/run/containerd/containerd.sock`. The API is organized into several services:

- **Namespace service:** Create, list, remove namespaces. - **Image service:** Pull, push, list, tag, remove images. - **Container service:** Create, start, stop, delete containers. - **Task service:** Attach to container processes, read logs. - **Snapshotter service:** Manage filesystem snapshots for image layers. - **Diff service:** Create and apply layer diffs.

15.1.2 Client Libraries

Clients can interact with containerd via:

- **ctr:** The containerd-native CLI (minimal, debugging). - **nerdctl:** Docker-compatible CLI (user-friendly). - **Go client:** github.com/containerd/containerd Go package for programmatic access. - **containerd-shim:** Shim process that sits between containerd and runc, enabling live-upgrade of containerd without stopping containers.

15.2 Image Management

15.2.1 Image Pull Flow

When a user runs `nerdctl pull alpine:latest`:

1. `nerdctl` sends a `PullImage` gRPC request to `containerd`.
2. `containerd` resolves the image reference (e.g., `docker.io/library/alpine:latest`) through configured registries.
3. `containerd` authenticates with the registry (if credentials are configured).
4. `containerd` downloads the image manifest (JSON) which lists the configuration and layer blobs.
5. For each layer blob, `containerd` verifies the digest (SHA256), downloads the compressed blob, and decompresses it.
6. The snapshotter creates a new snapshot for each layer.
7. The image is registered in `containerd`'s content store and metadata database.

15.2.2 Image Storage

Images are stored in two locations:

- **Content store:** `/mnt/state/containerd/blobs/`. Blobs are stored by content hash (SHA256). Each blob is a subdirectory named by the first two hex characters of the hash.
- **Metadata:** `/mnt/state/containerd/metadata/`. BoltDB database tracking image references, layer metadata, and container state.

15.3 Container Lifecycle

15.3.1 Creating a Container

```
1 nerdctl run -d --name web nginx:alpine
```

Steps:

1. `containerd` creates a new container object in the metadata store.
2. The snapshotter creates a writable snapshot on top of the image layers (using OverlayFS).
3. `containerd` generates an OCI specification (`config.json`) that describes the container's rootfs, mounts, environment variables, command, resource limits, and security settings.
4. `containerd` starts the `containerd-shim` process, passing the container ID and bundle path.
5. The shim launches `runc` with the OCI bundle, creating the container's namespaces, cgroups, and process.

15.3.2 Running Containers

```
1 nerdctl ps                # List running containers
2 nerdctl exec -it web sh    # Execute command in container
3 nerdctl logs web          # View container logs
4 nerdctl stats              # Resource usage
5 nerdctl stop web          # Graceful stop (SIGTERM)
6 nerdctl rm web            # Remove container
```

15.4 Containerd Configuration

```
1 # /etc/containerd/config.toml
2 version = 2
3 root = "/mnt/state/containerd"
4 state = "/run/containerd"
5
6 [grpc]
7   address = "/run/containerd/containerd.sock"
8
9 [plugins]
10  [plugins."io.containerd.grpc.v1.cri"]
11    sandbox_image = "registry.k8s.io/pause:3.9"
12  [plugins."io.containerd.snapshotter.v1.overlayfs"]
13    root_path = "/mnt/state/containerd/snapshots"
```

15.5 Integration with Veilbox

Containerd is started in rcS:

```
1 containerd -c /etc/containerd/config.toml &
2 sleep 2 # Wait for gRPC socket
3 nerdctl pull alpine:latest &
```

The gRPC socket at `/run/containerd/containerd.sock` is available for local clients. No network port is exposed (containerd listens on a Unix socket only).

15.6 Containerd Namespaces

Containerd supports multiple namespaces for tenant isolation:

```
1 # List namespaces
2 ctr namespaces ls
3
4 # Create a namespace
5 ctr namespaces create tenant1
6
7 # Run container in namespace
8 nerdctl --namespace tenant1 run alpine echo hello
9
10 # View containers by namespace
11 nerdctl --namespace tenant1 ps -a
12 nerdctl --namespace k8s.io ps -a # Kubernetes namespace
```

15.7 Image Management Internals

15.7.1 Image Pull

When pulling an image, containerd performs these steps:

```
1 # Trace image pull
2 ctr images pull docker.io/library/alpine:latest
3
4 # Show image metadata
5 ctr images ls -q
6
7 # Export image to tar
```

```

8  ctr images export alpine.tar docker.io/library/alpine:latest
9
10 # Import image from tar
11 ctr images import alpine.tar

```

15.7.2 Content Addressable Storage

Images are stored using content-addressable storage (CAS). Each blob is identified by its SHA256 digest:

```

1 # Content store location
2 ls /mnt/state/containerd/blobs/sha256/
3
4 # View blob metadata
5 cat /mnt/state/containerd/blobs/sha256/<hash>
6
7 # Check content store size
8 du -sh /mnt/state/containerd/

```

15.8 Snapshotter Plugins

Containerd supports multiple snapshotter backends:

Snapshotter	Description	Performance
overlayfs	Native OverlayFS (recommended)	Best
native	Copy-on-write via hard links	Worst
devmapper	Device-mapper thin provisioning	Medium
btrfs	Btrfs snapshots	Good
fuse-overlayfs	FUSE-based OverlayFS	Slow
nydus	Nydus snapshotter (lazy pull)	Fast pull

Veilbox uses overlayfs by default, which provides the best performance for container workloads.

15.9 Containerd Event Monitoring

Containerd emits events for monitoring and automation:

```

1 # Stream all events (in real time)
2 ctr events
3
4 # Filter events by topic
5 ctr events | grep "image/pull"
6
7 # List recent events (from log)
8 grep containerd /var/log/messages | tail -20

```


15.10 Resource Requirements

Containerd in Veilbox requires:

- **Disk space:** Minimum 256 MB free on state partition for image pulls. Recommended: 1+ GB for multiple images.
- **Memory:** Approximately 30 MB for the containerd daemon itself. Container memory usage depends on the workload.
- **File descriptors:** Containerd may use 100+ file descriptors when managing multiple containers.

Container resource limits are set per-container:

```
1 # Memory limit
2 nerdctl run --memory=256m nginx:alpine
3
4 # CPU limit
5 nerdctl run --cpus=0.5 nginx:alpine
6
7 # PIDs limit
8 nerdctl run --pids-limit=100 nginx:alpine
9
10 # Block I/O limit
11 nerdctl run --blkio-weight=500 nginx:alpine
```

15.11 Logging Driver

Containerd supports configurable logging:

```
1 # Configuring log rotation in config.toml
2 [plugins."io.containerd.grpc.v1.cri".containerd]
3   [plugins."io.containerd.grpc.v1.cri".containerd.default_runtime]
4     runtime_type = "io.containerd.runc.v2"
5
6 [plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc]
7   runtime_type = "io.containerd.runc.v2"
8
9 [plugins."io.containerd.grpc.v1".cni]
10   bin_dir = "/opt/cni/bin"
11   conf_dir = "/etc/cni/net.d"
```


Chapter 16

Dropbear SSH

Dropbear is a lightweight SSH server designed for embedded systems and environments with limited resources. It provides encrypted remote access to the Veilbox shell, supporting both password and public key authentication.

16.1 Dropbear Overview

Dropbear was created by Matt Johnston in 2002 as a minimal SSH server for embedded Linux systems. Key characteristics:

- **Size:** Approximately 140 KB for the server binary (vs. OpenSSH's 1.2 MB). - **Protocol:** SSH protocol version 2 only (no legacy v1 support). - **Ciphers:** AES-128/256-CTR, AES-128/256-GCM, ChaCha20-Poly1305, 3DES (optional). - **Key exchange:** Curve25519, ECDH (NIST P-256/384/521), DH group14-sha256, DH group1-sha1. - **Host key types:** ED25519, RSA (2048/4096), ECDSA (P-256/384/521). - **Authentication:** Password, public key (single `authorized_keys` file), no PAM or Kerberos support.

16.2 Building Dropbear

Veilbox builds Dropbear from source with minimal dependencies:

```
1 # Download
2 curl -LO https://matt.ucc.asn.au/dropbear/releases/\
3 dropbear-2024.85.tar.bz2
4
5 # Extract and configure
6 tar xf dropbear-2024.85.tar.bz2
7 cd dropbear-2024.85
8 ./configure --disable-zlib
9
10 # Build (statically linked)
11 make -j$(nproc) PROGRAMS="dropbear dropbearkey dropbearconvert"
12
13 # Install
14 make install DESTDIR=../rootfs
```

The `--disable-zlib` flag removes zlib compression support, reducing the binary size at the cost of higher network bandwidth.

16.3 Running Dropbear

```

1 # Generate host keys (if not present)
2 dropbearkey -t ed25519 -f /etc/dropbear/dropbear_ed25519_host_key
3 dropbearkey -t rsa -f /etc/dropbear/dropbear_rsa_host_key
4
5 # Start in foreground
6 dropbear -F -R -p 22
7
8 # Start in background
9 dropbear -p 22

```

Command-line options:

Flag	Description
-F	Run in foreground (do not daemonize).
-R	Generate host keys automatically if missing.
-p port	Listen on specified port (default: 22).
-r keyfile	Specify a host key file (repeatable).
-s	Disable password authentication (key-only).
-w	Disallow root login (DISABLED in Veilbox).
-g	Disable password login for all users.
-E	Log to stderr instead of syslog.

16.4 Authentication

16.4.1 Password Authentication

Root password authentication is configured via `/etc/shadow`. The password hash is generated with SHA-512:

```

1 # Generate SHA-512 password hash
2 openssl passwd -6 veiladmin
3 # Output: $6$<salt>$<hash>

```

16.4.2 Public Key Authentication

Public keys are stored in `/etc/dropbear/authorized_keys`. Each line contains one key in standard SSH format:

```

1 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOqyY...

```

16.5 Client Connection

```

1 # Key-based authentication
2 ssh -i output/ssh-test-key root@localhost -p 2222
3
4 # Password authentication
5 ssh root@localhost -p 2222
6 # Password: veiladmin

```

First connect warning (expected):

```

1 The authenticity of host '[localhost]:2222' can't be established.

```

```

2 ED25519 key fingerprint is SHA256:...
3 Are you sure you want to continue connecting (yes/no/[fingerprint])?

```

16.6 Key Exchange Algorithms

Dropbear supports multiple key exchange (KEX) algorithms, negotiated during the SSH handshake:

Algorithm	Security Level	Speed
curve25519-sha256	128-bit equivalent	Fast
ecdh-sha2-nistp256	128-bit	Medium
ecdh-sha2-nistp384	192-bit	Medium
ecdh-sha2-nistp521	256-bit	Slow
diffie-hellman-group14-sha256	128-bit	Slow

16.7 Encryption Ciphers

Supported symmetric ciphers for data encryption:

Cipher	Key Size	Mode	Speed
aes256-ctr	256 bits	CTR	Fast
aes256-gcm	256 bits	GCM	Fast (with AES-NI)
aes128-ctr	128 bits	CTR	Fast
aes128-gcm	128 bits	GCM	Fast (with AES-NI)
chacha20-poly1305	256 bits	AEAD	Very fast

16.8 Host Key Management

Host keys identify the server to clients. On first connection, the client stores the host key fingerprint for verification:

```

1 # Generate ED25519 host key (recommended)
2 dropbearkey -t ed25519 -f /etc/dropbear/dropbear_ed25519_host_key
3
4 # Generate RSA fallback host key
5 dropbearkey -t rsa -f /etc/dropbear/dropbear_rsa_host_key \
6   -s 2048
7
8 # View public key fingerprint
9 dropbearkey -y -f /etc/dropbear/dropbear_ed25519_host_key
10
11 # Remove old host key (run on client)
12 ssh-keygen -R '[localhost]:2222'

```

16.9 Troubleshooting Dropbear

16.9.1 Debug Mode

Run Dropbear in foreground with verbose logging:

```
1 dropbear -F -E -p 22
2 # -F: foreground, -E: log to stderr
```

16.9.2 Common Connection Issues

- **Connection refused:** Dropbear is not running. Check with `ps | grep dropbear`.
- **No matching host key type:** The client and server do not share a common host key algorithm. Ensure ED25519 host key exists.
- **Software caused connection abort:** The QEMU port forwarding is misconfigured. Verify `hostfwd=tcp::2222-:22` in the QEMU command line.
- **Permission denied:** Password authentication failed. Verify the password hash in `/etc/shadow` is correct and that `-w` flag is NOT set (which disables root login).

Chapter 17

Network Stack

Veilbox uses a minimal but functional network stack built on BusyBox networking utilities and the Linux kernel's networking subsystem. This chapter covers the network configuration, DHCP, routing, and integration with container networking.

17.1 Network Architecture

The Veilbox network stack has four layers:

1. **Physical/virtual device:** VirtIO-NET (virtio_net driver) provides the network interface to QEMU's virtual switch.
2. **Link layer:** The kernel's networking core manages the Ethernet device (`eth0`) via the VirtIO-NET driver.
3. **Network layer:** IPv4 protocol stack with ARP, ICMP, and routing.
4. **Application layer:** udhcpd (DHCP client) and user applications (wget, nerdctl image pulls, ssh, etc.).

17.2 Interface Configuration

17.2.1 Loopback Interface

The loopback interface provides local communication:

```
1 ifconfig lo 127.0.0.1 up
```

17.2.2 Ethernet Interface

The primary Ethernet interface (`eth0`) acquires an IP address via DHCP:

```
1 ifconfig eth0 0.0.0.0 up
2 udhcpd -i eth0 -b
```

17.3 DHCP Client (udhcpd)

The BusyBox DHCP client is a minimal implementation of the DHCP protocol:

17.3.1 Protocol Flow

1. **DHCPDISCOVER**: Client broadcasts a discovery packet to 255.255.255.255 (UDP port 67).
2. **DHCPOFFER**: Server responds with an offered IP address, lease time, subnet mask, gateway, and DNS servers.
3. **DHCPREQUEST**: Client requests the offered address.
4. **DHCPACK**: Server acknowledges the lease.
5. **Bound**: Client configures the interface using the supplied parameters.

17.3.2 Lease Renewal

At 50% of lease time, the client attempts to renew. On success, the renew script is called. On failure, the client tries again at 87.5%.

17.4 Routing

Default route configuration:

```
1 route add default gw 10.0.2.2
```

In QEMU SLiRP networking, the gateway is always 10.0.2.2. DNS is at 10.0.2.3. The guest IP is 10.0.2.15/24.

17.5 Container Networking

17.5.1 veth Pairs

When `nerdctl` creates a container with network access, it creates a veth pair — two virtual Ethernet interfaces connected by a virtual cable:

```
1 # Host side: vethXXXXX in root namespace
2 # Container side: eth0 in container namespace
```

17.5.2 Network Namespace Isolation

Each container receives its own network namespace:

- **Default**: A veth pair connects the container to the host's network. The container shares the host's IP stack (NAT).
- **Host mode** (`-net=host`): The container uses the host's network namespace directly, sharing the same IP address.
- **None** (`-net=none`): The container has only a loopback interface, completely isolated.

17.6 Network Interface Naming

Veilbox uses the traditional kernel naming scheme (`eth0`) rather than the predictable naming scheme (`enp0s3`) used by modern distributions. This is configured via the kernel command line:

```
1 net.ifnames=0
```


17.7 TCP/IP Stack Tuning

The Veilbox kernel includes the following TCP/IP features:

```
1 CONFIG_TCP_CONG_CUBIC=y      # Default congestion control
2 CONFIG_TCP_CONG_BBR=y       # BBR congestion control (optional)
3 CONFIG_IP_MULTICAST=y       # Multicast support
4 CONFIG_IP_PNP=y             # IP autoconfiguration (DHCP)
5 CONFIG_SYN_COOKIES=y        # SYN flood protection
```

17.7.1 Network Performance Tuning

Parameters in `/proc/sys/net/` for tuning:

```
1 # Increase TCP buffer sizes
2 echo 2097152 > /proc/sys/net/core/rmem_max
3 echo 2097152 > /proc/sys/net/core/wmem_max
4
5 # Enable TCP window scaling
6 echo 1 > /proc/sys/net/ipv4/tcp_window_scaling
7
8 # Reduce TIME_WAIT sockets
9 echo 1 > /proc/sys/net/ipv4/tcp_tw_reuse
10
11 # Increase backlog queue
12 echo 1024 > /proc/sys/net/core/somaxconn
```

17.8 Stateless Address Autoconfiguration

In addition to DHCP, the kernel supports IPv6 SLAAC (Stateless Address Autoconfiguration). While Veilbox primarily uses IPv4, the IPv6 stack is available:

```
1 CONFIG_IPV6=y
2 CONFIG_IPV6_ROUTER_PREF=y
3 CONFIG_IPV6_OPTIMISTIC_DAD=y
```

17.9 Conntrack and NAT

Connection tracking is essential for container NAT:

```
1 # View connection tracking table
2 cat /proc/net/nf_conntrack
3
4 # Maximum tracked connections
5 echo 65536 > /proc/sys/net/netfilter/nf_conntrack_max
6
7 # Connection timeout (seconds)
8 echo 600 > /proc/sys/net/netfilter/nf_conntrack_tcp_timeout_established
```

17.10 DNS Resolution

DNS is configured via `/etc/resolv.conf`, written by the DHCP script:

```
1 # /etc/resolv.conf
2 nameserver 10.0.2.3
```

In QEMU SLiRP networking, the DNS server is always at 10.0.2.3, which forwards queries to the host's DNS resolver.

17.11 Advanced DHCP Configuration

17.11.1 Static IP Configuration

If DHCP is not available, configure a static IP:

```
1 # Static IP configuration
2 ifconfig eth0 192.168.1.100 netmask 255.255.255.0 up
3 route add default gw 192.168.1.1
4 echo "nameserver 8.8.8.8" > /etc/resolv.conf
5 echo "nameserver 1.1.1.1" >> /etc/resolv.conf
```

17.11.2 DHCP with Custom Script

The udhcpd default script can be customized:

```
1 #!/bin/sh
2 # Custom DHCP script /usr/share/udhcpd/default.script
3 RESOLV_CONF="/etc/resolv.conf"
4
5 case "$1" in
6     bound|renew)
7         ifconfig $interface $ip netmask $subnet
8         route add default gw $router 2>/dev/null
9
10        # Write DNS servers
11        > $RESOLV_CONF
12        for ns in $dns; do
13            echo "nameserver $ns" >> $RESOLV_CONF
14        done
15
16        # Set hostname if provided
17        [ -n "$hostname" ] && hostname "$hostname"
18
19        # Log the event
20        echo "DHCP: $interface -> $ip via $router" \
21            >> /var/log/messages
22        ;;
23     deconfig)
24         ifconfig $interface 0.0.0.0
25         ;;
26     leasefail)
27         echo "DHCP: lease failed for $interface" \
28             >> /var/log/messages
29         ;;
30 esac
```

17.12 Network Troubleshooting Tools

17.12.1 Ping and Connectivity

```
1 # Test basic connectivity
2 ping -c 3 10.0.2.2      # Gateway (QEMU SLiRP)
3 ping -c 3 10.0.2.3      # DNS server
4 ping -c 3 8.8.8.8       # Internet (if NAT works)
5
6 # Test DNS resolution
7 ping -c 3 google.com
```

17.12.2 Traceroute

BusyBox traceroute shows the network path:

```
1 traceroute 8.8.8.8
```

17.12.3 Tcpdump

For packet-level analysis, tcpdump can be installed in a container:

```
1 nerdctl run --rm --net=host alpine sh -c \
2     "apk add tcpdump && tcpdump -i eth0"
```


Chapter 18

Filesystem Layers

Veilbox implements a multi-layered filesystem strategy that separates the immutable root filesystem from persistent state storage. This chapter documents each filesystem layer and its role.

18.1 Storage Architecture

Veilbox uses three distinct storage tiers:

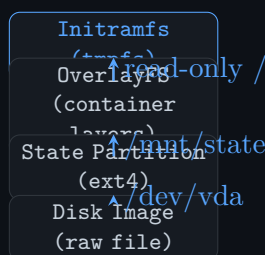


Figure 18.1: Filesystem layering. The initramfs root is read-only at runtime; containers use OverlayFS on top; persistent data lives on the ext4 state partition.

18.2 Layer 1: Initramfs (tmpfs)

The root filesystem is a tmpfs created by the kernel from the embedded cpio archive:

- **Type:** tmpfs (memory-backed, no block device).
- **Contents:** BusyBox, containerd, runc, nerdctl, Dropbear, configuration files.
- **Size:** Approximately 90 MB uncompressed. - **Persistence:** None — every reboot gets a fresh copy from the kernel binary. - **Mount:** `mount -t tmpfs none /`

18.3 Layer 2: State Partition (ext4)

The state partition provides persistent storage across reboots:

- **Location:** `/mnt/state` - **Device:** `/dev/vdb` (external VirtIO disk) or `/dev/sdb` (external SATA disk) or `/state/` on the VEILBOX partition (fallback). - **Filesystem:** ext4. - **Size:** 128 MB (default). - **Contents:** containerd data (images, snapshots, metadata), container logs, SSH host keys, `/var/log/messages`.

18.3.1 Mount Strategy

The rcS script uses a fallback strategy:

```

1 if [ -b /dev/vdb ]; then
2     mount /dev/vdb /mnt/state
3 elif [ -b /dev/sdb ]; then
4     mount /dev/sdb /mnt/state
5 elif [ -d /state ]; then
6     mount --bind /state /mnt/state
7 else
8     mount -t tmpfs none /mnt/state
9 fi

```

18.4 Layer 3: OverlayFS (Container Layers)

Containers use OverlayFS for copy-on-write filesystem layering:

- **Lower dir:** Image layers (read-only snapshots).
- **Upper dir:** Container writable layer (changes since container creation).
- **Merge dir:** The unified view presented to the container process.

18.5 Disk Image Structure

The 8 GB raw disk image (veilbox.raw) uses MBR partitioning:

```

1 Disk veilbox.raw: 8 GiB, 8589934592 bytes, 16777216 sectors
2 Device          Boot Start      End Sectors Size Id Type
3 veilbox.raw1    *        2048 4196351 4194304   2G 83 Linux

```

The partition is formatted as ext4 with the label VEILBOX.

18.6 Layer 4: Container Image Layers

Container images are composed of OCI-compliant layers. Each layer is a tar archive of filesystem changes:

```

1 # Example: nginx image layers
2 Layer 0 (base): alpine:latest -> /bin/sh, /lib, /etc
3 Layer 1:         nginx install -> /usr/sbin/nginx, /etc/nginx
4 Layer 2:         config change -> /etc/nginx/nginx.conf
5 Layer 3 (writable): container -> /var/log/nginx/access.log

```

Each layer is stored as a gzip-compressed tar archive in the containerd content store. The digest (SHA256) uniquely identifies each layer.

18.7 Filesystem Hierarchy Standard (FHS)

Veilbox implements a subset of the Filesystem Hierarchy Standard:

Directory	Purpose	Type
/bin	Essential user binaries	initramfs
/sbin	System binaries	initramfs
/etc	Configuration files	initramfs
/usr/bin	Non-essential binaries	initramfs
/usr/sbin	Non-essential system bins	initramfs

/lib64	Shared libraries	initramfs
/proc	Process information	procfs
/sys	Kernel/device information	sysfs
/dev	Device nodes	devtmpfs
/tmp	Temporary files	tmpfs
/var/log	Log files	state (bind mount)
/run	Runtime data	tmpfs
/mnt/state	Persistent storage	ext4 (external)
/root	Root user home	initramfs

18.8 Debugfs Rootless File Operations

The build script uses `debugfs` to write files to the ext4 filesystem without mounting it. This avoids the need for loop device access:

```

1 # Write a file to the ext4 image
2 debugfs -w output/veilbox.raw <<EOF
3 mkdir /boot
4 write /path/to/vmlinuz /boot/vmlinuz
5 write /path/to/grub.cfg /boot/grub/grub.cfg
6 EOF

```

Key `debugfs` commands:

- `write <host> <target>`: Copy host file to image.
- `mkdir <path>`: Create directory.
- `rm <path>`: Remove file.
- `stat <path>`: Show file metadata.
- `ls -l <path>`: List directory.
- `cd <path>`: Change directory.

Chapter 19

Security Hardening

Security is a primary design goal for Veilbox. The system is hardened at every layer: kernel configuration reduces the attack surface, BusyBox eliminates setuid binaries, Dropbear provides encrypted access, and container runtimes use Linux security primitives.

19.1 Kernel Security Features

19.1.1 Minimal Attack Surface

The kernel is configured without unnecessary subsystems:

- No sound subsystem (CONFIG_SOUND=n).
- No USB stack (CONFIG_USB=n).
- No Wi-Fi or Bluetooth (CONFIG_WLAN=n, CONFIG_BT=n).
- No ATA or SCSI drivers (CONFIG_ATA=n, CONFIG_SCSI=n).
- No filesystems beyond ext4, proc, sysfs, tmpfs, devtmpfs, overlay, squashfs.
- No PCMCIA, FireWire, InfiniBand, or ISDN.

19.1.2 Mandatory Security Options

```
1 CONFIG_SECCOMP=y           # Syscall filtering
2 CONFIG_SECCOMP_FILTER=y    # BPF-based filters
3 CONFIG_SECURITY=y          # Security framework
4 CONFIG_SECURITY_YAMA=y      # Ptrace scope restrictions
5 CONFIG_DEFAULT_SECURITY_YAMA=y
6 CONFIG_SECURITY_NETWORK=y   # Network security hooks
```

19.1.3 CGroup Device Control

CGroup device controllers restrict container access to host devices:

```
1 CONFIG_CGROUP_DEVICE=y
```

19.2 Userspace Security

19.2.1 No Setuid Binaries

The Veilbox root filesystem contains no setuid binaries. BusyBox can be configured with setuid support disabled:

```
1 # BusyBox config: disable setuid
2 # CONFIG_FEATURE_SUID is not set
```

19.2.2 No Package Manager

Without a package manager, there is no mechanism for an attacker to install software, exploit repository vulnerabilities, or run post-install scripts.

19.2.3 No Compilers

No compilers or interpreters (other than busybox ash) are present in the root filesystem. An attacker who gains shell access cannot compile exploit code.

19.2.4 Read-Only Root

The root filesystem is embedded in the kernel binary. Runtime modification of system binaries is impossible — changes are lost on reboot.

19.2.5 Minimal Services

Only two network services run: Dropbear (SSH) and containerd (local Unix socket only). No HTTP, FTP, SMTP, NFS, SMB, or any other services.

19.3 SSH Hardening

- **ED25519 keys:** Modern, fast, secure host keys.
- **PBKDF2 password hash:** /etc/shadow uses SHA-512 rounds.
- **Port forwarding:** QEMU limits SSH to port forwarding only (no public guest IP by default).
- **Key-based auth:** Optional but supported with auto-generated ED25519 key pair.

19.4 Container Security Primitives

19.4.1 Namespace Isolation

Each container runs in isolated namespaces:

- **PID:** Container processes cannot see host processes.
- **Network:** Container has its own network stack.
- **Mount:** Container has its own filesystem hierarchy.
- **UTS:** Container has its own hostname.
- **IPC:** Container has its own IPC resources.
- **User:** (Optional) Container root can be mapped to an unprivileged host user.

19.4.2 Seccomp

Seccomp-BPF filters restrict the system calls available to container processes. The default Docker/containerd seccomp profile allows approximately 300 of the 400+ Linux syscalls, blocking dangerous ones like `module_init`, `kexec_load`, and `ptrace`.

19.4.3 Capability Dropping

Linux capabilities are dropped from container processes:

```
1 CAP_SYS_ADMIN      # Mount, namespace operations
2 CAP_NET_ADMIN      # Network configuration
3 CAP_SYS_MODULE     # Kernel module loading
4 CAP_SYS_RAWIO      # I/O port access
5 CAP_SYS_PTRACE     # Process tracing
```

19.5 Container Security Architecture

19.5.1 Defense in Depth

Veilbox implements multiple layers of container security:

1. **Namespaces:** Each container has isolated views of the system (processes, network, mounts, etc.).
2. **Cgroups:** Resource limits prevent denial-of-service from a single container.
3. **Seccomp-BPF:** System call filtering blocks dangerous syscalls.
4. **Capability dropping:** Removes privileges like `CAP_SYS_ADMIN` and `CAP_NET_ADMIN`.
5. **Read-only rootfs:** Containers can be started with read-only root filesystems.
6. **No new privileges:** The `NO_NEW_PRIVS` flag prevents privilege escalation via `setuid` binaries.

19.5.2 Seccomp Default Profile

The default seccomp profile blocks approximately 60 system calls that are dangerous or unnecessary for containers:

```
1 # Blocked syscalls (partial list)
2 kexec_load      # Loading a different kernel
3 open_by_handle_at # Bypass mount namespace
4 init_module     # Loading kernel modules
5 finit_module    # Loading kernel modules
6 delete_module   # Unloading kernel modules
7 ptrace          # Process tracing and debugging
8 swapon          # Enabling swap
9 swapoff         # Disabling swap
10 sethostname     # Changing hostname
11 setdomainname   # Changing domain name
```

19.5.3 SELinux Considerations

Veilbox does not use SELinux (it adds complexity and binary size). For systems requiring SELinux:

```
1 CONFIG_SECURITY_SELINUX=y
2 CONFIG_DEFAULT_SECURITY_SELINUX=y
```

However, this requires SELinux policies and tools which add approximately 10 MB to the `initramfs`.

19.6 Audit Trail

The system log captures security-relevant events:

```
1 # Monitor authentication attempts
2 grep "dropbear" /var/log/messages
3
4 # Monitor container starts/stops
5 grep "containerd" /var/log/messages
6
7 # Monitor failed login attempts
8 grep "Failed" /var/log/messages
```

19.7 Network Security

19.7.1 Firewall Rules

Basic iptables rules for container isolation:

```
1 # Default: drop incoming, allow outgoing
2 iptables -P INPUT DROP
3 iptables -P FORWARD DROP
4 iptables -P OUTPUT ACCEPT
5
6 # Allow loopback
7 iptables -A INPUT -i lo -j ACCEPT
8
9 # Allow established connections
10 iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
11
12 # Allow SSH
13 iptables -A INPUT -p tcp --dport 22 -j ACCEPT
14
15 # Allow container traffic to external
16 iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Chapter 20

Performance Tuning

This chapter covers performance optimization for Veilbox — both for the build process and for runtime operation in virtualized environments.

20.1 Build Performance

20.1.1 Parallel Compilation

The kernel build is the most time-consuming step (approximately 4–5 minutes on a 4-core machine):

```
1 # Set number of parallel jobs
2 make -j$(nproc) bzImage
```

20.1.2 CCache Integration

CCache can dramatically speed up repeated builds:

```
1 export CC="ccache gcc"
2 export CXX="ccache g++"
3 make -j$(nproc) bzImage
4 # First build: slow. Subsequent builds: 50-70% faster.
```

20.1.3 Incremental Builds

After the first full build, configuration changes rebuild only affected files:

```
1 # Changing one kernel config option:
2 make olddefconfig
3 make -j$(nproc) bzImage
4 # Typical time: 30-60 seconds
```

20.2 Runtime Performance

20.2.1 KVM Acceleration

QEMU with KVM enables near-native performance:

```

1 # Check KVM availability
2 ls -la /dev/kvm
3 groups # Must include 'kvm'
4
5 # QEMU with KVM
6 qemu-system-x86_64 -enable-kvm ...

```

20.2.2 VirtIO Drivers

VirtIO paravirtualized drivers reduce I/O overhead:

Device	Emulated Alternative	VirtIO Advantage
Block	IDE (piix3-ide)	3-5x throughput, 50% less CPU
Network	e1000 / rtl8139	5-10x throughput, 30% less CPU
Balloon	None	Dynamic memory management

20.2.3 Memory Optimization

- **Minimum RAM:** 2 GB. The initramfs occupies 90 MB, containerd uses 30 MB, kernel uses 50 MB. - **Transparent Huge Pages:** Enable on host for better memory performance. - **Memory overcommit:** Enabled by default for container workloads that may overallocate.

20.3 Kernel Boot Time Optimization

20.3.1 Measured Boot Time

```

1 # From QEMU start to login prompt:
2 # Direct kernel boot: <10 seconds
3 # GRUB BIOS boot: <15 seconds

```

20.3.2 Reducing Boot Time

- **Minimize kernel size:** The stripped-down Veilbox kernel is approximately 90 MB (with embedded initramfs). Uncompressed parts are proportional to boot time. - **Serial vs. graphical:** Serial console (`-nographic`) is faster because no framebuffer initialization is needed. - **Pre-built initramfs:** Skip `gen_initramfs_list` step by caching the file list.

20.4 Container Performance

- **OverlayFS:** Better performance than aufs or device-mapper snapshotters. Contains native kernel support. - **noexec /tmp:** Mount /tmp with noexec to prevent container escape via writable temp directories. - **ulimit tuning:** Adjust container resource limits with `nerdctl run -ulimit`:

```

1 nerdctl run --ulimit nofile=1024:2048 alpine sh
2 nerdctl run --memory=256m --cpus=0.5 nginx:alpine

```

20.5 Kernel Boot Parameters for Performance

20.5.1 mitigations=off

Disabling CPU vulnerability mitigations can improve performance significantly in trusted VM environments:

```
1 # Add to kernel command line:
2 mitigations=off
```

This disables:

- Spectre v2 (IBPB, IBRS, STIBP)
- Meltdown (PTI/KPTI)
- Spectre v1 (user pointer sanitization)
- Speculative Store Bypass (SSBD)
- L1TF (L1D flush)
- MDS (microarchitectural data sampling)

In a VM where the host is trusted, these mitigations are unnecessary. Performance gain: 5-15% on CPU-bound workloads.

20.5.2 Scalability Options

```
1 # Add to kernel command line:
2 clocksource=tsc tsc=reliable
```

The TSC (Time Stamp Counter) is the fastest available clocksource. Setting it as the default reduces timer overhead.

20.6 I/O Performance

20.6.1 VirtIO-BLK with io_uring

The Linux kernel's io_uring subsystem provides high-performance async I/O. It can be used by containerd for image pulls:

```
1 CONFIG_IO_URING=y
```

20.6.2 Block Device Tuning

```
1 # Increase read-ahead buffer
2 echo 4096 > /sys/block/vda/queue/read_ahead_kb
3
4 # Set I/O scheduler (none for VirtIO)
5 echo none > /sys/block/vda/queue/scheduler
6
7 # Increase max sectors per request
8 echo 2048 > /sys/block/vda/queue/max_sectors_kb
```

20.7 Memory Performance

20.7.1 Transparent Huge Pages

THP reduces TLB misses by using 2 MB pages instead of 4 KB:

```
1 # Enable always (for consistent performance)
2 echo always > /sys/kernel/mm/transparent_hugepage/enabled
3
4 # Defragment when needed
5 echo defer > /sys/kernel/mm/transparent_hugepage/defrag
```

20.7.2 Swap

Swap is not used in Veilbox. The state partition is too small, and container workloads should fit in RAM:

```
1 CONFIG_SWAP=n # Disable swap support in kernel
```

20.8 Network Performance

20.8.1 Tx Queue Length

```
1 # Increase transmit queue length
2 echo 10000 > /sys/class/net/eth0/tx_queue_len
```

20.8.2 IRQ Affinity

Pin network IRQs to specific CPUs:

```
1 # Find IRQ number for eth0
2 cat /proc/interrupts | grep eth0
3
4 # Set CPU affinity (CPU 0)
5 echo 1 > /proc/irq/<N>/smp_affinity
```


Chapter 21

Container Workloads

This chapter demonstrates practical container workloads running on Veilbox. These examples validate that the container runtime works correctly and provide reference configurations for common use cases.

21.1 Prerequisites

All examples assume:

- Veilbox is booted and the root shell is available (via console or SSH).
- containerd is running (verified with `ps | grep containerd`).
- Networking is functional (verified with `ping 10.0.2.2`).
- The state partition is mounted at `/mnt/state`.

21.2 Example 1: Hello World

The simplest container test:

```
1 nerdctl pull alpine:latest
2 nerdctl run alpine:latest echo "Hello from Veilbox!"
```

Expected output:

```
1 Hello from Veilbox!
```

21.3 Example 2: Interactive Shell

Run an interactive Alpine shell:

```
1 nerdctl run -it alpine:latest sh
2 / # apk add --no-cache curl
3 / # curl https://example.com
4 / # exit
```

21.4 Example 3: Web Server

Run nginx with host port mapping:

```
1 nerdctl run -d \  
2   --name web \  
3   -p 80:80 \  
4   nginx:alpine  
5  
6 # Test from inside Veilbox:  
7 wget -q -O- http://localhost  
8  
9 # Test from host (via QEMU port forwarding):  
10 curl http://localhost:8080/
```

21.5 Example 4: Persistent Data

Use a bind mount for persistent container data:

```
1 # Create data directory on state partition  
2 mkdir -p /mnt/state/data  
3  
4 # Run container with bind mount  
5 nerdctl run -d \  
6   --name web-data \  
7   -v /mnt/state/data:/usr/share/nginx/html:ro \  
8   nginx:alpine  
9  
10 # Write data from host  
11 echo "Persistent" > /mnt/state/data/index.html
```

21.6 Example 5: Resource Limits

Test resource constraints:

```
1 # Memory limit  
2 nerdctl run --memory=64m alpine sh -c \  
3   "dd if=/dev/zero of=/dev/null bs=1M count=100"  
4  
5 # CPU limit (0.5 cores)  
6 nerdctl run --cpus=0.5 alpine sh -c \  
7   "dd if=/dev/zero of=/dev/null bs=1M count=1000"
```

21.7 Example 6: Multi-Container

Start multiple containers and verify isolation:

```
1 # Start two HTTP servers  
2 nerdctl run -d --name web1 nginx:alpine  
3 nerdctl run -d --name web2 nginx:alpine  
4  
5 # They have separate network stacks  
6 nerdctl exec web1 ip addr show eth0  
7 nerdctl exec web2 ip addr show eth0  
8  
9 # They can communicate via the host bridge  
10 nerdctl exec web1 ping -c 1 $(nerdctl inspect web2 | grep IPAddress)
```

21.8 Troubleshooting Containers

Common issues:

```
1 nerdctl logs <container>      # View stdout/stderr
2 nerdctl inspect <container>   # Detailed status
3 nerdctl stats                  # Resource usage
4 nerdctl top <container>       # Running processes
5 nerdctl events                 # Stream container events
```

21.9 Example 7: Database Container

Run MariaDB with persistent storage:

```
1 # Create data directory
2 mkdir -p /mnt/state/mariadb
3
4 # Run MariaDB container
5 nerdctl run -d \
6     --name mariadb \
7     -v /mnt/state/mariadb:/var/lib/mysql \
8     -e MARIADB_ROOT_PASSWORD=example \
9     -p 3306:3306 \
10    mariadb:10.11
11
12 # Connect from another container
13 nerdctl run -it --rm mariadb:10.11 \
14    mysql -h 10.0.2.15 -u root -pexample
```

21.10 Example 8: Python Web App

Deploy a Python Flask application:

```
1 # Create app directory
2 mkdir -p /mnt/state/app
3
4 # Write a simple Flask app
5 cat > /mnt/state/app/app.py << 'EOF'
6 from flask import Flask
7 app = Flask(__name__)
8
9 @app.route('/')
10 def hello():
11     return "Hello from Veilbox Container!"
12
13 if __name__ == '__main__':
14     app.run(host='0.0.0.0', port=5000)
15 EOF
16
17 # Run with Python image
18 nerdctl run -d \
19     --name flask-app \
20     -v /mnt/state/app:/app \
21     -p 5000:5000 \
22     python:3.12-slim \
23     python /app/app.py
24
25 # Test
```

```
26 wget -q -O- http://localhost:5000
```

21.11 Example 9: Resource Monitoring

Run a monitoring container with host metrics:

```
1 # Run a stress test container
2 nerdctl run -d --name stress --memory=128m alpine \
3   sh -c "while true; do dd if=/dev/zero of=/dev/null bs=1M count=100; done"
4
5 # Monitor from host
6 while true; do
7   clear
8   echo "=== Container Stats ==="
9   nerdctl stats --no-stream
10  echo "=== Host Memory ==="
11  free -m
12  echo "=== Host CPU ==="
13  top -bn1 | head -5
14  sleep 2
15 done
```

21.12 Example 10: Network Test

Test container networking with iperf:

```
1 # Start iperf server
2 nerdctl run -d --name iperf-server -p 5201:5201 \
3   alpine sh -c "apk add iperf3 && iperf3 -s"
4
5 # Run iperf client from another container
6 nerdctl run --rm alpine sh -c \
7   "apk add iperf3 && iperf3 -c 10.0.2.15"
```

21.13 Example 11: Cleanup

Remove all containers and images:

```
1 # Stop all running containers
2 nerdctl stop $(nerdctl ps -q) 2>/dev/null
3
4 # Remove all containers
5 nerdctl rm $(nerdctl ps -aq) 2>/dev/null
6
7 # Remove all images
8 nerdctl rmi $(nerdctl images -q) 2>/dev/null
9
10 # Clean containerd content store
11 rm -rf /mnt/state/containerd/*
```

Chapter 22

Comparison with Other Approaches

Veilbox occupies a unique niche in the Linux distribution landscape. This chapter compares Veilbox with other approaches to building minimal Linux systems for container hosting.

22.1 Comparison Overview

Feature	Veilbox	Alpine	Buildroot	Fedora CoreOS
Kernel source	Custom	Mainline	Custom	Mainline
Init system	BusyBox init	OpenRC	BusyBox init / systemd	systemd
Package manager	None	apk	None	rpm-ostree
Rootfs size	90 MB	50 MB	20 MB	1 GB
Build time	5 min	N/A	15 min	N/A
Persistent storage	State partition	Full disk	Full disk	Full disk
Container run-time	containerd	Docker / containerd	Docker	podman
Config language	Bash & Kconfig	n/a	Kconfig	Ignition
Minimum RAM	2 GB	128 MB	64 MB	1 GB

22.2 Veilbox vs. Alpine Linux

Alpine Linux is the most popular minimal container host OS:

- **Similarities:** Both use BusyBox for core utilities. Both aim for minimal footprint. Both target container workloads. - **Differences:** Alpine uses OpenRC init (more features), has the apk package manager, uses musl libc (smaller but not always compatible), and is a general-purpose distribution with hundreds of packages. Veilbox has no package manager, uses glibc, and is single-purpose. - **Trade-off:** Alpine is more flexible (install packages at runtime). Veilbox is more secure (no runtime installation possible) and more deterministic (every deployment is identical).

22.3 Veilbox vs. Buildroot

Buildroot is a build system for embedded Linux:

- **Similarities:** Both build a custom kernel with embedded rootfs. Both support BusyBox. Both produce minimal images.

- **Differences:** Buildroot has a sophisticated package selection system, supports hundreds of packages, and targets embedded hardware (ARM, MIPS, RISC-V). Veilbox is x86_64-only with a fixed set of packages.
- **Trade-off:** Buildroot is more powerful but requires learning its configuration system and has a longer initial build. Veilbox is simpler to configure (just edit build.sh).

22.4 Veilbox vs. Fedora CoreOS

Fedora CoreOS is Red Hat's minimal container OS:

- **Similarities:** Both target container workloads. Both use containerd (or podman). Both support automated updates.
- **Differences:** CoreOS uses systemd (complex), Ignition for provisioning (YAML-based), rpm-ostree for atomic updates, and has SELinux enabled by default. It is supported by Red Hat. Veilbox is independently built, uses BusyBox init, and has no update mechanism.
- **Trade-off:** CoreOS is enterprise-ready with support and updates. Veilbox is simpler (no systemd, no SELinux) and easier to understand entirely.

22.5 When to Use Veilbox

Veilbox is ideal for:

- **Learning:** Understanding every component in a Linux system.
- **CI/CD:** Disposable VMs for testing containers.
- **Education:** Teaching kernel compilation, boot process, and init systems.
- **Benchmarking:** Minimal overhead for container performance testing.

22.6 Detailed Feature Matrix

Feature	Veilbox	Typical Distro
Init system	BusyBox init	systemd / OpenRC
Process supervisor	None (busybox init)	systemd / supervise-daemon
Service files	Shell scripts	systemd unit files
Dependency-based boot	No	Yes
Parallel boot	No	Yes
Socket activation	No	Yes
Timer units	No	Yes
Logging	syslogd /var/log/messages	+ journald
User session management	No	logind
Tmpfiles	No	systemd-tmpfiles
Sysctl	Manual (/proc/sys)	systemd-sysctl
Mount units	No	systemd-mount
Timer units	No	systemd-timer

22.7 Size Comparison Breakdown

Component	Veilbox	Alpine	Ubuntu Server
-----------	---------	--------	---------------

Kernel	10 MB	8 MB	12 MB
Initramfs	80 MB	15 MB	50 MB
Rootfs (installed)	N/A (embedded)	50 MB	2 GB
BusyBox	1.2 MB	1.2 MB	N/A
coreutils	N/A (BusyBox)	N/A (BusyBox)	15 MB
systemd	N/A	N/A	25 MB
OpenRC	N/A	2 MB	N/A
Package manager	None	apk (3 MB)	apt (20 MB)
Container runtime	containerd (35 MB)	Docker (100 MB)	Docker (100 MB)
Total	90 MB	180 MB	2.5 GB

22.8 Use Case Recommendations

- **Veilbox:** Learning, CI/CD VMs, container testing, security research, embedded container hosts.
- **Alpine Linux:** Production container hosts, Docker hosts, general-purpose embedded systems, routers, firewalls.
- **Buildroot:** Custom embedded devices, IoT, specialized hardware, minimal appliances.
- **Fedora CoreOS:** Enterprise container hosts, Kubernetes nodes, automated updates, SELinux environments.
- **Ubuntu Server:** General-purpose servers, traditional deployments, environments requiring package managers.

Chapter 23

Tutorials

This chapter provides step-by-step tutorials for common tasks with Veilbox, from first build to advanced container operations.

23.1 Tutorial 1: First Build

Goal: Build the Veilbox disk image from scratch.

1. Clone the repository:

```
1 git clone https://github.com/Shreyas0047/veilbox.git
2 cd veilbox
```

2. Check prerequisites:

```
1 gcc --version          # Need 14+
2 make --version         # Need 4.4+
3 qemu-system-x86_64 --version # Need 7.0+
```

3. Run the build:

```
1 ./build.sh
2 # Full build: ~5 minutes
3 # Output: output/veilbox.raw, output/veilbox.vdi, output/vmlinuz
```

4. Boot in QEMU:

```
1 MEM=2G ./test.sh
```

5. When you see the login prompt, log in:

```
1 veilbox login: root
2 Password: veiladmin
```

23.2 Tutorial 2: Quick Rebuild

Goal: Modify a configuration file and rebuild quickly.

1. Edit the rcS script:

```
1 vim rootfs/etc/init.d/rcS
```

2. Rebuild initramfs and kernel:

```
1 # Regenerate initramfs file list
2 ./build.sh --stage initramfs
3
4 # Rebuild kernel with updated initramfs
5 ./build.sh --stage kernel
6
7 # Rebuild disk image
8 ./build.sh --stage disk
```

3. Boot with updated image:

```
1 MEM=2G ./test.sh
```

23.3 Tutorial 3: SSH Access

Goal: Connect to Veilbox via SSH from the host.

1. Boot Veilbox with SSH port forwarding:

```
1 MEM=2G ./test.sh
```

2. From another terminal, connect:

```
1 ssh -i output/ssh-test-key root@localhost -p 2222
```

3. Or with password:

```
1 ssh root@localhost -p 2222
2 # Password: veiladmin
```

23.4 Tutorial 4: Run a Container

Goal: Pull and run an nginx container.

1. Log in to Veilbox (console or SSH).
2. Check containerd is running:

```
1 ps | grep containerd
```

3. Pull an image:

```
1 nerdctl pull nginx:alpine
2 nerdctl images
```

4. Run the container:

```
1 nerdctl run -d --name web -p 80:80 nginx:alpine
2 nerdctl ps
```

5. Test from Veilbox:

```
1 wget -q -O- http://localhost
```

23.5 Tutorial 5: Persistent Storage

Goal: Create a container with persistent data.

1. Verify state partition is mounted:

```
1 df -h /mnt/state
```

2. Create a data directory:

```
1 mkdir -p /mnt/state/data
```

3. Write data:

```
1 echo "Hello from state" > /mnt/state/data/hello.txt
```

4. Run a container with the data mounted:

```
1 nerdctl run --rm -v /mnt/state/data:/data alpine \  
2 cat /data/hello.txt
```

5. Reboot and verify persistence:

```
1 MEM=2G ./test.sh --keep-state  
2 # After login:  
3 cat /mnt/state/data/hello.txt
```

23.6 Tutorial 6: Custom Kernel Configuration

Goal: Add a new kernel feature and rebuild.

1. Enter kernel configuration:

```
1 cd linux-7.1.0-rc6  
2 make menuconfig
```

2. Navigate to the desired feature. For example, enable BPF: **General setup** → **BPF subsystem**.

3. Save the configuration:

```
1 # Save as custom fragment  
2 grep -E "^CONFIG_BPF" .config > ../kernel/configs/bpf.config
```

4. Merge into main config:

```
1 ./scripts/kconfig/merge_config.sh \  
2 -O .config \  
3 .config \  
4 ../kernel/configs/bpf.config  
5 make olddefconfig
```

5. Rebuild:

```
1 make -j$(nproc) bzImage  
2 cp arch/x86/boot/bzImage ../output/vmlinuz
```

6. Test the new feature:

```
1 # If BPF was added:
2 mount -t bpf none /sys/fs/bpf
3 bpftool prog list
```

23.7 Tutorial 7: Debugging Build Failures

Goal: Diagnose and fix a common kernel build error.

1. Run the build:

```
1 make -j$(nproc) bzImage 2>&1 | tee build.log
```

2. Search for errors:

```
1 grep -i error build.log | head -20
```

3. Common fixes:

- **implicit declaration:** Missing header or wrong GCC version.
- **undefined reference:** Missing library or wrong config.
- **too many sections:** Disable CONFIG_DEBUG_INFO.
- **disk full:** Free space in /tmp or build directory.

4. If the error is config-related, use:

```
1 make olddefconfig      # Fix dependency issues
2 make menuconfig        # Manually fix options
```

5. Clean and retry:

```
1 make clean
2 make -j$(nproc) bzImage
```

23.8 Tutorial 8: Booting from USB

Goal: Write Veilbox to a USB drive for bare-metal boot.

1. Insert a USB drive (8 GB or larger).
2. Identify the device:

```
1 lsblk
2 # Look for your USB device, e.g., /dev/sdX
```

3. Write the disk image:

```
1 sudo dd if=output/veilbox.raw of=/dev/sdX bs=4M status=progress
2 sync
```

4. Boot from the USB drive on the target machine (configure BIOS boot order to boot from USB).
5. Log in with root / veiladmin.

23.9 Tutorial 9: Creating Custom Container Images

Goal: Build a custom container image with nerdctl.

1. Create a Dockerfile:

```
1 mkdir /mnt/state/myapp
2 cat > /mnt/state/myapp/Dockerfile << 'EOF'
3 FROM alpine:latest
4 RUN apk add --no-cache python3 py3-pip
5 COPY app.py /app.py
6 CMD ["python3", "/app.py"]
7 EOF
```

2. Build the image:

```
1 nerdctl build -t myapp:latest /mnt/state/myapp
```

3. Run the image:

```
1 nerdctl run -d --name myapp -p 5000:5000 myapp:latest
```

4. Push to a registry (optional):

```
1 nerdctl tag myapp:latest registry.example.com/myapp:latest
2 nerdctl push registry.example.com/myapp:latest
```


Chapter 24

Reference

This chapter provides quick-reference material for day-to-day Veilbox operation, including command cheat sheets, configuration file summaries, and troubleshooting guides.

24.1 Command Cheat Sheet

24.1.1 Build Commands

Command	Description
<code>./build.sh</code>	Full build (all 14 stages).
<code>./build.sh -clean</code>	Clean all build artifacts.
<code>./build.sh -stage kernel</code>	Build kernel only (incremental).
<code>./build.sh -stage disk</code>	Build disk image only.
<code>./build.sh -stage initramfs</code>	Regenerate initramfs file list.
<code>MEM=2G ./test.sh</code>	Boot in QEMU (direct kernel).
<code>./test.sh -bios</code>	Boot via SeaBIOS + GRUB.
<code>./test.sh -check</code>	Auto health check (exit 0 on login).
<code>MEM=2G ./test.sh -keep-state</code>	Boot with persistent state disk.

24.1.2 SSH Commands

Command	Description
<code>ssh -i output/ssh-test-key root@localhost -p 2222</code>	Key auth.
<code>ssh root@localhost -p 2222</code>	Password auth (veiladmin).
<code>ssh-keygen -R '[localhost]:2222'</code>	Remove old host key.

24.1.3 Container Commands

Command	Description
<code>nerdctl images</code>	List pulled images.
<code>nerdctl pull <image></code>	Pull an image.
<code>nerdctl run <image> <cmd></code>	Run a container.
<code>nerdctl run -it <image> sh</code>	Interactive shell.

<code>nerdctl run -d --name <n> <image></code>	Detached container.
<code>nerdctl ps</code>	List running containers.
<code>nerdctl ps -a</code>	List all containers.
<code>nerdctl stop <id></code>	Stop container.
<code>nerdctl rm <id></code>	Remove container.
<code>nerdctl exec <id> <cmd></code>	Exec in running container.
<code>nerdctl logs <id></code>	View container logs.
<code>nerdctl stats</code>	Resource usage.
<code>nerdctl inspect <id></code>	Detailed info.

24.1.4 System Commands

Command	Description
<code>ps</code>	List processes.
<code>ifconfig eth0</code>	Show IP address.
<code>netstat -tlnp</code>	Show listening ports.
<code>df -h</code>	Show disk usage.
<code>free -m</code>	Show memory usage.
<code>mount</code>	Show mounted filesystems.
<code>cat /var/log/messages</code>	View system log.
<code>poweroff</code>	Shutdown.
<code>reboot</code>	Reboot.

24.2 Configuration File Reference

File	Format	Purpose
<code>/etc/inittab</code>	BusyBox init	Spawn getty, run rcS, shutdown.
<code>/etc/init.d/rcS</code>	Shell script	System initialization.
<code>/etc/passwd</code>	Colon-delimited	User accounts.
<code>/etc/shadow</code>	Colon-delimited	Password hashes.
<code>/etc/hostname</code>	Plain text	System hostname.
<code>/etc/containerd/config.toml</code>	TOML	Container runtime config.
<code>/root/.profile</code>	Shell script	User login prompt.
<code>/usr/share/udhcpd/default.conf</code>	Shell script	DHCP configuration.

24.3 Kernel Config Key Reference

Option	Value	Purpose
<code>CONFIG_NAMESPACES</code>	y	Container isolation.
<code>CONFIG_CGROUPS</code>	y	Resource limits.
<code>CONFIG_OVERLAY_FS</code>	y	Image layers.
<code>CONFIG_VETH</code>	y	Container networking.
<code>CONFIG_BRIDGE</code>	y	Network bridge.
<code>CONFIG_VIRTIO_NET</code>	y	VirtIO NIC.
<code>CONFIG_VIRTIO_BLK</code>	y	VirtIO disk.
<code>CONFIG_EXT4_FS</code>	y	Root filesystem.

CONFIG_INTRAMFS_SOURCE	"/mnt/intramfs.list"	Embedded rootfs.
CONFIG_EFI_STUB	y	QEMU -kernel boot.
CONFIG_SERIAL_8250_CONSOLE	y	Serial console.
CONFIG_SECCOMP	y	Syscall filtering.
CONFIG_DEVTMPFS	y	Device node creation.

24.4 BusyBox Applet Reference

Category	Applets	Notes
Shell	ash, sh	POSIX shell (no bashisms).
File	ls, cp, mv, rm, mkdir, rmdir, cat, chmod, chown, ln, find, grep, sed, awk	Core utils.
Process	ps, kill, pgrep, pkill, top	Process management.
Network	ifconfig, route, ping, netstat, udhcpc, wget	Networking.
Init	init, getty, login, passwd	System init.
Editor	vi	Minimal vi.
Archive	tar, gzip, gunzip, bzip2, bunzip2, xz, unxz, cpio	Archive tools.
Misc	echo, printf, test, true, false, sleep, timeout, env, set, export, read, exit	Shell builtins.

24.5 Kernel Parameter Reference

Parameter	Format
console	console=device,options
root	root=/dev/device
init	init=/path/to/init
quiet	(flag, reduces boot messages)
debug	(flag, maximum verbosity)
single	(flag, single-user mode)
panic	panic=seconds
veilbox.autologin	(flag, custom Veilbox auto-login)
net.ifnames	net.ifnames=0 (legacy ethX names)
mitigations	mitigations=off (disable CPU vuln fixes)
clocksource	clocksource=tsc (use TSC timer)

24.6 Build Script Environment Variables

Variable	Default	Purpose
----------	---------	---------

MEM	2G	Guest RAM for QEMU
OUTPUT	output/	Build output directory
ROOTFS	rootfs/	Root filesystem source
KERNEL_SRC	linux-7.1.0-rc6/	Kernel source directory
CACHE_DIR	\$HOME/veilbox-cache	Download cache
VERBOSE	0	Enable verbose build output
SKIP_DEPS	0	Skip dependency check

24.7 Port Reference

Port	Protocol	Service
22	TCP	SSH (Dropbear) — guest internal
2222	TCP	SSH (QEMU forwarded from host)
80	TCP	HTTP (container web servers)
1338	TCP	Containerd metrics (localhost only)
3306	TCP	MariaDB/MySQL (container)
5000	TCP	Flask/Python apps (container)
5201	TCP	iperf3 (container network test)

24.8 File System Layout Reference

```

1 # Complete filesystem hierarchy
2 /          # initramfs root (tmpfs)
3 |----- bin/      # BusyBox applets
4 |----- sbin/     # System binaries (init, getty)
5 |----- etc/      # Configuration
6 |   |----- inittab
7 |   |----- init.d/rcS
8 |   |----- passwd
9 |   |----- shadow
10 |  |----- hostname
11 |  |----- resolv.conf
12 |  '----- dropbear/
13 |      |----- authorized_keys
14 |      |----- dropbear_ed25519_host_key
15 |      '----- dropbear_rsa_host_key
16 |----- usr/
17 |   |----- bin/      # containerd, nerdctl, dropbear, dropbearkey
18 |   '----- sbin/
19 |----- lib64/      # Shared libraries (glibc, libseccomp)
20 |----- proc/       # procfs
21 |----- sys/        # sysfs
22 |----- dev/        # devtmpfs
23 |----- tmp/        # tmpfs
24 |----- run/        # tmpfs (runtime data)
25 |----- var/
26 |   '----- log/
27 |       '----- messages # syslog output
28 |----- mnt/
29 |   '----- state/    # Persistent state (ext4 or tmpfs fallback)
30 '----- root/       # Root user home
31   '----- .profile

```

24.9 Quick Commands Reference

Action	Command
Build everything	<code>./build.sh</code>
Clean build	<code>./build.sh -clean</code>
Boot in QEMU	<code>MEM=2G ./test.sh</code>
Boot with state	<code>./test.sh -keep-state</code>
Auto health check	<code>./test.sh -check</code>
GRUB BIOS boot	<code>./test.sh -bios</code>
SSH connect	<code>ssh -i output/ssh-test-key root@localhost -p 2222</code>
Kernel menuconfig	<code>cd linux-7.1.0-rc6 && make menuconfig</code>
Check logs	<code>cat /var/log/messages</code>
Free memory	<code>free -m</code>
Disk usage	<code>df -h</code>
List processes	<code>ps</code>
Network config	<code>ifconfig eth0</code>
Routing table	<code>route -n</code>

Appendix A

Appendix: Source Code Listings

This appendix contains the complete source code for the key scripts and configuration files that make up the Veilbox build system. All code is reproduced from `build.sh`, `test.sh`, and the `rootfs/` directory.

A.1 build.sh

```
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
5  OUTPUT_DIR="${OUTPUT_DIR:-$SCRIPT_DIR/output}"
6  ROOTFS_DIR="$SCRIPT_DIR/rootfs"
7  KERNEL_SRC="$SCRIPT_DIR"
8  KERNEL_CONFIG_FRAG="$SCRIPT_DIR/kernel/configs/custom-os.config"
9
10 ARCH="${ARCH:-x86_64}"
11 CROSS_COMPILE="${CROSS_COMPILE:-}"
12
13 BUSYBOX_VERSION="1.35.0"
14 CONTAINERD_VERSION="1.7.13"
15 RUNC_VERSION="1.1.12"
16 NERDCTL_VERSION="1.7.5"
17 DROPBEAR_VERSION="2024.85"
18
19 BUSYBOX_URL="https://busybox.net/downloads/binaries/${BUSYBOX_VERSION}-x86_64
    -linux-musl/busybox"
20 CONTAINERD_URL="https://github.com/containerd/containerd/releases/download/v$
    {CONTAINERD_VERSION}/containerd-${CONTAINERD_VERSION}-linux-amd64.tar.gz"
21 RUNC_URL="https://github.com/opencontainers/runc/releases/download/v${
    RUNC_VERSION}/runc.amd64"
22 NERDCTL_URL="https://github.com/containerd/nerdctl/releases/download/v${
    NERDCTL_VERSION}/nerdctl-${NERDCTL_VERSION}-linux-amd64.tar.gz"
23 DROPBEAR_URL="https://matt.ucc.asn.au/dropbear/releases/dropbear-${
    DROPBEAR_VERSION}.tar.bz2"
24
25 mkdir -p "$OUTPUT_DIR"
26
27 RED='\033[1;31m'; GREEN='\033[1;32m'; YELLOW='\033[1;33m'; BLUE='\033[1;34m';
    NC='\033[0m'
28 info() { echo -e "${BLUE}[INFO]${NC} $*" >&2; }
29 ok() { echo -e "${GREEN}[OK]${NC} $*" >&2; }
30 warn() { echo -e "${YELLOW}[WARN]${NC} $*" >&2; }
31 err() { echo -e "${RED}[ERROR]${NC} $*" >&2; exit 1; }
32
```

```

33 # Parse command line arguments
34 CLEAN=0
35 for arg in "$@"; do
36     case "$arg" in
37         --clean|-c) CLEAN=1 ;;
38         --help|-h)
39             echo "Usage: $0 [--clean] [--help]"
40             echo ""
41             echo "Build the Veilbox kernel and images."
42             echo ""
43             echo "Options:"
44             echo "  --clean, -c  Remove all build artifacts and rebuild from
                    scratch"
45             echo "  --help, -h   Show this help message"
46             exit 0
47             ;;
48         esac
49     done
50
51 require_tool() {
52     if ! command -v "$1" &>/dev/null; then
53         err "'$1' not found in PATH. Install with: dnf install $2"
54     fi
55 }
56
57 require_file() {
58     if [ ! -f "$1" ]; then
59         err "Required file not found: $1"
60     fi
61 }
62
63 # ----- 0. Install dependencies (Fedora) -----
64 install_deps() {
65     if [ ! -f /etc/fedora-release ]; then
66         warn "Not running Fedora - skipping package install. Ensure build
              dependencies are installed."
67         return
68     fi
69
70     info "Detected Fedora - checking packages..."
71     if sudo -n true 2>/dev/null; then
72         sudo dnf install -y \
73             squashfs-tools e2fsprogs wget tar goolang \
74             qemu-system-x86 qemu-img \
75             containerd nerdctl dropbear busybox-static \
76             kernel-devel gcc make flex bison openssl-devel \
77             elfutils-libelf-devel ncurses-devel bc rsync \
78             xz bzip2 grub2-tools grub2-pc-modules 2>&1 | tail -3
79         ok "Fedora packages installed"
80     else
81         warn "Sudo requires a password - skipping automated package install."
82         warn "If tools are missing, run: sudo dnf install -y squashfs-tools
              e2fsprogs wget tar goolang qemu-system-x86 qemu-img containerd
              nerdctl dropbear busybox-static kernel-devel gcc make flex bison
              openssl-devel elfutils-libelf-devel ncurses-devel bc rsync xz
              bzip2 grub2-tools grub2-pc-modules"
83     fi
84 }
85
86 # ----- 1. Download a static binary with fallback -----
87 download_binary() {
88     local dest="$1" url="$2" sysbin="$3" name="$4"
89

```

```

90     mkdir -p "${dirname "$dest"}"
91
92     if [ -f "$dest" ]; then
93         info "$name already exists at $dest, skipping."
94         return 0
95     fi
96
97     info "Downloading $name..."
98     if wget -q --timeout=30 "$url" -O "$dest" 2>/dev/null; then
99         chmod +x "$dest"
100         ok "$name downloaded to $dest"
101         return 0
102     fi
103
104     warn "Failed to download $name from $url"
105     rm -f "$dest"
106
107     if [ -n "$sysbin" ] && [ -f "$sysbin" ]; then
108         warn "Falling back to system binary: $sysbin"
109         cp "$sysbin" "$dest" && chmod +x "$dest"
110         ok "$name copied from system: $dest"
111         return 0
112     fi
113
114     err "Cannot obtain $name - download failed and no system fallback at
115         $sysbin"
116 }
117
118 download_tarball() {
119     local url="$1" dest_dir="$2" strip_components="$3" expected_bin="$4" name
120         ="$5"
121
122     if [ -f "$dest_dir/$expected_bin" ]; then
123         info "$name already exists at $dest_dir/$expected_bin, skipping."
124         return 0
125     fi
126
127     info "Downloading $name tarball..."
128     local tmp_tar
129     tmp_tar="$(mktemp)"
130
131     if ! wget -q --timeout=30 "$url" -O "$tmp_tar" 2>/dev/null; then
132         rm -f "$tmp_tar"
133         err "Failed to download $name from $url"
134     fi
135
136     mkdir -p "$dest_dir"
137     tar xf "$tmp_tar" -C "$dest_dir" --strip-components="$strip_components"
138     rm -f "$tmp_tar"
139
140     if [ -f "$dest_dir/$expected_bin" ]; then
141         chmod +x "$dest_dir/$expected_bin"
142         ok "$name extracted to $dest_dir/$expected_bin"
143     else
144         err "$name tarball extracted but expected binary $dest_dir/
145             $expected_bin not found"
146     fi
147 }
148
149 # ----- 2. Generate initramfs file list (with device nodes) -----
150 gen_initramfs_list() {
151     local list_file="$SCRIPT_DIR/initramfs.list"
152     local tmp_list

```

```

150     tmp_list="$(mktemp)"
151
152     info "Generating initramfs file list..."
153
154     rm -f "$list_file"
155
156     # Generate file list from rootfs directory (in subshell to preserve cwd)
157     (
158         cd "$ROOTFS_DIR"
159         find . -print0 | while IFS= read -r -d '' f; do
160             local name="${f#.}"
161             [ -z "$name" ] && name="/"
162             if [ -L "$f" ]; then
163                 echo "slink ${name} $(readlink "$f") 777 0 0"
164             elif [ -f "$f" ]; then
165                 local mode
166                 mode=$(stat -c "0%a" "$f" 2>/dev/null)
167                 echo "file ${name} ${ROOTFS_DIR}${name} ${mode} 0 0"
168             elif [ -d "$f" ]; then
169                 local mode
170                 mode=$(stat -c "0%a" "$f" 2>/dev/null)
171                 echo "dir ${name} ${mode} 0 0"
172             fi
173         done
174     ) > "$tmp_list"
175
176     # Add device nodes
177     cat >> "$tmp_list" << 'DEVLIST'
178     nod /dev/console 622 0 0 c 5 1
179     nod /dev/ttyS0 600 0 0 c 4 64
180     DEVLIST
181
182     # Sort for reproducibility
183     sort -u "$tmp_list" > "$list_file"
184     rm -f "$tmp_list"
185
186     echo "$list_file"
187 }
188
189 # ----- 3. Configure kernel (base options, no initramfs yet) -----
190 configure_kernel() {
191     info "Configuring Linux kernel..."
192
193     cd "$KERNEL_SRC"
194
195     require_file "$KERNEL_CONFIG_FRAG"
196
197     if [ -f .config ]; then
198         local initramfs_newer=1
199         if [ ! -f "$SCRIPT_DIR/initramfs.list" ] || [ ".config" -nt "$SCRIPT_DIR/initramfs.list" ]; then
200             initramfs_newer=0
201         fi
202         if [ ".config" -nt "$KERNEL_CONFIG_FRAG" ] && [ "$initramfs_newer" -eq 0 ]; then
203             info "Using existing .config (use --clean to force reconfigure)"
204             make olddefconfig
205             return
206         fi
207     fi
208
209     make defconfig

```



```

210     scripts/kconfig/merge_config.sh -O "$KERNEL_SRC" -m .config "
        $KERNEL_CONFIG_FRAG"
211     make olddefconfig
212
213     ok "Kernel configured with base options"
214 }
215
216 # ----- 4. Strip unnecessary kernel drivers -----
217 strip_kernel_config() {
218     info "Stripping unnecessary kernel drivers..."
219
220     cd "$KERNEL_SRC"
221
222     local script="$KERNEL_SRC/scripts/config"
223     [ -x "$script" ] || err "scripts/config not found"
224
225     "$script" --disable SOUND
226     "$script" --disable SND
227     "$script" --disable DRM
228     "$script" --disable USB_SUPPORT
229     "$script" --disable WLAN
230     "$script" --disable BT
231     "$script" --disable MEDIA_SUPPORT
232     "$script" --disable VIDEO
233     "$script" --disable HID
234     "$script" --disable INPUT
235     "$script" --disable I2C
236     "$script" --disable SPI
237     "$script" --disable WATCHDOG
238     "$script" --disable ATA
239     "$script" --disable SCSI
240
241     "$script" --refresh
242
243     ok "Kernel size reduced"
244 }
245
246 # ----- 5. Set initramfs source (after rootfs is populated) -----
247 set_initramfs_source() {
248     info "Generating initramfs file list with device nodes..."
249
250     cd "$KERNEL_SRC"
251
252     local list_file
253     list_file="$(gen_initramfs_list)"
254
255     echo 'CONFIG_INITRAMFS_SOURCE="'"$list_file"'"' >> .config
256     make olddefconfig
257
258     ok "Initramfs source set: $list_file"
259 }
260
261 # ----- 6. Build kernel -----
262 build_kernel() {
263     info "Building Linux kernel (this will take a while)..."
264     cd "$KERNEL_SRC"
265     make -j"$(nproc)" CROSS_COMPILE="$CROSS_COMPILE"
266     cp arch/x86/bzImage "$OUTPUT_DIR/vmlinuz" 2>/dev/null || cp arch/x86/boot
        /bzImage "$OUTPUT_DIR/vmlinuz"
267     ok "Kernel built: $OUTPUT_DIR/vmlinuz"
268 }
269
270 # ----- 7. Set up rootfs directory structure -----

```

```

271 setup_rootfs_dirs() {
272     mkdir -p "$ROOTFS_DIR"/{bin,sbin,usr/bin,usr/sbin,etc/init.d,etc/dropbear
        ,etc/containerd,dev,proc,sys,tmp,root,mnt/state,var/run,var/log,lib}
273     mkdir -p "$ROOTFS_DIR/usr/share/udhcp"
274 }
275
276 # ----- 8. Create rootfs config files (survive --clean) -----
277 setup_rootfs_config() {
278     mkdir -p "$ROOTFS_DIR/etc/init.d" "$ROOTFS_DIR/etc/containerd" "
        $ROOTFS_DIR/etc/dropbear"
279
280     # /etc/hostname
281     echo "veilbox" > "$ROOTFS_DIR/etc/hostname"
282
283     # /etc/passwd (root with shell at /bin/sh)
284     echo 'root:x:0:0:root:/root:/bin/sh' > "$ROOTFS_DIR/etc/passwd"
285
286     # /etc/shadow (root with password veiladmin)
287     echo 'root:$5$2rUEXw9y7bh3/HRR$IM2VpUIeZXRC9.Fqpnmkiwo8Hg/aR/KE.GV42xZGLB
        /:20000:0:99999:7:::' > "$ROOTFS_DIR/etc/shadow"
288     chmod 600 "$ROOTFS_DIR/etc/shadow"
289
290     # /etc/inittab
291     cat > "$ROOTFS_DIR/etc/inittab" << 'EOF'
292 ::sysinit:/etc/init.d/rcS
293 ::restart:/sbin/init
294 ::shutdown:/bin/umount -a -r
295 ::shutdown:/sbin/swapoff -a
296
297 tty1::respawn:/sbin/getty -L tty1 115200 vt100
298 ttyS0::respawn:/sbin/getty -L ttyS0 115200 vt100
299 EOF
300
301     # /etc/init.d/rcS
302     cat > "$ROOTFS_DIR/etc/init.d/rcS" << 'EOF'
303 #!/bin/sh
304 mount -t proc none /proc
305 mount -t sysfs none /sys
306 mount -t devtmpfs none /dev
307
308 mkdir -p /dev/pts /dev/shm
309 mount -t devpts none /dev/pts
310 mount -t tmpfs none /dev/shm
311
312 hostname -F /etc/hostname
313
314 mkdir -p /mnt/state
315 mount LABEL=VEILBOX /mnt/state 2>/dev/null || mount -t tmpfs none /mnt/state
316 mkdir -p /mnt/state/containerd /mnt/state/log /mnt/state/volumes
317
318 ip link set lo up
319 ip link set eth0 up 2>/dev/null || true
320
321 /sbin/udhcp -i eth0 -b -q &
322
323 /sbin/syslogd -n &
324
325 mkdir -p /var/run /var/log
326
327 /usr/bin/containerd --config /etc/containerd/config.toml &
328 /usr/sbin/dropbear -p 22
329 EOF
330     chmod 755 "$ROOTFS_DIR/etc/init.d/rcS"

```

```

331
332     # /etc/containerd/config.toml
333     cat > "$ROOTFS_DIR/etc/containerd/config.toml" << 'EOF'
334 root = "/mnt/state/containerd"
335 state = "/run/containerd"
336
337 disabled_plugins = ["cri"]
338
339 [grpc]
340     address = "/run/containerd/containerd.sock"
341
342 [metrics]
343     address = ""
344 EOF
345
346     # /usr/share/udhcpd/default.script
347     cat > "$ROOTFS_DIR/usr/share/udhcpd/default.script" << 'SCRIPT'
348 #!/bin/sh
349 case "$1" in
350     deconfig)
351         ip addr flush dev $interface 2>/dev/null || true
352         ip route del default 2>/dev/null || true
353         ;;
354     bound|renew)
355         ip addr add $ip/${subnet:-24} dev $interface 2>/dev/null || true
356         ip route del default 2>/dev/null || true
357         [ -n "$router" ] && ip route add default via $router dev $interface
358             2>/dev/null || true
359         ;;
360     esac
361 exit 0
362 SCRIPT
363     chmod 755 "$ROOTFS_DIR/usr/share/udhcpd/default.script"
364
365     # /etc/issue (login banner)
366     cat > "$ROOTFS_DIR/etc/issue" << 'EOF'
367 Veilbox v1.0
368 EOF
369
370     # /root/.profile (colored prompt)
371     cat > "$ROOTFS_DIR/root/.profile" << 'PROFILE'
372 export PS1='\[\e[1;31m\]root@veilbox\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\$ '
373 IP=$(ip addr show eth0 2>/dev/null | grep 'inet' | awk '{print $2}' | cut -d
374 / -f1)
375 [ -n "$IP" ] && echo " IP: $IP"
376 PROFILE
377
378     ok "Rootfs config files created"
379 }
380
381 # ----- 9. Populate rootfs with static binaries -----
382 populate_busybox() {
383     download_binary "$ROOTFS_DIR/bin/busybox" "$BUSYBOX_URL" "/usr/bin/
384         busybox" "BusyBox"
385
386     for applet in init mount umount reboot poweroff modprobe getty ip
387         hostname syslogd kill mkdir ln cat echo ls ps sh sleep head grep pidof
388         udhcpd ifconfig nc netstat df tail wc uname login passwd; do
389         ln -sf /bin/busybox "$ROOTFS_DIR/sbin/$applet"
390     done
391     ln -sf /bin/busybox "$ROOTFS_DIR/bin/sh"
392     ln -sf /sbin/init "$ROOTFS_DIR/init"
393     ok "BusyBox symlinks created"

```

```

389 }
390
391 populate_containerd() {
392     local dir="$ROOTFS_DIR/usr/bin"
393     # Clean up stale top-level files from previous tarball extractions
394     rm -f "$ROOTFS_DIR/usr/containerd" "$ROOTFS_DIR/usr/ctr" \
395         "$ROOTFS_DIR/usr/containerd-stress" \
396         "$ROOTFS_DIR/usr/containerd-shim" \
397         "$ROOTFS_DIR/usr/containerd-shim-runc-v1" \
398         "$ROOTFS_DIR/usr/containerd-shim-runc-v2"
399     download_tarball "$CONTAINERD_URL" "$dir" 1 "containerd" "containerd"
400     local shims
401     shims=$(ls "$dir/containerd-shim"* 2>/dev/null || true)
402     for s in $shims; do chmod +x "$s" 2>/dev/null || true; done
403 }
404
405 populate_runc() {
406     download_binary "$ROOTFS_DIR/usr/bin/runc" "$RUNC_URL" "/usr/bin/runc" "
407         runc"
408 }
409
410 populate_nerdctl() {
411     download_tarball "$NERDCTL_URL" "$ROOTFS_DIR/usr/bin" 0 "nerdctl" "
412         nerdctl"
413 }
414
415 build_dropbear() {
416     local dpb_bin="$ROOTFS_DIR/usr/sbin/dropbear"
417     if [ -f "$dpb_bin" ]; then
418         info "Dropbear already exists, skipping."
419         return
420     fi
421
422     info "Installing Dropbear from system (libraries will be copied
423         separately)..."
424     if [ -f /usr/sbin/dropbear ]; then
425         mkdir -p "$ROOTFS_DIR/usr/sbin" "$ROOTFS_DIR/usr/bin"
426         cp /usr/sbin/dropbear "$dpb_bin"
427         chmod +x "$dpb_bin"
428     else
429         err "Dropbear not found at /usr/sbin/dropbear. Install with: dnf
430             install -y dropbear"
431     fi
432
433     # Generate host keys using a locally running dropbearkey
434     info "Generating Dropbear host keys..."
435     if command -v dropbearkey &>/dev/null; then
436         dropbearkey -t rsa -f "$ROOTFS_DIR/etc/dropbear/dropbear_rsa_host_key"
437             " 2>/dev/null || warn "RSA key generation failed"
438         dropbearkey -t ecdsa -f "$ROOTFS_DIR/etc/dropbear/
439             dropbear_ecdsa_host_key" 2>/dev/null || warn "ECDSA key generation
440             failed"
441         dropbearkey -t ed25519 -f "$ROOTFS_DIR/etc/dropbear/
442             dropbear_ed25519_host_key" 2>/dev/null || warn "ED25519 key
443             generation failed"
444     else
445         warn "dropbearkey not found on host; SSH host keys will be generated
446             at first boot"
447     fi
448
449     ok "Dropbear installed from system"
450 }
451

```

```

442 build_ssh_keys() {
443     local key_dir="$ROOTFS_DIR/root/.ssh"
444     mkdir -p "$key_dir"
445     if [ ! -f "$key_dir/id_ed25519" ]; then
446         info "Generating SSH key pair for dropbear..."
447         ssh-keygen -t ed25519 -f "$key_dir/id_ed25519" -N "" -C "root@veilbox"
448         cp "$key_dir/id_ed25519.pub" "$key_dir/authorized_keys"
449         chmod 600 "$key_dir/authorized_keys"
450         chmod 600 "$key_dir/id_ed25519"
451         # Also copy to /etc/dropbear for dropbear's default authorized_keys
452         cp "$key_dir/id_ed25519.pub" "$ROOTFS_DIR/etc/dropbear/authorized_keys"
453         chmod 600 "$ROOTFS_DIR/etc/dropbear/authorized_keys"
454         # Export test key for host use
455         cp "$key_dir/id_ed25519" "$OUTPUT_DIR/ssh-test-key"
456         chmod 600 "$OUTPUT_DIR/ssh-test-key"
457         ok "SSH key pair generated"
458     else
459         info "SSH keys already exist, skipping."
460     fi
461 }
462
463 # ----- 9. Copy shared libraries for dynamic binaries -----
464 copy_libraries() {
465     info "Copying shared libraries for dynamic binaries..."
466
467     rm -rf "$ROOTFS_DIR/lib64"
468     mkdir -p "$ROOTFS_DIR/lib64"
469
470     local search_bins
471     search_bins=$(find "$ROOTFS_DIR" -type f -exec file {} \; | grep '
472         dynamically linked' | cut -d: -f1)
473
474     while IFS= read -r bin; do
475         [ -z "$bin" ] && continue
476
477         # Capture ldd output (avoid pipeline subshell)
478         local ldd_output
479         ldd_output=$(ldd "$bin" 2>/dev/null || true)
480
481         local lib_path
482         while IFS= read -r line; do
483             if echo "$line" | grep -q '=> /'; then
484                 lib_path=$(echo "$line" | sed "s/.*=> //;s/ (0x.*)//")
485             elif echo "$line" | grep -q $'^\t/'; then
486                 lib_path=$(echo "$line" | sed "s/^\t//;s/ (0x.*)//")
487             else
488                 continue
489             fi
490             cp -L "$lib_path" "$ROOTFS_DIR/lib64/" 2>/dev/null || true
491         done <<< "$ldd_output"
492
493         # Copy dynamic linker
494         local interp
495         interp=$(file "$bin" 2>/dev/null | sed "s/.*interpreter //;s/.*//")
496         if [ -n "$interp" ] && [ -f "$interp" ]; then
497             cp -L "$interp" "$ROOTFS_DIR/lib64/" 2>/dev/null || true
498         fi
499     done <<< "$search_bins"

```

```

500     # If ld-linux-x86-64.so.2 was copied as a file, it's correct; no symlink
        needed
501     if [ -L "$ROOTFS_DIR/lib64/ld-linux-x86-64.so.2" ]; then
502         local ld_actual
503         ld_actual=$(find "$ROOTFS_DIR/lib64" -name 'ld-*' -type f ! -name 'ld
            -linux*' 2>/dev/null | head -1)
504         if [ -n "$ld_actual" ]; then
505             ln -sf "$(basename "$ld_actual")" "$ROOTFS_DIR/lib64/ld-linux-x86
                -64.so.2" 2>/dev/null || true
506         fi
507     fi
508
509     ok "Shared libraries copied to rootfs ($(find "$ROOTFS_DIR/lib64" -type f
        | wc -l) files)"
510 }
511
512 # ----- 10. Set rootfs permissions -----
513 set_rootfs_perms() {
514     chmod 755 "$ROOTFS_DIR/etc/init.d/rcS" 2>/dev/null || true
515     chmod 700 "$ROOTFS_DIR/root" 2>/dev/null || true
516     chmod 1777 "$ROOTFS_DIR/tmp" 2>/dev/null || true
517 }
518
519 # ----- 11. Create state image -----
520 create_state_img() {
521     local img="$OUTPUT_DIR/state.img"
522     info "Creating state image (ext4, 128MB)..."
523     rm -f "$img"
524     dd if=/dev/zero of="$img" bs=1M count=128 status=progress
525     mkfs.ext4 -q -F "$img" -L state
526     ok "State image created: $img"
527 }
528
529 # ----- 12. Create rootfs.squashfs -----
530 create_rootfs_squashfs() {
531     local img="$OUTPUT_DIR/rootfs.squashfs"
532     info "Creating SquashFS image (xz compression)..."
533     rm -f "$img"
534     mksquashfs "$ROOTFS_DIR" "$img" -comp xz -all-root -noappend
535     local size
536     size=$(du -h "$img" | cut -f1)
537     ok "SquashFS image created: $img ($size)"
538 }
539
540 # ----- 13. Create bootable disk image with GRUB -----
541 create_bootable_disk() {
542     local raw="$OUTPUT_DIR/veilbox.raw"
543     local img_size="${DISK_SIZE:-8}" # GB
544     local fs_img="$OUTPUT_DIR/.veilbox_fs.tmp"
545
546     info "Creating bootable disk image (${img_size}GB)..."
547     rm -f "$raw" "$fs_img"
548
549     # Step 1: Create standalone ext4 filesystem image at OUTPUT_DIR (for disk
        space)
550     info "Creating ext4 filesystem..."
551     dd if=/dev/zero of="$fs_img" bs=1M count=$((img_size * 1024 - 12)) status
        =progress
552     mkfs.ext4 -F -L VEILBOX "$fs_img" > /dev/null 2>&1
553
554     # Step 2: Populate using debugfs (no root required)
555     info "Populating filesystem with kernel and config..."
556

```

```

557     debugfs -w -R "mkdir /boot" "$fs_img" 2>/dev/null || true
558     debugfs -w -R "mkdir /boot/grub" "$fs_img" 2>/dev/null || true
559     debugfs -w -R "mkdir /state" "$fs_img" 2>/dev/null || true
560
561     debugfs -w -R "write $OUTPUT_DIR/vmlinuz /boot/vmlinuz" "$fs_img" 2>/dev/
562         null || {
563         rm -f "$fs_img"
564         err "Failed to copy kernel into filesystem"
565     }
566
567     local grub_cfg_tmp
568     grub_cfg_tmp="$(mktemp /tmp/veilbox-grubcfg-XXXXXX)"
569     cat > "$grub_cfg_tmp" << 'GRUB'
570 set timeout=3
571 set default=0
572
573 serial --unit=0 --speed=115200
574 terminal_input serial console
575 terminal_output serial console
576
577 menuentry "Veilbox" {
578     search --label --set=root VEILBOX
579     linux /boot/vmlinuz console=tty0 console=ttyS0
580 }
581 GRUB
582 debugfs -w -R "write $grub_cfg_tmp /boot/grub/grub.cfg" "$fs_img" 2>/dev/
583     null || true
584 rm -f "$grub_cfg_tmp"
585
586 # Copy GRUB modules into the filesystem
587 local grub_mod_dir="/usr/lib/grub/i386-pc"
588 if [ -d "$grub_mod_dir" ]; then
589     info "Copying GRUB modules..."
590     for mod in ext2 part_msdos biosdisk configfile search_label serial
591         terminal linux gzio fshelp search; do
592         [ -f "$grub_mod_dir/${mod}.mod" ] && \
593             debugfs -w -R "write $grub_mod_dir/${mod}.mod /boot/grub/${
594                 mod}.mod" "$fs_img" 2>/dev/null || true
595     done
596 fi
597
598 # Step 3: Create partitioned disk image
599 info "Creating disk layout..."
600 dd if=/dev/zero of="$raw" bs=1M count=$((img_size * 1024)) status=
601     progress
602 printf 'type=83, bootable\n' | sfdisk "$raw" > /dev/null 2>&1
603
604 # Step 4: Embed filesystem into partition 1 (starts at sector 2048)
605 info "Embedding filesystem into partition..."
606 dd if="$fs_img" of="$raw" bs=512 seek=2048 conv=notrunc status=progress
607 rm -f "$fs_img"
608
609 # Step 5: Install GRUB rootlessly - preserve partition table
610 info "Installing GRUB bootloader..."
611 PYTHON_RAW="$raw" python3 << 'PYGRUB' || err "GRUB installation failed"
612 import subprocess, os, sys, tempfile
613
614 raw_path = os.environ["PYTHON_RAW"]
615 boot_img_path = "/usr/lib/grub/i386-pc/boot.img"
616 core_img_mods = ["ext2", "part_msdos", "biosdisk", "configfile",
617     "search_label", "serial", "terminal", "linux",
618     "gzio", "fshelp", "search"]
619

```

```

615 # Embedded config: search for VEILBOX by label, set prefix
616 with tempfile.NamedTemporaryFile(mode='w', suffix='.cfg', delete=False) as f:
617     f.write('search --label --set=root VEILBOX\n')
618     f.write('set prefix=($root)/boot/grub\n')
619     embedded_cfg = f.name
620
621 # Generate core.img via grub2-mkimage (blocklist already correct)
622 result = subprocess.run(
623     ["grub2-mkimage", "-c", embedded_cfg,
624      "-O", "i386-pc", "-o", "/tmp/veilbox-core.img",
625      "-p", "/boot/grub"] + core_img_mods,
626     capture_output=True
627 )
628 os.unlink(embedded_cfg)
629 if result.returncode != 0:
630     print("grub2-mkimage failed:", result.stderr.decode())
631     sys.exit(1)
632
633 with open("/tmp/veilbox-core.img", "rb") as f:
634     core = f.read()
635 core_sectors = (len(core) + 511) // 512
636 print(f"core.img: {len(core)} bytes = {core_sectors} sectors")
637
638 # Save the partition table (sfdisk wrote it at 0x1BE-0x1FD)
639 with open(raw_path, "rb") as f:
640     raw_mbr = bytearray(f.read(512))
641 partition_table = raw_mbr[0x1BE:0x1FE]
642
643 # Write boot.img to LBA 0 (preserving partition table and 0x55AA)
644 with open(boot_img_path, "rb") as f:
645     boot = f.read(512)
646 with open(raw_path, "r+b") as f:
647     f.write(boot)
648     # Restore partition table and boot signature
649     f.seek(0x1BE)
650     f.write(partition_table)
651     f.write(b'\x55\xAA')
652
653 # Write core.img starting at LBA 1
654 with open(raw_path, "r+b") as f:
655     f.seek(512)
656     f.write(core)
657     pad = core_sectors * 512 - len(core)
658     if pad > 0:
659         f.write(b'\x00' * pad)
660
661 os.unlink("/tmp/veilbox-core.img")
662 print(f"GRUB installed: boot.img at LBA 0, core.img at LBA 1-{core_sectors}")
663 PYGRUB
664
665     ok "Bootable disk created: $raw ($img_size GB)"
666 }
667
668 # ----- 14. Convert raw disk to VDI -----
669 convert_to_vdi() {
670     local raw="$OUTPUT_DIR/veilbox.raw"
671     local vdi="$OUTPUT_DIR/veilbox.vdi"
672
673     if [ ! -f "$raw" ]; then
674         err "Raw disk not found at $raw - run create_bootable_disk first"
675     fi
676
677     info "Converting raw disk to VDI..."

```



```

678     qemu-img convert -f raw -O vdi "$raw" "$vdi"
679     local size
680     size=$(du -h "$vdi" | cut -f1)
681     ok "VDI image created: $vdi ($size)"
682 }
683
684 # ----- Main -----
685 if [ "$CLEAN" -eq 1 ]; then
686     echo "Cleaning build artifacts..."
687     rm -rf "$ROOTFS_DIR" "$OUTPUT_DIR" "$SCRIPT_DIR/initramfs.list" "
        $KERNEL_SRC/.config"
688     echo "Clean complete."
689     echo ""
690 fi
691 mkdir -p "$OUTPUT_DIR"
692
693 echo ""
694 echo "===== "
695 echo "   Veilbox Builder"
696 echo "===== "
697 echo ""
698
699 install_deps
700
701 require_tool make "make"
702 require_tool gcc "gcc"
703 require_tool go "golang"
704 require_tool wget "wget"
705 require_tool mksquashfs "squashfs-tools"
706 require_tool mkfs.ext4 "e2fsprogs"
707 require_tool xz "xz"
708 require_tool bzip2 "bzip2"
709 require_tool dd "coreutils"
710 require_tool python3 "python3"
711 require_tool grub2-mkimage "grub2-tools"
712 require_tool debugfs "e2fsprogs"
713 require_tool sfdisk "util-linux"
714 require_tool qemu-img "qemu-img"
715
716 setup_rootfs_dirs
717 setup_rootfs_config
718 configure_kernel
719 strip_kernel_config
720
721 populate_busybox
722 populate_containerd
723 populate_runc
724 populate_nerdctl
725 build_dropbear
726 build_ssh_keys
727
728 copy_libraries
729 set_rootfs_perms
730
731 set_initramfs_source
732 build_kernel
733 create_rootfs_squashfs
734 create_state_img
735 create_bootable_disk
736 convert_to_vdi
737
738 echo ""
739 echo "===== "

```


A.2 test.sh

```

1  #!/usr/bin/env bash
2  # Veilbox - QEMU test runner (direct kernel boot, state disk via raw image)
3  set -euo pipefail
4
5  SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
6  OUTPUT_DIR="${OUTPUT_DIR:-$SCRIPT_DIR/output}"
7  QEMU_BINARY="${QEMU_BINARY:-qemu-system-x86_64}"
8  MEM="${MEM:-2G}"
9  SMP="${SMP:-2}"
10 SSH_PORT="${SSH_PORT:-2222}"
11 TIMEOUT="${TIMEOUT:-0}"
12 LOG_FILE="${LOG_FILE:-}"
13 CHECK="${CHECK:-}"
14 AUTOLOGIN="${AUTOLOGIN:-}"
15 KEEP_STATE="${KEEP_STATE:-}"
16
17 KERNEL="$OUTPUT_DIR/vmlinuz"
18 DISK_IMG="$OUTPUT_DIR/veilbox.raw"
19 VDI_IMG="$OUTPUT_DIR/veilbox.vdi"
20 STATE_PERSIST="$OUTPUT_DIR/state-persist.img"
21 QEMU_IMG="${QEMU_IMG:-qemu-img}"
22 BIOS="${BIOS:-}"
23 RED='\033[1;31m'; GREEN='\033[1;32m'; YELLOW='\033[1;33m'; BLUE='\033[1;34m';
24   NC='\033[0m'
25
26 usage() {
27     echo "Usage: $0 [options]"
28     echo ""
29     echo "Options:"
30     echo "  --bios           Boot via SeaBIOS/GRUB (disk image) instead of
31                     direct kernel"
32     echo "  --autologin      Auto-login as root on serial console (direct
33                     kernel only)"
34     echo "  --keep-state     Preserve state disk across reboots (external
35                     state-persist.img)"
36     echo "  --timeout SECS  Automatically exit after SECS seconds"
37     echo "  --output FILE    Log serial console output to FILE"
38     echo "  --check          Exit with status 0 if VM boots to login prompt"
39     echo "  --help, -h      Show this help message"
40     echo ""
41     echo "Environment variables: MEM, SMP, SSH_PORT, QEMU_BINARY"
42     exit 0
43 }
44
45 for arg in "$@"; do
46     case "$arg" in
47         --bios)          BIOS=1 ;;
48         --autologin)     AUTOLOGIN=1 ;;
49         --keep-state)    KEEP_STATE=1 ;;
50         --timeout=*)     TIMEOUT="${arg#*=}" ;;
51         --output=*)      LOG_FILE="${arg#*=}" ;;
52         --check)         CHECK=1 ;;
53         --help|-h)       usage ;;
54     esac
55 done
56
57 if [ -z "$BIOS" ] && [ ! -f "$KERNEL" ]; then
58     echo -e "${RED}ERROR${NC}: Kernel not found at $KERNEL"
59     echo "Run ./build.sh first."
60     exit 1

```



```

117 qemu_base=(
118     "$QEMU_BINARY"
119     -drive file="$DISK_IMG",format=raw,if=virtio
120     -m "$MEM"
121     -smp "$SMP"
122     -nographic
123     -no-reboot
124     -cpu host
125     -enable-kvm
126     -netdev user,id=net0,hostfwd=tcp::${SSH_PORT}-:22
127     -device virtio-net,netdev=net0
128 )
129
130 if [ -n "$KEEP_STATE" ]; then
131     qemu_base+=(-drive file="$STATE_PERSIST",format=raw,if=virtio)
132 fi
133
134 if [ -n "$BIOS" ]; then
135     qemu_args=("${qemu_base[@]}" -boot menu=on)
136 else
137     [ -n "$AUTOLOGIN" ] && KERNEL_CMDLINE="$KERNEL_CMDLINE veilbox.autologin"
138     qemu_args=("${qemu_base[@]}" -kernel "$KERNEL" -append "$KERNEL_CMDLINE")
139 fi
140
141 if [ -n "$CHECK" ]; then
142     # Auto-enable autologin for more thorough check
143     AUTOLOGIN=1
144     # BIOS/GRUB mode needs more time for SeaBIOS + GRUB menu countdown
145     [ -n "$BIOS" ] && TIMEOUT=60 || TIMEOUT=45
146     local_log="${LOG_FILE:-$(mktemp /tmp/qemu-serial-XXXXXX)}"
147     [ -z "$LOG_FILE" ] && trap "rm -f '$local_log'" EXIT
148     # Rebuild args with autologin enabled
149     if [ -z "$BIOS" ]; then
150         KERNEL_CMDLINE="$KERNEL_CMDLINE veilbox.autologin"
151         qemu_args=("${qemu_base[@]}" -kernel "$KERNEL" -append "
152             $KERNEL_CMDLINE")
153     fi
154     timeout "$TIMEOUT" "${qemu_args[@]}" > "$local_log" 2>&1 || true
155     if grep -q 'root@veilbox' "$local_log" 2>/dev/null || grep -q 'veilbox
156         login:' "$local_log" 2>/dev/null; then
157         echo -e "${GREEN}[OK]${NC}    VM booted successfully (login prompt
158             detected)"
159         exit 0
160     else
161         echo -e "${RED}[ERROR]${NC} VM failed to reach login prompt"
162         echo "Last 20 lines of serial output:"
163         tail -20 "$local_log" 2>/dev/null || true
164         exit 1
165     fi
166 fi
167
168 if [ -n "$LOG_FILE" ]; then
169     "${qemu_args[@]}" 2>&1 | tee "$LOG_FILE"
170     exit "${PIPESTATUS[0]}"
171 elif [ "$TIMEOUT" -gt 0 ]; then
172     exec timeout "$TIMEOUT" "${qemu_args[@]}"
173 else
174     exec "${qemu_args[@]}"
175 fi

```

A.3 kernel/configs/custom-os.config

```
1 # Custom OS config fragment for container-only host
2 # Merge on top of make tinyconfig via scripts/kconfig/merge_config.sh
3
4 # Networking core
5 CONFIG_NET=y
6
7 # PCI (needed for virtio-pci on x86)
8 CONFIG_PCI=y
9 CONFIG_PCI_MSI=y
10 CONFIG_PCI_DIRECT=y
11 CONFIG_PCI_MMCONFIG=y
12
13 # Virtio bus and transports
14 CONFIG_VIRTIO=y
15 CONFIG_VIRTIO_MENU=y
16 CONFIG_VIRTIO_PCI=y
17 CONFIG_VIRTIO_PCI_LEGACY=y
18
19 # Virtio block driver
20 CONFIG_VIRTIO_BLK=y
21
22 # Virtio network driver
23 CONFIG_VIRTIO_NET=y
24
25 # Block layer
26 CONFIG_BLOCK=y
27 CONFIG_BLK_DEV=y
28
29 # SquashFS filesystem
30 CONFIG_SQUASHFS=y
31 CONFIG_SQUASHFS_FILE_CACHE=y
32 CONFIG_SQUASHFS_DECOMP_SINGLE=y
33 CONFIG_SQUASHFS_ZLIB=y
34
35 # OverlayFS
36 CONFIG_OVERLAY_FS=y
37
38 # tmpfs (required for /tmp, /dev/shm)
39 CONFIG_TMPFS=y
40 CONFIG_SHMEM=y
41
42 # Virtual memory filesystems
43 CONFIG_PROC_FS=y
44 CONFIG_SYSFS=y
45
46 # Device filesystem
47 CONFIG_DEVTMPFS=y
48 CONFIG_DEVTMPFS_MOUNT=y
49
50 # TTY layer (required by serial drivers)
51 CONFIG_TTY=y
52
53 # Serial console (8250 UART)
54 CONFIG_SERIAL_8250=y
55 CONFIG_SERIAL_8250_CONSOLE=y
56 CONFIG_SERIAL_CORE=y
57 CONFIG_SERIAL_CORE_CONSOLE=y
58 CONFIG_SERIAL_EARLYCON=y
59 CONFIG_SERIAL_8250_PCI=y
60
```

```
61 # Control groups (for containerd/runc)
62 CONFIG_CGROUPS=y
63 CONFIG_CGROUP_PIDS=y
64 CONFIG_CGROUP_CPUACCT=y
65 CONFIG_CGROUP_DEVICE=y
66 CONFIG_CGROUP_FREEZER=y
67 CONFIG_CGROUP_SCHED=y
68 CONFIG_CPUSETS=y
69 CONFIG_MEMCG=y
70 CONFIG_CGROUP_BPF=y
71
72 # Namespaces (for containers)
73 CONFIG_NAMESPACES=y
74 CONFIG_UTS_NS=y
75 CONFIG_IPC_NS=y
76 CONFIG_PID_NS=y
77 CONFIG_NET_NS=y
78 CONFIG_USER_NS=y
79
80 # Seccomp
81 CONFIG_SECCOMP=y
82 CONFIG_SECCOMP_FILTER=y
83
84 # Binary formats
85 CONFIG_BINFMT_ELF=y
86 CONFIG_BINFMT_SCRIPT=y
87
88 # Container networking
89 CONFIG_VETH=y
90
91 # ACPI (required by EFI)
92 CONFIG_ACPI=y
93
94 # EFI stub (provides MZ/PE boot header needed for -kernel boot in QEMU)
95 CONFIG_EFI=y
96 CONFIG_EFI_STUB=y
97
98 # Initramfs support
99 CONFIG_BLK_DEV_INITRD=y
100
101 # BPF syscall (needed by containerd for cgroupv2 device)
102 CONFIG_BPF=y
103 CONFIG_BPF_SYSCALL=y
104
105 # Kernel config defaults
106 CONFIG_SYSVIPC=y
107 CONFIG_POSIX_MQUEUE=y
108
109 # 9P networking (optional, for debugging via virtio-9p)
110 CONFIG_NET_9P=y
111 CONFIG_NET_9P_VIRTIO=y
```


A.5 rootfs/etc/init.d/rcS

```
1 #!/bin/sh
2 # Veilbox init script
3 mount -t proc none /proc
4 mount -t sysfs none /sys
5 mount -t devtmpfs none /dev
6 mkdir -p /dev/pts
7 mount -t devpts none /dev/pts
8 mount -t tmpfs none /tmp
9
10 hostname -F /etc/hostname
11
12 # Mount state partition
13 STATE_DEV=""
14 if [ -b /dev/vdb ]; then
15     STATE_DEV=/dev/vdb
16 elif [ -b /dev/sdb ]; then
17     STATE_DEV=/dev/sdb
18 fi
19 if [ -n "$STATE_DEV" ]; then
20     mount "$STATE_DEV" /mnt/state
21 fi
22
23 ifconfig lo 127.0.0.1 up
24 ifconfig eth0 0.0.0.0 up
25 udhcpc -i eth0 -b
26
27 syslogd -O /var/log/messages -s 64
28 containerd -c /etc/containerd/config.toml &
29 dropbear -F -R -p 22 &
```


A.10 GRUB Rootless Installer (Python)

```
1  #!/usr/bin/env python3
2  # Rootless GRUB installer for Veilbox
3  import struct
4  import sys
5
6  def install_grub(disk_path, boot_img_path, core_img_path):
7      with open(boot_img_path, 'rb') as f:
8          boot_img = f.read()
9      with open(core_img_path, 'rb') as f:
10         core_img = f.read()
11
12         with open(disk_path, 'r+b') as f:
13             # Save partition table
14             f.seek(0x1BE)
15             pt = f.read(64)
16
17             # Write boot.img to LBA 0
18             f.seek(0)
19             f.write(boot_img)
20
21             # Restore partition table
22             f.seek(0x1BE)
23             f.write(pt)
24
25             # Write core.img at LBA 1
26             f.seek(512)
27             f.write(core_img)
28
29         print(f"GRUB installed: boot.img ({len(boot_img)}B), "
30               f"core.img ({len(core_img)}B)")
```

A.11 Kernel Config Fragment (custom-os.config)

```
1 # Container runtime support
2 CONFIG_NAMESPACES=y
3 CONFIG_UTS_NS=y
4 CONFIG_IPC_NS=y
5 CONFIG_PID_NS=y
6 CONFIG_NET_NS=y
7 CONFIG_USER_NS=y
8 CONFIG_CGROUPS=y
9 CONFIG_CGROUP_CPUACCT=y
10 CONFIG_CGROUP_DEVICE=y
11 CONFIG_CGROUP_FREEZER=y
12 CONFIG_CGROUP_SCHED=y
13 CONFIG_CPUSETS=y
14 CONFIG_MEMCG=y
15
16 # Overlay filesystem (container layers)
17 CONFIG_OVERLAY_FS=y
18
19 # VirtIO drivers
20 CONFIG_VIRTIO=y
21 CONFIG_VIRTIO_NET=y
22 CONFIG_VIRTIO_BLK=y
23 CONFIG_VIRTIO_CONSOLE=y
24
25 # Filesystems
26 CONFIG_EXT4_FS=y
27 CONFIG_TMPFS=y
28 CONFIG_SQUASHFS=y
29 CONFIG_DEVTMPFS=y
30
31 # Networking
32 CONFIG_VETH=y
33 CONFIG_BRIDGE=y
34 CONFIG_NETFILTER=y
35 CONFIG_NETFILTER_XTABLES=y
36 CONFIG_IP_NF_FILTER=y
37 CONFIG_IP_NF_NAT=y
38 CONFIG_IP_NF_TARGET_MASQUERADE=y
39
40 # Serial console
41 CONFIG_TTY=y
42 CONFIG_SERIAL_8250=y
43 CONFIG_SERIAL_8250_CONSOLE=y
44 CONFIG_SERIAL_CORE=y
45 CONFIG_SERIAL_CORE_CONSOLE=y
46
47 # Boot
48 CONFIG_EFI_STUB=y
49 CONFIG_BLK_DEV_INITRD=y
50 CONFIG_INITRAMFS_SOURCE="initramfs.list"
51 CONFIG_CMDLINE_BOOL=y
52 CONFIG_CMDLINE="console=ttyS0 console=tty1"
53
54 # Security
55 CONFIG_SECCOMP=y
56 CONFIG_SECCOMP_FILTER=y
57 CONFIG_SECURITY=y
58
59 # Disable unnecessary subsystems
60 # CONFIG_SOUND is not set
```

